

ACPI SOURCE LANGUAGE (ASL) REFERENCE

This section formally defines the ACPI Source Language (ASL). ASL is a source language for defining ACPI objects including writing ACPI control methods. OEMs and platform firmware developers define objects and write control methods in ASL and then use a translator tool (compiler) to generate ACPI Machine Language (AML) versions of the control methods. For a formal definition of AML, see the *ACPI Machine Language (AML) Specification* chapter.

AML and ASL are different languages though they are closely related.

Every ACPI-compatible OS must support AML. A given user can define some arbitrary source language (to replace ASL) and write a tool to translate it to AML.

An OEM or platform firmware vendor needs to write ASL and be able to single-step AML for debugging. (Debuggers and similar tools are expected to be AML-level tools, not source-level tools.) An ASL translator implementer must understand how to read ASL and generate AML. An AML interpreter author must understand how to execute AML.

This section has two parts:

- The ASL grammar, which is the formal ASL specification and also serves as a quick reference.
- A full ASL reference, which includes for each ASL operator: the operator invocation syntax, the type of each argument, and a description of the action and use of the operator.

19.1 ASL 2.0 Symbolic Operators and Expressions

For the math and logical operations, ASL supports standard symbolic operators and expressions that are similar to the C language. Compound assignment operators are also supported. The AML code that is generated from the symbolic operators and expressions is identical to the AML code generated for the equivalent legacy ASL operators.

The tables below summarize the ASL 2.0 support for symbolic operators, compared to the legacy ASL equivalent.

Math operators

ASL 2.0 Syntax	Legacy ASL Equivalent
$Z = X + Y$	Add (X, Y, Z)
$Z = X / Y$	Divide (X, Y, , Z)
$Z = X \% Y$	Mod (X, Y, Z)
$Z = X * Y$	Multiply (X, Y, Z)
$Z = X - Y$	Subtract (X, Y, Z)
$Z = X << Y$	ShiftLeft (X, Y, Z)
$Z = X >> Y$	ShiftRight (X, Y, Z)
$Z = X \& Y$	And (X, Y, Z)
$Z = X Y$	Or (X, Y, Z)
$Z = X \wedge Y$	Xor (X, Y, Z)
$Z = \sim X$	Not (X, Z)
$X++$	Increment (X)
$X--$	Decrement (X)

Logical operators

ASL 2.0 Syntax	Legacy ASL Equivalent
$(X == Y)$	LEqual (X, Y)
$(X != Y)$	LNotEqual (X, Y)
$(X < Y)$	LLess (X, Y)
$(X > Y)$	LGreater (X, Y)
$(X <= Y)$	LLessEqual (X, Y)
$(X >= Y)$	LGreaterEqual (X, Y)
$(X \&\& Y)$	LAnd (X, Y)
$(X Y)$	LOr (X, Y)
$!X$	LNot (X)

Assignment and Compound Assignment operations

ASL 2.0 Syntax	Legacy ASL Equivalent
$X = Y$	Store (Y, X)
$X += Y$	Add (X, Y, X)
$X /= Y$	Divide (X, Y, , X)
$X \% = Y$	Mod (X, Y, X)
$X *= Y$	Multiply (X, Y, X)
$X -= Y$	Subtract (X, Y, X)
$X << = Y$	ShiftLeft (X, Y, X)
$X >> = Y$	ShiftRight (X, Y, X)
$X \& = Y$	And (X, Y, X)
$X = Y$	Or (X, Y, X)
$X \wedge = Y$	Xor (X, Y, X)

Miscellaneous

ASL 2.0 Syntax	Legacy ASL Equivalent
$Z = X[Y]$	Index (X, Y, Z)

19.2 ASL Language Grammar

The purpose of this section is to unambiguously state the grammar rules used by the syntax checker of an ASL compiler.

ASL statements declare objects. Each object has three parts, one of which is required and two of which are optional:

Object := ObjectType FixedList VariableList

FixedList refers to a list, of known length, that supplies data that all instances of a given ObjectType must have. A fixed list is written as (a , b , c , ...) where the number of arguments depends on the specific ObjectType, and some elements can be nested objects, that is (a , b , (q , r , s , t) , d). Arguments to a FixedList can have default values, in which case they can be skipped. Thus, (a , c) will cause the default value for the second argument to be used. Some ObjectTypes can have a null FixedList, which is simply omitted. Trailing arguments of some object types can be left out of a fixed list, in which case the default value is used.

VariableList refers to a list, not of predetermined length, of child objects that help define the parent. It is written as { x , y , z , aa , bb , cc } where any argument can be a nested object. ObjectType determines what terms are legal elements of the VariableList. Some ObjectTypes may have a null variable list, which is simply omitted.

Other rules for writing ASL statements are the following:

- Multiple blanks are the same as one. Blank, (,) , ‘ , ’ and newline are all token separators.
- // marks the beginning of a comment, which continues from the // to the end of the line.
- /* marks the beginning of a comment, which continues from the /* to the next */.
- “” (quotes) surround an ASCII string.
- Numeric constants can be written in three ways: ordinary decimal, octal (using 0ddd) or hexadecimal, using the notation 0xdd.
- Nothing indicates an empty item. For example, { Nothing } is equivalent to { }.

19.2.1 ASL Grammar Notation

The notation used to express ASL grammar is specified in the following table.

Table 19.1: ASL Grammar Notation

Notation Convention	Description	Example
Term := Term Term ...	The term to the left of := can be expanded into the sequence of terms on the right.	aterm := bterm cterm means that aterm can be expanded into the two-term sequence of bterm followed by cterm.
Angle brackets (< >)	used to group items.	<a b> <c d> means either a b or c d.
Arrow (=>)	Indicates required run-time reduction of an ASL argument to an AML data type. Means “reduces to” or “evaluates to” at run-time.	“TermArg => Integer” means that the argument must be an ASL TermArg that must resolve to an Integer data type when it is evaluated by an AML interpreter.

continues on next page

Table 19.1 – continued from previous page

Notation Convention	Description	Example
Bar symbol ()	Separates alternatives.	aterm := bterm <cterm dterm> means the following constructs are possible: bterm cterm dterm aterm := <bterm cterm> dterm means the following constructs are possible: bterm dterm cterm dterm
Term Term Term	Terms separated from each other by spaces form an ordered list.	N/A
Word in bold	Denotes the name of a term in the ASL grammar, representing any instance of such a term. ASL terms are not case-sensitive.	In the following ASL term definition: Thermal-Zone (ZoneName) {TermList} the item in bold is the name of the term.
Word in italics	Names of arguments to objects that are replaced for a given instance.	In the following ASL term definition: Thermal-Zone (ZoneName) {TermList} the italicized item is an argument. The item that is not bolded or italicized is defined elsewhere in the ASL grammar.
Single quotes (' ')	Indicate constant characters.	'A'
0xdd	Refers to a byte value expressed as two hexadecimal digits.	0x21 means a value of hexadecimal 21, or decimal 37. Notice that a value expressed in hexadecimal must start with a leading zero (0).
Dash character (-)	Indicates a range.	1-9 means a single digit in the range 1 to 9 inclusive.

19.2.2 ASL Name and Pathname Terms

// Name and path characters supported

```

LeadNameChar :=
    'A'-'Z' | 'a'-'z' | '_'

DigitChar :=
    '0'-'9'

NameChar :=
    DigitChar | LeadNameChar

RootChar :=
    '^'

ParentPrefixChar :=
    '^'

PathSeparatorChar :=
    ';'

CommaChar :=
    ','

```

SemicolonDelimiter :=

Nothing | ‘;’

// Names and paths

NameSeg :=

<*LeadNameChar*> |
 <*LeadNameChar NameChar*> |
 <LeadNameChar NameChar NameChar> |
 <LeadNameChar NameChar NameChar NameChar>

NameString :=

<*RootChar NamePath*> | <*ParentPrefixChar PrefixPath NamePath*> | *NonEmptyNamePath*

NamePath :=

Nothing | <*NameSeg NamePathTail*>

NamePathTail :=

Nothing | <*PathSeparatorChar NameSeg NamePathTail*>

NonEmptyNamePath :=

NameSeg | <*NameSeg NamePathTail*>

PrefixPath :=

Nothing | <*ParentPrefixChar PrefixPath*>

19.2.3 ASL Root and Secondary Terms

// Root Terms

ASLCode :=

DefinitionBlockList

DefinitionBlockList :=

DefinitionBlockTerm | <DefinitionBlockTerm DefinitionBlockList>

// Major Terms

SuperName :=

NameString | *ArgTerm* | *LocalTerm* | *DebugTerm* | *ReferenceTypeOpcode* | *MethodInvocationTerm*

Target :=

Nothing | *SuperName*

TermArg :=

ExpressionOpcode | *DataObject* | *ArgTerm* | *LocalTerm* | *NameString* | SymbolicExpression

MethodInvocationTerm :=

NameString (// NameString => Method
 ArgList
) => Nothing | *DataRefObject*

// List Terms

ArgList :=

Nothing | <*TermArg ArgListTail*>

ArgListTail :=

Nothing | <*CommaChar TermArg ArgListTail*>

ByteList :=
Nothing | *<ByteConstExpr ByteListTail>*

ByteListTail :=
Nothing | *<CommaChar ByteConstExpr ByteListTail>*

DWordList :=
Nothing | *<DWordConstExpr DWordListTail>*

DWordListTail :=
Nothing | *<CommaChar DWordConstExpr DWordListTail>*

ExtendedAccessAttribTerm :=
ExtendedAccessAttribKeyword (
 AccessLength //ByteConst
)

FieldUnitList :=
Nothing | *<FieldUnit FieldUnitListTail>*

FieldUnitListTail :=
Nothing | *<CommaChar FieldUnit FieldUnitListTail>*

FieldUnit :=
FieldUnitEntry | *OffsetTerm* | *AccessAsTerm* | *ConnectionTerm*

FieldUnitEntry :=
<Nothing | NameSeg> CommaChar Integer

PackageList :=
Nothing | *<PackageElement PackageListTail>*

PackageListTail :=
Nothing | *<CommaChar PackageElement PackageListTail>*

PackageElement :=
DataObject | NameString

ParameterTypePackage :=
ObjectTypeKeyword | {Nothing | *ParameterTypePackageList*}

ParameterTypePackageList :=
ObjectTypeKeyword | *<ObjectTypeKeyword CommaChar ParameterTypePackageList>*

ParameterTypesPackage :=
ObjectTypeKeyword | {Nothing | *ParameterTypesPackageList*}

ParameterTypesPackageList :=
ParameterTypePackage | *<ParameterTypePackage CommaChar ParameterTypesPackageList>*

TermList :=
Nothing | *<Term SemicolonDelimiter TermList>*

Term :=
Object | StatementOpcode | ExpressionOpcode | SymbolicExpression

Object :=
CompilerDirective | NamedObject | NameSpaceModifier

// Conditional Execution List Terms

CaseTermList :=
Nothing | *CaseTerm | DefaultTerm DefaultTermList | CaseTerm* CaseTermList

DefaultTermList :=
 Nothing | *CaseTerm* | *CaseTerm* DefaultTermList

IfExistsTerm :=
IfTerm ElseTerm

19.2.4 ASL Data and Constant Terms

// Numeric Value Terms

LeadDigitChar :=
 '1'-'9'

HexDigitChar :=
 DigitChar | 'A'-'F' | 'a'-'f'

OctalDigitChar :=
 '0'-'7'

NullChar :=
 0x00

// Data Terms

BufferData :=
BufferTypeOpcode | *BufferTerm*

ComputationalData :=
BufferData | *IntegerData* | *StringData*

DataObject :=
BufferData | *PackageData* | *IntegerData* | *StringData*

DataRefObject :=
DataObject | *ObjectReference*

IntegerData :=
IntegerTypeOpcode | Integer | *ConstTerm*

PackageData :=
PackageTerm

StringData :=
StringTypeOpcode | *String*

// Integer Terms

Integer :=
DecimalConst | *OctalConst* | *HexConst*

DecimalConst :=
LeadDigitChar | <DecimalConst *DigitChar*>

OctalConst :=
 '0' | <OctalConst *OctalDigitChar*>

HexConst :=
 <0x *HexDigitChar*> | <0X *HexDigitChar*> | <HexConst *HexDigitChar*>

ByteConst :=
 Integer => 0x00-0xFF

WordConst :=
Integer => 0x0000-0xFFFF

DWordConst :=
Integer => 0x00000000-0xFFFFFFFF

QWordConst :=
Integer => 0x0000000000000000-0xFFFFFFFFFFFFFFFF

ByteConstExpr :=
<*IntegerTypeOpcode* | *ConstExprTerm* | Integer> => *ByteConst*

WordConstExpr :=
<*IntegerTypeOpcode* | *ConstExprTerm* | Integer> => *WordConst*

DWordConstExpr :=
<*IntegerTypeOpcode* | *ConstExprTerm* | Integer> => *DWordConst*

QWordConstExpr :=
<*IntegerTypeOpcode* | *ConstExprTerm* | Integer> => *QWordConst*

ConstTerm :=
ConstExprTerm | Revision

ConstExprTerm :=
Zero | One | Ones

// String Terms

String :=
Utf8CharList

Utf8CharList :=
Nothing | <*EscapeSequence* *Utf8CharList*> | <*Utf8Char* *Utf8CharList*>

Utf8Char :=
0x01-0x21 | 0x23-0x5B | 0x5D-0x7F | 0xC2-0xDF 0x80-0xBF | 0xE0 0xA0-0xBF 0x80-0xBF |
0xE1-0xEC 0x80-0xBF 0x80-0xBF | 0xED 0x80-0x9F 0x80-0xBF | 0xEE-0xEF 0x80-0xBF 0x80-
0xBF | 0xF0 0x90-0xBF 0x80-0xBF 0x80-0xBF | 0xF1-0xF3 0x80-0xBF 0x80-0xBF 0x80-0xBF

// Escape sequences

EscapeSequence :=
SimpleEscapeSequence | *OctalEscapeSequence* | *HexEscapeSequence*

HexEscapeSequence :=
\x *HexDigitChar* | \x *HexDigitChar* *HexDigitChar*

SimpleEscapeSequence :=
\' | \' | \a | \b | \f | \n | \r | \t | \v | \

OctalEscapeSequence :=
\ *OctalDigitChar* | \ OctalDigitChar OctalDigitChar | \ OctalDigitChar OctalDigitChar OctalDigitChar

// Miscellaneous Data Type Terms

ObjectReference :=
Integer

Boolean :=
True | False

True :=
Ones

False :=
Zero

// Symbolic Operator terms

Operators :=
'+' | '-' | '*' | '/' | '%' | '&' | '|' | '^' | '~' | '<' | '>' | '!' | '='

CompoundOperators :=
"<<" | ">>" | "++" | "--" | "==" | "!=" | "<=" | ">=" | "&&" | "||" | "+=" | "-=" | "*=" | "/=" | "%=" | "<<=" | ">>=" | "&=" | "|=" | "^="

19.2.5 ASL Opcode Terms

CompilerDirective :=
IncludeTerm | *ExternalTerm*

NamedObject :=
BankFieldTerm | *CreateBitFieldTerm* | *CreateByteFieldTerm* | *CreateDWordFieldTerm* | *CreateFieldTerm* | *CreateQWordFieldTerm* | *CreateWordFieldTerm* | *DataRegionTerm* | *DeviceTerm* | *EventTerm* | *FieldTerm* | *FunctionTerm* | *IndexFieldTerm* | *MethodTerm* | *MutexTerm* | *OpRegionTerm* | *PowerResTerm* | *ProcessorTerm* | *ThermalZoneTerm*

NameSpaceModifier :=
AliasTerm | *NameTerm* | *ScopeTerm*

SymbolicExpressionTerm :=
(*TermArg*) | *AddSymbolicTerm* | *AndSymbolicTerm* | *DecSymbolicTerm* | *DivideSymbolicTerm* | *IncSymbolicTerm* | *LAndSymbolicTerm* | *LEqualSymbolicTerm* | *LGreaterEqualSymbolicTerm* | *LGreaterSymbolicTerm* | *LLessEqualSymbolicTerm* | *LLessSymbolicTerm* | *LNotEqualSymbolicTerm* | *LNotSymbolicTerm* | *LOrSymbolicTerm* | *ModSymbolicTerm* | *MultiplySymbolicTerm* | *NotSymbolicTerm* | *OrSymbolicTerm* | *ShiftLeftSymbolicTerm* | *ShiftRightSymbolicTerm* | *SubtractSymbolicTerm* | *XorSymbolicTerm*

SymbolicAssignmentTerm :=
StoreSymbolicTerm | *AddCompoundTerm* | *AndCompoundTerm* | *DivideCompoundTerm* | *ModCompoundTerm* | *MultiplyCompoundTerm* | *OrCompoundTerm* | *ShiftLeftCompoundTerm* | *ShiftRightCompoundTerm* | *SubtractCompoundTerm* | *XorCompoundTerm*

StatementOpcode :=
BreakTerm | *BreakPointTerm* | *ContinueTerm* | *FatalTerm* | *ForTerm* | *IfElseTerm* | *NoOpTerm* | *NotifyTerm* | *ReleaseTerm* | *ResetTerm* | *ReturnTerm* | *SignalTerm* | *SleepTerm* | *StallTerm* | *SwitchTerm* | *UnloadTerm* | *WhileTerm*

A statement opcode term does not return a value and can only be used standalone on a line of ASL code. Since these opcodes do not return a value, they cannot be used as a term in an expression.

ExpressionOpcode :=
AcquireTerm | *AddTerm* | *AndTerm* | *ConcatTerm* | *ConcatResTerm* | *CondRefOfTerm* | *CopyObjectTerm* | *DecTerm* | *DerefOfTerm* | *DivideTerm* | *FindSetLeftBitTerm* | *FindSetRightBitTerm* | *FprintfTerm* | *FromBCDTerm* | *IncTerm* | *IndexTerm* | *LAndTerm* | *LEqualTerm* | *LGreaterTerm* | *LGreaterEqualTerm* | *LLessTerm* | *LLessEqualTerm* | *LNotTerm* | *LNotEqualTerm* | *LOrTerm* | *MatchTerm* | *MidTerm* | *ModTerm* | *MultiplyTerm* | *NAndTerm* | *NOrTerm* | *NotTerm* | *ObjectTypeTerm* | *OrTerm* | *PrintfTerm* | *RefOfTerm* | *ShiftLeftTerm* | *ShiftRightTerm* | *SizeOfTerm* | *StoreTerm* | *SubtractTerm* | *TimerTerm* | *ToBCDTerm* | *ToBufferTerm* | *ToDecimalStringTerm* | *ToHexStringTerm* |

ToIntegerTerm | ToStringTerm | WaitTerm | XorTerm | MethodInvocationTerm | SymbolicExpressionTerm | SymbolicAssignmentTerm

An expression opcode returns a value and can be used in an expression.

IntegerTypeOpcode :=

AddTerm | AndTerm | DecTerm | DerefOfTerm | DivideTerm | EISAIDTerm | FindSetLeftBitTerm | FindSetRightBitTerm | FromBCDTerm | IncTerm | LAndTerm | LEqualTerm | LGreaterTerm | LGreaterEqualTerm | LLessTerm | LLessEqualTerm | LNotTerm | LNotEqualTerm | MatchTerm | ModTerm | MultiplyTerm | NAndTerm | NOOrTerm | NotTerm | OrTerm | ShiftLeftTerm | ShiftRightTerm | SubtractTerm | ToBCDTerm | ToIntegerTerm | XorTerm | SymbolicExpressionTerm

Integer opcodes are a subset of expression opcodes that return an Integer value and can be used in an expression that evaluates to a constant. These opcodes may be evaluated at ASL compile-time. To ensure that these opcodes will evaluate to a constant, the following rules apply: The term cannot have a destination (target) operand, and must have either an IntegerTypeOpcode, StringTypeOpcode, BufferTypeOpcode, ConstExprTerm, Integer, BufferTerm, Package, or String for all arguments.

StringTypeOpcode :=

ConcatTerm | DerefOfTerm | FprintfTerm | MidTerm | PrintfTerm | ToDecimalStringTerm | ToHexStringTerm | ToStringTerm

String type opcodes are a subset of expression opcodes that return a String value and can be used in an expression that evaluates to a constant. These opcodes may be evaluated at ASL compile-time. To ensure that these opcodes will evaluate to a constant, the following rules apply: The term cannot have a destination (target) operand, and must have either an IntegerTypeOpcode, StringTypeOpcode, BufferTypeOpcode, ConstExprTerm, Integer, BufferTerm, Package, or String for all arguments.

BufferTypeOpcode :=

ConcatTerm | ConcatResTerm | DerefOfTerm | MidTerm | ResourceTemplateTerm | ToBufferTerm | ToPLDTerm | ToUUIDTerm | UnicodeTerm

Buffer type opcodes are a subset of expression opcodes that return a Buffer value and can be used in an expression that evaluates to a constant. These opcodes may be evaluated at ASL compile-time. To ensure that these opcodes will evaluate to a constant, the following rules apply: The term cannot have a destination (target) operand, and must have either an IntegerTypeOpcode, StringTypeOpcode, BufferTypeOpcode, ConstExprTerm, Integer, BufferTerm, Package, or String for all arguments.

ReferenceTypeOpcode :=

RefOfTerm | DerefOfTerm | IndexTerm | IndexSymbolicTerm | UserTermObj

Reference type opcodes are a subset of expression opcodes that return a Reference value and can be used in an expression. They cannot be evaluated at compile time. Reference type also includes the UserTerm, which is a control method invocation.

19.2.6 ASL Primary (Terminal) Terms

AccessAsTerm :=

```
AccessAs (
    AccessType, // AccessTypeKeyword
    AccessAttribute // Nothing | ByteConstExpr | AccessAttribKeyword |
    ExtendedAccessAttribTerm
)
```

AcquireTerm :=

```
Acquire (
```

```

        SyncObject, // SuperName => Mutex
        TimeoutValue // WordConstExpr
    ) => Boolean // True means the operation timed out and the Mutex was not acquired

AddCompoundTerm :=
    Addend1-Result // TermArg => Integer => Target += Addend2 // TermArg => Integer => Integer

AddSymbolicTerm :=
    Addend1 // TermArg => Integer + Addend2 // TermArg => Integer => Integer

AddTerm :=
    Add (
        Addend1, // TermArg => Integer
        Addend2, // TermArg => Integer
        Result // Target
    ) => Integer

AliasTerm :=
    Alias (
        SourceObject, // NameString
        AliasObject // NameString
    )

AndCompoundTerm :=
    Source1-Result // TermArg => Integer => Target
    &=
    Source2 // TermArg => Integer
    => Integer

AndSymbolicTerm :=
    Source1 // TermArg => Integer
    &
    Source2 // TermArg => Integer
    => Integer

AndTerm :=
    And (
        Source1, // TermArg => Integer
        Source2, // TermArg => Integer
        Result // Target
    ) => Integer

ArgTerm :=
    Arg0 | Arg1 | Arg2 | Arg3 | Arg4 | Arg5 | Arg6

BankFieldTerm :=
    BankField (
        RegionName, // NameString => OperationRegion
        BankName, // NameString => FieldUnit
        BankValue, // TermArg => Integer
        AccessType, // AccessTypeKeyword
        LockRule, // LockRuleKeyword
    )

```

```

        UpdateRule // UpdateRuleKeyword
    ) { FieldUnitList }

BreakPointTerm :=
    BreakPoint

BreakTerm :=
    Break

BufferTerm :=
    Buffer (
        BuffSize // Nothing | TermArg => Integer
    ) { StringData | ByteList } => Buffer

CaseTerm :=
    Case (
        Value // DataObject
    ) { TermList }

ConcatResTerm :=
    ConcatenateResTemplate (
        Source1, // TermArg => Buffer
        Source2, // TermArg => Buffer
        Result // Target
    ) => Buffer

ConcatTerm :=
    Concatenate (
        Source1, // TermArg => SuperName
        Source2, // TermArg => SuperName
        Result // Target
    ) => Buffer | String

ConnectionTerm :=
    Connection (
        ConnectionResource // NameString | ResourceMacroTerm
    )

CondRefOfTerm :=
    CondRefOf (
        Source // NameString | ArgTerm | LocalTerm | DerefOfTerm
        Destination // Target
    ) => Boolean

ContinueTerm :=
    Continue

CopyObjectTerm :=
    CopyObject (
        Source, // TermArg => DataRefObject
        Result, // NameString | LocalTerm | ArgTerm
    ) => DataRefObject

```

CreateBitFieldTerm :=

```

CreateBitField (
    SourceBuffer, // TermArg => Buffer
    BitIndex, // TermArg => Integer
    BitFieldName // NameString
)

```

CreateByteFieldTerm :=

```

CreateByteField (
    SourceBuffer, // TermArg => Buffer
    ByteIndex, // TermArg => Integer
    ByteFieldName // NameString
)

```

CreateDWordFieldTerm :=

```

CreateDWordField (
    SourceBuffer, // TermArg => Buffer
    ByteIndex, // TermArg => Integer
    DWordFieldName // NameString
)

```

CreateFieldTerm :=

```

CreateField (
    SourceBuffer, // TermArg => Buffer
    BitIndex, // TermArg => Integer
    NumBits, // TermArg => Integer
    FieldName // NameString
)

```

CreateQWordFieldTerm :=

```

CreateQWordField (
    SourceBuffer, // TermArg => Buffer
    ByteIndex, // TermArg => Integer
    QWordFieldName // NameString
)

```

CreateWordFieldTerm :=

```

CreateWordField (
    SourceBuffer, // TermArg => Buffer
    ByteIndex, // TermArg => Integer
    WordFieldName // NameString
)

```

DataRegionTerm :=

```

DataTableRegion (
    RegionName, // NameString
    SignatureString, // TermArg => String
    OemIDString, // TermArg => String
    OemTableIDString // TermArg => String
)

```

```

    )

DebugTerm :=
    Debug

DecSymbolicTerm :=
    Minuend // SuperName => Integer
    -
    => Integer

DecTerm :=
    Decrement (
        Minuend // SuperName
    ) => Integer

DefaultTerm :=
    Default { TermList }

DefinitionBlockTerm := | DefinitionBlock ( | AMLFileName, // String | TableSignature, // String | ComplianceRevision, // ByteConst | OEMID, // String | TableID, // String | OEMRevision // DWordConst | ) { TermList }

DerefOfTerm := | DerefOf ( | Source // NameString | ArgTerm | LocalTerm | RefOfTerm | CondRefOfTerm | // IndexTerm | MethodInvocationTerm | ) => DataRefObject

DeviceTerm := | Device ( | DeviceName // NameString | ) { TermList }

DivideCompoundTerm := | Dividend-Result // TermArg => Integer => Target | /= | Divisor // TermArg => Integer | => Integer

DivideSymbolicTerm :=
    Dividend // TermArg => Integer
    /
    Divisor // TermArg => Integer
    => Integer

DivideTerm :=
    Divide (
        Dividend, // TermArg => Integer
        Divisor, // TermArg => Integer
        Remainder, // Target
        Result // Target
    ) => Integer // Returns Result

EISAIDTerm :=
    EISAID (
        EisaIdString // StringData
    ) => DWordConst

ElseIfTerm :=
    ElseIf (
        Predicate // TermArg => Integer
    ) { TermList } ElseTerm

```

```

ElseTerm :=
    Else { TermList } | ElseIfTerm | Nothing

EventTerm :=
    Event (
        EventName // NameString
    )

ExternalTerm :=
    External (
        ObjName, // NameString
        ObjType, // Nothing | ObjectTypeKeyword
        ResultType, // Nothing | ParameterTypePackage
        ParameterTypes // Nothing | ParameterTypesPackage
    )

FatalTerm :=
    Fatal (
        Type, // ByteConstExpr
        Code, // DWordConstExpr
        Arg // TermArg => Integer
    )

FieldTerm :=
    Field (
        RegionName, // NameString => OperationRegion
        AccessType, // AccessTypeKeyword
        LockRule, // LockRuleKeyword
        UpdateRule // UpdateRuleKeyword
    ) {FieldUnitList}

FindSetLeftBitTerm :=
    FindSetLeftBit (
        Source, // TermArg => Integer
        Result // Target
    ) => Integer

FindSetRightBitTerm :=
    FindSetRightBit (
        Source, // TermArg => Integer
        Result // Target
    ) => Integer

ForTerm :=
    For (
        Initialize, // Nothing | TermArg => ComputationalData
        Predicate, // Nothing | TermArg => ComputationalData
        Update // Nothing | TermArg => ComputationalData
    ) {TermList}

```

FprintfTerm :=

```

Fprintf (
    TermArg,
    String,
    PrintfArgList
) => String

```

FromBCDTerm :=

```

FromBCD (
    BCDValue, // TermArg => Integer
    Result // Target
) => Integer

```

FunctionTerm :=

```

Function (
    FunctionName, // NameString
    ReturnType, // Nothing | ParameterTypePackage
    ParameterTypes // Nothing | ParameterTypesPackage
) {TermList}

```

IfTerm :=

```

If (
    Predicate // TermArg => Integer
) {TermList}

```

IncludeTerm :=

```

Include (
    FilePathName // StringData
)

```

IncSymbolicTerm :=

```

Addend // SuperName => Integer
++
=> Integer

```

IncTerm :=

```

Increment (
    Addend // SuperName
) => Integer

```

IndexFieldTerm :=

```

IndexField (
    IndexName, // NameString => FieldUnit
    DataName, // NameString => FieldUnit
    AccessType, // AccessTypeKeyword
    LockRule, // LockRuleKeyword
    UpdateRule // UpdateRuleKeyword
) {FieldUnitList}

```

IndexSymbolicTerm :=


```

Source // TermArg => <string | buffer | packageterm>
[Index] // TermArg => Integer
=> ObjectReference

```

IndexTerm :=

```

Index (
    Source, // TermArg => <string | buffer | packageterm>
    Index, // TermArg => Integer
    Destination // Target
) => ObjectReference

```

LAndSymbolicTerm :=

```

Source1 // TermArg => Integer
&&
Source2 // TermArg => Integer
=> Boolean

```

LAndTerm :=

```

LAnd (
    Source1, // TermArg => Integer
    Source2 // TermArg => Integer
) => Boolean

```

LEqualSymbolicTerm :=

```

Source1 // TermArg => ComputationalData
==
Source2 // TermArg => ComputationalData
=> Boolean

```

LEqualTerm :=

```

LEqual (
    Source1, // TermArg => ComputationalData
    Source2 // TermArg => ComputationalData
) => Boolean

```

LGreaterEqualSymbolicTerm :=

```

Source1 // TermArg => ComputationalData
>=
Source2 // TermArg => ComputationalData
=> Boolean

```

LGreaterEqualTerm :=

```

LGreaterEqual (
    Source1, // TermArg => ComputationalData
    Source2 // TermArg => ComputationalData
) => Boolean

```

LGreaterSymbolicTerm :=

```

Source1 // TermArg => ComputationalData
>

```

Source2 // TermArg => ComputationalData
=> Boolean

LGreaterTerm :=

LGreater (
 Source1, // *TermArg* => *ComputationalData*
 Source2 // TermArg => ComputationalData
) => Boolean

LLessEqualSymbolicTerm :=

Source1 // *TermArg* => *ComputationalData*
<=
Source2 // TermArg => ComputationalData
=> Boolean

LLessEqualTerm :=

LLessEqual (
 Source1, // *TermArg* => *ComputationalData*
 Source2 // TermArg => ComputationalData
) => Boolean

LLessSymbolicTerm :=

Source1 // *TermArg* => *ComputationalData*
<
Source2 // TermArg => ComputationalData
=> Boolean

LLessTerm :=

LLess (
 Source1, // *TermArg* => *ComputationalData*
 Source2 // TermArg => ComputationalData
) => Boolean

LNotEqualTerm :=

LNotEqual (
 Source1, // *TermArg* => *ComputationalData*
 Source2 // TermArg => ComputationalData
) => Boolean

LNotEqualSymbolicTerm :=

Source1 // *TermArg* => *ComputationalData*
!=
Source2 // TermArg => ComputationalData
=> Boolean

LNotSymbolicTerm :=

!
Source // *TermArg* => Integer
=> Boolean

LNotTerm :=

```
LNot (
    Source, // TermArg => Integer
) => Boolean
```

LOrSymbolicTerm :=

```
Source1 // TermArg => Integer
||
Source2 // TermArg => Integer
=> Boolean
```

LoadTableTerm :=

```
LoadTable (
    SignatureString, // TermArg => String
    OemIDString, // TermArg => String
    OemTableIDString, // TermArg => String
    RootPathString, // Nothing | TermArg => String
    ParameterPathString, // Nothing | TermArg => String
    ParameterData // Nothing | TermArg => DataRefObject
) => Boolean // True (non-zero) means table was successfully loaded
```

LoadTerm :=

```
Load (
    Object, // NameString
    Result // SuperName => Boolean – True (non-zero) // means the table was successfully loaded
) => Boolean // True (Ones) means the table was successfully loaded
```

LocalTerm :=

```
Local0 | Local1 | Local2 | Local3 | Local4 | Local5 | Local6 | Local7
```

LOrTerm :=

```
LOr (
    Source1, // TermArg => Integer
    Source2 // TermArg => Integer
) => Boolean
```

MatchTerm :=

```
Match (
    SearchPackage, // TermArg => Package
    Op1, // MatchOpKeyword
    MatchObject1, // TermArg => ComputationalData
    Op2, // MatchOpKeyword
    MatchObject2, // TermArg => ComputationalData
    StartIndex // TermArg => Integer
) => <ones | integer>
```

MethodTerm :=

```
Method (
    MethodName, // NameString
    NumArgs, // Nothing | ByteConstExpr
    SerializeRule, // Nothing | SerializeRuleKeyword
```

```

        SyncLevel, // Nothing | ByteConstExpr
        ReturnType, // Nothing | ParameterTypePackage
        ParameterTypes // Nothing | ParameterTypesPackage
    ) {TermList}

```

MidTerm :=

```

    Mid (
        Source, // TermArg => <buffer | String>
        Index, // TermArg => Integer
        Length, // TermArg => Integer
        Result // Target
    ) => <buffer | string>

```

ModCompoundTerm :=

```

    Dividend-Result // TermArg => Integer => Target
    %=
    Divisor // TermArg => Integer
    => Integer

```

ModSymbolicTerm :=

```

    Dividend // TermArg => Integer
    %=
    Divisor // TermArg => Integer
    => Integer

```

ModTerm :=

```

    Mod (
        Dividend, // TermArg => Integer
        Divisor, // TermArg => Integer
        Result // Target
    ) => Integer // Returns Result

```

MultiplyCompoundTerm :=

```

    Multiplicand-Result // TermArg => Integer => Target
    *=
    Multiplier // TermArg => Integer
    => Integer

```

MultiplySymbolicTerm :=

```

    Multiplicand // TermArg => Integer
    *
    Multiplier // TermArg => Integer
    => Integer

```

MultiplyTerm :=

```

    Multiply (
        Multiplicand, // TermArg => Integer
        Multiplier, // TermArg => Integer
        Result // Target
    ) => Integer

```

MutexTerm :=

```

Mutex (
    MutexName, // NameString
    SyncLevel // ByteConstExpr
)

```

NameTerm :=

```

Name (
    ObjectName, // NameString
    Object // DataObject
)

```

NAndTerm :=

```

NAnd (
    Source1, // TermArg => Integer
    Source2, // TermArg => Integer
    Result // Target
) => Integer

```

NoOpTerm :=

```

NoOp

```

NOrTerm :=

```

NOr (
    Source1, // TermArg => Integer
    Source2, // TermArg => Integer
    Result // Target
) => Integer

```

NotifyTerm :=

```

Notify (
    Object, // SuperName => <thermalzone | processor | device>
    NotificationValue // TermArg => Integer
)

```

NotSymbolicTerm :=

```

~
Source // TermArg => Integer
=> Integer

```

NotTerm :=

```

Not (
    Source, // TermArg => Integer
    Result // Target
) => Integer

```

ObjectTypeTerm :=

```

ObjectType (
    Object // NameString | ArgTerm | LocalTerm | DebugTerm |
    // RefOfTerm | DerefOfTerm | IndexTerm
)

```

```

    ) => Integer
OffsetTerm :=
    Offset (
        ByteOffset // IntegerData
    )
OpRegionTerm :=
    OperationRegion (
        RegionName, // NameString
        RegionSpace, // RegionSpaceKeyword
        Offset, // TermArg => Integer
        Length // TermArg => Integer
    )
OrCompoundTerm :=
    Source1-Result // TermArg => Integer => Target
    |=
    Source2 // TermArg => Integer
    => Integer
OrSymbolicTerm :=
    Source1 // TermArg => Integer
    |
    Source2 // TermArg => Integer
    => Integer
OrTerm :=
    Or (
        Source1, // TermArg => Integer
        Source2, // TermArg => Integer
        Result // Target
    ) => Integer
PackageTerm :=
    Package (
        NumElements // Nothing | ByteConstExpr | TermArg => Integer
    ) {PackageList} => Package
PLDKeyword :=
    PLD_Revision | PLD_IgnoreColor | PLD_Red | PLD_Green | PLD_Blue | PLD_Width |
    PLD_Height | PLD_UserVisible | PLD_Dock | PLD_Lid | PLD_Panel | PLD_VerticalPosition
    | PLD_HorizontalPosition | PLD_Shape | PLD_GroupOrientation | PLD_GroupToken |
    PLD_GroupPosition | PLD_Bay | PLD_Ejectable | PLD_EjectRequired | PLD_CabinetNumber
PLDKeywordList :=
    PLDKeyword = StringDataPLD_Revision | PLDKeyword = IntegerDataPLD_Revision | PLDKey-
    word = StringDataPLD_Revision, PLDKeywordListPLD_Revision, PLDKeyword = IntegerData-
    PLD_Revision, PLDKeywordListPLD_Revision
PowerResTerm :=
    PowerResource (

```

```

        ResourceName, // NameString
        SystemLevel, // ByteConstExpr
        ResourceOrder // WordConstExpr
    ) {TermList}

PrintfArgList :=
    TermArg | TermArg , PrintfArgList

PrintfTerm :=
    Printf (
        String,
        PrintfArgList
    ) => String

ProcessorTerm :=
    Processor (
        ProcessorName, // NameString
        ProcessorID, // ByteConstExpr
        PBlockAddress, // DWordConstExpr | Nothing (=0)
        PblockLength // ByteConstExpr | Nothing (=0)
    ) {TermList}

RawDataBufferTerm :=
    RawDataBuffer (
        BuffSize // Nothing | WordConst
    ) { ByteList } => RawDataBuffer

RefOfTerm :=
    RefOf (
        Source // NameString | ArgTerm | LocalTerm | DerefOfTerm
    ) => ObjectReference

ReleaseTerm :=
    Release (
        SyncObject // SuperName
    )

ResetTerm :=
    Reset (
        SyncObject // SuperName
    )

ReturnTerm :=
    Return (
        Arg // Nothing | TermArg => DataRefObject
    )

ScopeTerm :=
    Scope (
        Location // NameString
    ) { TermList }

```

ShiftLeftCompoundTerm :=

```

Source-Result // TermArg => Integer => Target
<<=
ShiftCount // TermArg => Integer
=> Integer

```

ShiftLeftSymbolicTerm :=

```

Source // TermArg => Integer
<<
ShiftCount // TermArg => Integer
=> Integer

```

ShiftLeftTerm :=

```

ShiftLeft (
    Source, // TermArg => Integer
    ShiftCount, // TermArg => Integer
    Result // Target
) => Integer

```

ShiftRightCompoundTerm :=

```

Source-Result // TermArg => Integer => Target
>>=
ShiftCount // TermArg => Integer
=> Integer

```

ShiftRightSymbolicTerm :=

```

Source // TermArg => Integer
>>
ShiftCount // TermArg => Integer
=> Integer

```

ShiftRightTerm :=

```

ShiftRight (
    Source, // TermArg => Integer
    ShiftCount, // TermArg => Integer
    Result // Target
) => Integer

```

SignalTerm :=

```

Signal (
    SyncObject // SuperName
)

```

SizeOfTerm :=

```

SizeOf (
    DataObject // SuperName => <string | buffer | package>
) => Integer

```

SleepTerm :=

```

Sleep (

```



```

        MilliSeconds // TermArg => Integer
    )
StallTerm :=
    Stall (
        MicroSeconds // TermArg => Integer
    )
StoreSymbolicTerm :=
    Destination // SuperName
    =
    Source // TermArg => DataRefObject
    => DataRefObject
StoreTerm :=
    Store (
        Source, // TermArg => DataRefObject
        Destination // SuperName
    ) => DataRefObject
SubtractCompoundTerm :=
    Minuend-Result // TermArg => Integer => Target
    -=
    Subtrahend // TermArg => Integer
    => Integer
SubtractSymbolicTerm :=
    Minuend // TermArg => Integer
    Subtrahend // TermArg => Integer
    => Integer
SubtractTerm :=
    Subtract (
        Minuend, // TermArg => Integer
        Subtrahend, // TermArg => Integer
        Result // Target
    ) => Integer
SwitchTerm :=
    Switch (
        Predicate // TermArg => ComputationalData
    ) { CaseTermList }
ThermalZoneTerm :=
    ThermalZone (
        ThermalZoneName // NameString
    ) { TermList }
TimerTerm :=
    Timer => Integer
ToBCDTerm :=

```

```

ToBCD (
    Value, // TermArg => Integer
    Result // Target
) => Integer

ToBufferTerm :=
ToBuffer (
    Data, // TermArg => ComputationalData
    Result // Target
) => ComputationalData

ToDecimalStringTerm :=
ToDecimalString (
    Data, // TermArg => ComputationalData
    Result // Target
) => String

ToHexStringTerm :=
ToHexString (
    Data, // TermArg => ComputationalData
    Result // Target
) => String

ToIntegerTerm :=
ToInteger (
    Data, // TermArg => ComputationalData
    Result // Target
) => Integer

ToPLDTerm :=
ToPLD (
    PLDKeywordList
) => Buffer

ToStringTerm :=
ToString (
    Source, // TermArg => Buffer
    Length, // Nothing | TermArg => Integer
    Result // Target
) => String

ToUUIDTerm :=
ToUUID (
    String // StringData
) => Buffer

UnicodeTerm :=
Unicode (
    String // StringData
) => Buffer

```

UnloadTerm :=

```

Unload (
    DDBHandle // SuperName
)

```

WaitTerm :=

```

Wait (
    SyncObject, // SuperName => Event
    TimeoutValue // TermArg => Integer
) => Boolean // True means timed-out

```

WhileTerm :=

```

While (
    Predicate // TermArg => Integer
) {TermList}

```

XorCompoundTerm :=

```

Source1-Result // TermArg => Integer => Target
^=
Source2 // TermArg => Integer
=> Integer

```

XorSymbolicTerm :=

```

Source1 // TermArg => Integer
^
Source2 // TermArg => Integer
=> Integer

```

XOrTerm :=

```

XOr (
    Source1, // TermArg => Integer
    Source2, // TermArg => Integer
    Result // Target
) => Integer

```

19.2.7 ASL Parameter Keyword Terms**AccessAttribKeyword :=**

AttribQuick | AttribSendReceive | AttribByte | AttribBytes (n) | AttribRawBytes (n) | AttribRawProcessBytes (n) | AttribWord | AttribBlock | AttribProcessCall | AttribBlockProcessCall // Note: Used for SMBus and GenericSerialBus BufferAcc only |

AccessTypeKeyword :=

AnyAcc | ByteAcc | WordAcc | DWordAcc | QWordAcc | BufferAcc

AddressKeyword :=

AddressRangeMemory | AddressRangeReserved | AddressRangeNVS | AddressRangeACPI

AddressSpaceKeyword :=

RegionSpaceKeyword

AddressingModeKeyword :=
 AddressingMode7Bit | AddressingMode10Bit

ByteLengthKeyword :=
 DataBitsFive | DataBitsSix | DataBitsSeven | DataBitsEight | DataBitsNine

BusMasterKeyword :=
 BusMaster | NotBusMaster

ClockPhaseKeyword :=
 ClockPhaseFirst | ClockPhaseSecond

ClockPolarityKeyword :=
 ClockPolarityLow | ClockPolarityHigh

DecodeKeyword :=
 SubDecode | PosDecode

EndianKeyword :=
 BigEndianin | LittleEndian

ExtendedAccessAttribKeyword :=
 AttribBytes | AttribRawBytes | AttribRawProcessBytes // Note: Used for GenericSerialBus Buffer-Acc only.

FlowControlKeyword :=
 FlowControlNone | FlowControlXon | FlowControlHardware

InterruptTypeKeyword :=
 Edge | Level

InterruptLevel :=
 ActiveHigh | ActiveLow

InterruptLevelKeyword :=
 ActiveHigh | ActiveLow | ActiveBoth

IODecodeKeyword :=
 Decode16 | Decode10

IoRestrictionKeyword :=
 IoRestrictionNone | IoRestrictionInputOnly | IoRestrictionOutputOnly | IoRestrictionNoneAndPre-serve

LockRuleKeyword :=
 Lock | NoLock

MatchOpKeyword :=
 MTR | MEQ | MLE | MLT | MGE | MGT

MaxKeyword :=
 MaxFixed | MaxNotFixed

MemTypeKeyword :=
 Cacheable | WriteCombining | Prefetchable | NonCacheable

MinKeyword :=
 MinFixed | MinNotFixed

ObjectTypeKeyword :=
 UnknownObj | IntObj | StrObj | BuffObj | PkgObj | FieldUnitObj | DeviceObj | EventObj | MethodObj
 | MutexObj | OpRegionObj | PowerResObj | ThermalZoneObj | BuffFieldObj

ParityKeyword :=
 ParityTypeNone | ParityTypeSpace | ParityTypeMark | ParityTypeOdd | ParityTypeEven

PinConfigKeyword :=
 PullDefault | PullUp | PullDown | PullNone

PolarityKeyword :=
 PolarityHigh | PolarityLow

RangeTypeKeyword :=
 ISAOnlyRanges | NonISAOnlyRanges | EntireRange

ReadWriteKeyword :=
 ReadWrite | ReadOnly

RegionSpaceKeyword :=
 SystemIO | SystemMemory | PCI_Config | EmbeddedControl | SMBus | SystemCMOS | PciBarTarget
 | IPMI | GeneralPurposeIO | GenericSerialBus | PCC | PRM | FFixedHW

ResourceTypeKeyword :=
 ResourceConsumer | ResourceProducer

SerializeRuleKeyword :=
 Serialized | NotSerialized

ShareTypeKeyword :=
 Shared | Exclusive | SharedAndWake | ExclusiveAndWake

SlaveModeKeyword :=
 ControllerInitiated | DeviceInitiated

StopBitsKeyword :=
 StopBitsZero | StopBitsOne | StopBitsOnePlusHalf | StopBitsTwo

TransferWidthKeyword :=
 Width8Bit | Width16Bit | Width32Bit | Width64Bit | Width128Bit | Width256Bit

TranslationKeyword :=
 SparseTranslation | DenseTranslation

TypeKeyword :=
 TypeTranslation | TypeStatic

UpdateRuleKeyword :=
 Preserve | WriteAsOnes | WriteAsZeros

UserDefRegionSpace :=
 IntegerData => 0x80 - 0xFF

XferTypeKeyword :=
 Transfer8 | Transfer16 | Transfer8_16

WireModeKeyword :=
 ThreeWireMode | FourWireMode

19.2.8 ASL Resource Template Terms

ResourceMacroList :=

Nothing | *<resourceterm resourceterm>*

ResourceMacroTerm :=

*DMA*Term | *DWordIO*Term | *DWordMemory*Term | *DWordSpace*Term | *EndDependentFn*Term | *ExtendedIO*Term | *ExtendedMemory*Term | *ExtendedSpace*Term | *FixedDMA*Term | *FixedIO*Term | *GpioInt*Term | *GpioIO*Term | *I2CSerialBus*Term | *Interrupt*Term | *IO*Term | *IRQNoFlags*Term | *IRQ*Term | *Memory24*Term | *Memory32Fixed*Term | *Memory32*Term | *PinConfig*Term | *PinFunction*Term | *PinGroup*Term | *PinGroupConfig*Term | *PinGroupFunction*Term | *QWordIO*Term | *QWordMemory*Term | *QWordSpace*Term | *Register*Term | *SPISerialBus*Term | *StartDependentFn*Term | *StartDependentFnNoPri*Term | *UARTSerialBus*Term | *VendorLong*Term | *VendorShort*Term | *WordBusNumber*Term | *WordIO*Term | *WordSpace*Term

DMATerm :=

DMA (
 DMAType, // DMATypeKeyword (_TYP)
 BusMaster, // *BusMasterKeyword* (_BM)
 XferType, // *XferTypeKeyword* (_SIZ)
 DescriptorName // Nothing | *NameString*
) {ByteList} // List of channels (0-7 bytes)

DWordIOTerm :=

DWordIO (
 ResourceUsage, // Nothing (ResourceConsumer) | *ResourceTypeKeyword*
 MinType, // Nothing (MinNotFixed) | *MinKeyword* (_MIF)
 MaxType, // Nothing (MaxNotFixed) | *MaxKeyword* (_MAF)
 Decode, // Nothing (PosDecode) | *DecodeKeyword* (_DEC)
 RangeType, // Nothing (EntireRange) | *RangeTypeKeyword* (_RNG)
 AddressGranularity, // *DWordConstExpr* (_GRA)
 MinAddress, // DWordConstExpr (_MIN)
 MaxAddress, // DWordConstExpr (_MAX)
 AddressTranslation, // DWordConstExpr (_TRA)
 AddressLength, // DWordConstExpr (_LEN)
 ResourceSourceIndex, // Nothing | *ByteConstExpr*
 ResourceSource, // Nothing | *StringData*
 DescriptorName, // Nothing | *NameString*
 TranslationType, // Nothing | *TypeKeyword* (_TTP)
 TranslationDensity // Nothing | *TranslationKeyword* (_TRS)
)

DWordMemoryTerm :=

DWordMemory (
 ResourceUsage, // Nothing (ResourceConsumer) | *ResourceTypeKeyword*
 Decode, // Nothing (PosDecode) | *DecodeKeyword* (_DEC)
 MinType, // Nothing (MinNotFixed) | *MinKeyword* (_MIF)
 MaxType, // Nothing (MaxNotFixed) | *MaxKeyword* (_MAF)
 MemType, // Nothing (NonCacheable) | *MemTypeKeyword* (_MEM)
 ReadWriteType, // *ReadWriteKeyword* (_RW)
)

```

    AddressGranularity, // DWordConstExpr (_GRA)
    MinAddress, // DWordConstExpr (_MIN)
    MaxAddress, // DWordConstExpr (_MAX)
    AddressTranslation, // DWordConstExpr (_TRA)
    AddressLength, // DWordConstExpr (_LEN)
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceSource, // Nothing | StringData
    DescriptorName, // Nothing | NameString
    MemoryRangeType, // Nothing | AddressKeyword (_MTP)
    TranslationType // Nothing | TypeKeyword (_TTP)
)

DWordSpaceTerm :=
    DWordSpace (
        ResourceType, // ByteConstExpr (_RT), 0xC0 - 0xFF
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
        MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
        MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
        TypeSpecificFlags, // ByteConstExpr (_TSF)
        AddressGranularity, // DWordConstExpr (_GRA)
        MinAddress, // DWordConstExpr (_MIN)
        MaxAddress, // DWordConstExpr (_MAX)
        AddressTranslation, // DWordConstExpr (_TRA)
        AddressLength, // DWordConstExpr (_LEN)
        ResourceSourceIndex, // Nothing | ByteConstExpr
        ResourceSource, // Nothing | StringData
        DescriptorName // Nothing | NameString
    )

EndDependentFnTerm :=
    EndDependentFn ()

ExtendedIOTerm :=
    ExtendedIO (
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
        MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
        Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
        RangeType, // Nothing (EntireRange) | RangeTypeKeyword (_RNG)
        AddressGranularity, // QWordConstExpr (_GRA)
        MinAddress, // QWordConstExpr (_MIN)
        MaxAddress, // QWordConstExpr (_MAX)
        AddressTranslation, // QWordConstExpr (_TRA)
        AddressLength, // QWordConstExpr (_LEN)
        TypeSpecificAttributes, // Nothing | QWordConstExpr
        DescriptorName, // Nothing | NameString
        TranslationType, // Nothing | TypeKeyword (_TTP)
        TranslationDensity // Nothing | TranslationKeyword (_TRS)
    )

```

)

ExtendedMemoryTerm :=

```

ExtendedMemory (
    ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    MemType, // Nothing (NonCacheable) | MemTypeKeyword (_MEM)
    ReadWriteType, // ReadWriteKeyword (_RW)
    AddressGranularity, // QWordConstExpr (_GRA)
    MinAddress, // QWordConstExpr (_MIN)
    MaxAddress, // QWordConstExpr (_MAX)
    AddressTranslation, // QWordConstExpr (_TRA)
    AddressLength, // QWordConstExpr (_LEN)
    TypeSpecificAttributes, // Nothing | QWordConstExpr
    DescriptorName, // Nothing | NameString
    MemoryRangeType, // Nothing | AddressKeyword (_MTP)
    TranslationType // Nothing | TypeKeyword (_TTP)
)

```

ExtendedSpaceTerm :=

```

ExtendedSpace (
    ResourceType, // ByteConstExpr (_RT), 0xC0 - 0xFF
    ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    TypeSpecificFlags, // ByteConstExpr (_TSF)
    AddressGranularity, // QWordConstExpr (_GRA)
    MinAddress, // QWordConstExpr (_MIN)
    MaxAddress, // QWordConstExpr (_MAX)
    AddressTranslation, // QWordConstExpr (_TRA)
    AddressLength, // QWordConstExpr (_LEN)
    TypeSpecificAttributes, // Nothing | QWordConstExpr (_ATT)
    DescriptorName // Nothing | NameString
)

```

FixedDMATerm :=

```

FixedDMA (
    DMAReq, // WordConstExpr (_DMA)
    Channel, // WordConstExpr (_TYP)
    XferWidth, // Nothing (Width32Bit) | TransferWidthKeyword (_SIZ)
    DescriptorName, // Nothing | NameString
)

```

FixedIOTerm :=

```

FixedIO (

```



```

    AddressBase, // WordConstExpr (_BAS)
    RangeLength, // ByteConstExpr (_LEN)
    DescriptorName // Nothing | NameString
)

GpioIntTerm :=
    GpioInt (
        InterruptType, // InterruptTypeKeyword (_MOD)
        InterruptLevel, // InterruptLevelKeyword (_POL)
        ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
        PinConfig, // PinConfigKeyword | ByteConstExpr (_PPI)
        DeBounceTime // Nothing | WordConstExpr (_DBT)
        ResourceSource, // StringData
        ResourceSourceIndex, // Nothing (0) | ByteConstExpr
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        DescriptorName, // Nothing | NameString
        VendorData // Nothing | RawDataBuffer (_VEN)
    ) {DWordList} // List of GPIO pins (_PIN)

GpioIOTerm :=
    GpioIO (
        ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
        PinConfig, // PinConfigKeyword | ByteConstExpr (_PPIC)
        DeBounceTime // Nothing | WordConstExpr (_DBT)
        DriveStrength // Nothing | WordConstExpr (_DRS)
        IORestriction // Nothing (None) | IORestrictionKeyword (_IOR)
        ResourceSource, // StringData
        ResourceSourceIndex, // Nothing (0) | ByteConstExpr
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        DescriptorName, // Nothing | NameString
        VendorData // Nothing | RawDataBuffer (_VEN)
    ) {DWordList} // List of GPIO pins (_PIN)

I2CSerialBusTerm :=
    I2CSerialBusV2 (
        SlaveAddress, // WordConstExpr (_ADR)
        SlaveMode, // Nothing (ControllerInitiated) | SlaveModeKeyword (_SLV)
        ConnectionSpeed, // DWordConstExpr (_SPE)
        AddressingMode, // Nothing (AddressingMode7Bit) | AddressModeKeyword (_MOD)
        ResourceSource, // StringData
        ResourceSourceIndex, // Nothing | ByteConstExpr
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        DescriptorName, // Nothing | NameString
        ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
        VendorData // Nothing | RawDataBuffer (_VEN)
    )

InterruptTerm :=

```

```

Interrupt (
    ResourceType, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    InterruptType, // InterruptTypeKeyword (_LL, _HE)
    InterruptLevel, // InterruptLevelKeyword (_LL, _HE)
    ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceSource, // Nothing | StringData
    DescriptorName // Nothing | NameString
) {DWordList} // list of interrupts (_INT)

```

IOTerm :=

```

IO (
    IODecode, // IODecodeKeyword (_DEC)
    MinAddress, // WordConstExpr (_MIN)
    MaxAddress, // WordConstExpr (_MAX)
    Alignment, // ByteConstExpr (_ALN)
    RangeLength, // ByteConstExpr (_LEN)
    DescriptorName // Nothing | NameString
)

```

IRQNoFlagsTerm :=

```

IRQNoFlags (
    DescriptorName // Nothing | NameString
) {ByteList} // list of interrupts (0-15 bytes)

```

IRQTerm :=

```

IRQ (
    InterruptType, // InterruptTypeKeyword (_LL, _HE)
    InterruptLevel, // InterruptLevelKeyword (_LL, _HE)
    ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
    DescriptorName // Nothing | NameString
) {ByteList} // list of interrupts (0-15 bytes)

```

Memory24Term :=

```

Memory24 (
    ReadWriteType, // ReadWriteKeyword (_RW)
    MinAddress[23:8], // WordConstExpr (_MIN)
    MaxAddress[23:8], // WordConstExpr (_MAX)
    Alignment, // WordConstExpr (_ALN)
    RangeLength, // WordConstExpr (_LEN)
    DescriptorName // Nothing | NameString
)

```

Memory32FixedTerm :=

```

Memory32Fixed (
    ReadWriteType, // ReadWriteKeyword (_RW)
    AddressBase, // DWordConstExpr (_BAS)
    RangeLength, // DWordConstExpr (_LEN)
    DescriptorName // Nothing | NameString
)

```

```

    )
Memory32Term :=
    Memory32 (
        ReadWriteType, // ReadWriteKeyword (_RW)
        MinAddress, // DWordConstExpr (_MIN)
        MaxAddress, // DWordConstExpr (_MAX)
        Alignment, // DWordConstExpr (_ALN)
        RangeLength, // DWordConstExpr (_LEN)
        DescriptorName // Nothing | NameString
    )
PinConfigTerm :=
    PinConfig (
        ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
        PinConfigType, // ByteData (_TYP)
        PinConfigValue, // ByteData (_VAL)
        ResourceSource, // StringData
        ResourceSourceIndex, // Nothing (0) | ByteConstExpr
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        DescriptorName, // Nothing | NameString
        VendorData // Nothing | RawDataBuffer (_VEN)
    ) {DWordList} (_PIN)
PinFunctionTerm :=
    PinFunction (
        ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
        PinPullConfiguration, // PinConfigKeyword | ByteConstExpr (_PPI)
        FunctionNumber, // WordData
        ResourceSource, // StringData
        ResourceSourceIndex, // Nothing (0) | ByteConstExpr
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        DescriptorName, // Nothing | NameString
        VendorData // Nothing | RawDataBuffer (_VEN)
    ) {DWordList} (_PIN)
PinGroupTerm :=
    PinGroup (
        ResourceLabel, // StringData
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        DescriptorName, // Nothing | NameString
        VendorData // Nothing | RawDataBuffer (_VEN)
    ) {DWordList} (_PIN)
PinGroupConfigTerm :=
    PinGroupConfig (
        ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
        PinConfigType, // ByteData (_TYP)
        PinConfigValue, // ByteData (_VAL)

```

```

ResourceSource, // StringData
ResourceSourceIndex, // Nothing (0) | ByteConstExpr
ResourceSourceLabel, // StringData
ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
DescriptorName, // Nothing | NameString
VendorData // Nothing | RawDataBuffer (_VEN)
)

```

PinGroupFunctionTerm :=

```

PinGroupFunction (
    ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
    FunctionNumber, // WordData (_FUN)
    ResourceSource, // StringData
    ResourceSourceIndex, // Nothing (0) | ByteConstExpr
    ResourceSourceLabel, // StringData
    ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    DescriptorName, // Nothing | NameString
    VendorData // Nothing | RawDataBuffer (_VEN)
)

```

QWordIOTerm :=

```

QWordIO (
    ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    RangeType, // Nothing (EntireRange) | RangeTypeKeyword (_RNG)
    AddressGranularity, // QWordConstExpr (_GRA)
    MinAddress, // QWordConstExpr (_MIN)
    MaxAddress, // QWordConstExpr (_MAX)
    AddressTranslation, // QWordConstExpr (_TRA)
    AddressLength, // QWordConstExpr (_LEN)
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceSource, // Nothing | StringData
    DescriptorName, // Nothing | NameString
    TranslationType, // Nothing | TypeKeyword (_TTP)
    TranslationDensity // Nothing | TranslationKeyword (_TRS)
)

```

QWordMemoryTerm :=

```

QWordMemory (
    ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    MemType, // Nothing (NonCacheable) | MemTypeKeyword (_MEM)
    ReadWriteType, // ReadWriteKeyword (_RW)
    AddressGranularity, // QWordConstExpr (_GRA)
)

```

```

    MinAddress, // QWordConstExpr (_MIN)
    MaxAddress, // QWordConstExpr (_MAX)
    AddressTranslation, // QWordConstExpr (_TRA)
    AddressLength, // QWordConstExpr (_LEN)
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceSource, // Nothing | StringData
    DescriptorName, // Nothing | NameString
    MemoryRangeType, // Nothing | AddressKeyword (_MTP)
    TranslationType // Nothing | TypeKeyword (_TTP)
)

```

QWordSpaceTerm :=

```

QWordSpace (
    ResourceType, // ByteConstExpr (_RT), 0xC0 - 0xFF
    ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    TypeSpecificFlags, // ByteConstExpr (_TSF)
    AddressGranularity, // QWordConstExpr (_GRA)
    MinAddress, // QWordConstExpr (_MIN)
    MaxAddress, // QWordConstExpr (_MAX)
    AddressTranslation, // QWordConstExpr (_TRA)
    AddressLength, // QWordConstExpr (_LEN)
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceSource, // Nothing | StringData
    DescriptorName // Nothing | NameString
)

```

RegisterTerm :=

```

Register (
    AddressSpaceID, // AddressSpaceKeyword (_ASI)
    RegisterBitWidth, // ByteConstExpr (_RBW)
    RegisterOffset, // ByteConstExpr (_RBO)
    RegisterAddress, // QWordConstExpr (_ADR)
    AccessSize, // ByteConstExpr (_ASZ)
    DescriptorName // Nothing | NameString
)

```

SPISerialBusTerm :=

```

SPISerialBusV2 (
    DeviceSelection, // WordConstExpr (_ADR)
    DeviceSelectionPolarity, // Nothing (PolarityLow) |
    DevicePolarityKeyword (_DPL)
    WireMode, // Nothing (FourWireMode) | WireModeKeyword (_MOD)
    DataBitLength, // ByteConstExpr (_LEN)
    SlaveMode, // Nothing (ControllerInitiated) | SlaveModeKeyword (_SLV)
    ConnectionSpeed, // DWordConstExpr (_SPE)
)

```

```

    ClockPolarity, // ClockPolarityKeyword (_POL)
    ClockPhase, // ClockPhaseKeyword (_PHA)
    ResourceSource, // StringData
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
    DescriptorName, // Nothing | NameString
    ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
    VendorData // Nothing | RawDataBuffer (_VEN)
)

StartDependentFnNoPriTerm :=
    StartDependentFnNoPri () {ResourceMacroList}

StartDependentFnTerm :=
    StartDependentFn (
        CompatPriority, // ByteConstExpr (0-2)
        PerfRobustPriority // ByteConstExpr (0-2)
    ) {ResourceMacroList}

UARTSerialBusTerm :=
    UARTSerialBusV2(
        Initial BaudRate, // DWordConstExpr (_SPE)
        BitsPerByte, // Nothing (DataBitsEight) | DataBitsKeyword (_LEN)
        StopBits, // Nothing (StopBitsOne) | StopBitsKeyword (_STB)
        LinesInUse, // ByteConstExpr (_LIN)
        IsBigEndian, // Nothing (LittleEndian) | EndianessKeyword (_END)
        Parity, // Nothing (ParityTypeNone) | ParityTypeKeyword (_PAR)
        FlowControl, // Nothing (FlowControlNone) | FlowControlKeyword (_FLC)
        ReceiveBufferSize, // WordConstExpr (_RXL)
        TransmitBufferSize, // WordConstExpr (_TXL)
        ResourceSource, // StringData
        ResourceSourceIndex, // Nothing | ByteConstExpr
        ResourceUsage, // Nothing (ResourceConsumer) | ResourceTypeKeyword
        DescriptorName, // Nothing | NameString
        ShareType, // Nothing (Exclusive) | ShareTypeKeyword (_SHR)
        VendorData // Nothing | Object (_VEN)
    )

VendorLongTerm :=
    VendorLong (
        DescriptorName // Nothing | NameString
    ) {ByteList}

VendorShortTerm :=
    VendorShort (
        DescriptorName // Nothing | NameString
    ) {ByteList} // Up to 7 bytes

WordBusNumberTerm :=

```

```

WordBusNumber (
    ResourceUsage, // Nothing (ResourceConsumer)| ResourceTypeKeyword
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    AddressGranularity, // WordConstExpr (_GRA)
    MinAddress, // WordConstExpr (_MIN)
    MaxAddress, // WordConstExpr (_MAX)
    AddressTranslation, // WordConstExpr (_TRA)
    AddressLength, // WordConstExpr (_LEN)
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceSource, // Nothing | StringData
    DescriptorName // Nothing | NameString
)

```

WordIOTerm :=

```

WordIO (
    ResourceUsage, // Nothing (ResourceConsumer)| ResourceTypeKeyword
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    RangeType, // Nothing (EntireRange) | RangeTypeKeyword (_RNG)
    AddressGranularity, // WordConstExpr (_GRA)
    MinAddress, // WordConstExpr (_MIN)
    MaxAddress, // WordConstExpr (_MAX)
    AddressTranslation, // WordConstExpr (_TRA)
    AddressLength, // WordConstExpr (_LEN)
    ResourceSourceIndex, // Nothing | ByteConstExpr
    ResourceSource, // Nothing | StringData
    DescriptorName, // Nothing | NameString
    TranslationType, // Nothing | TypeKeyword (_TTP)
    TranslationDensity // Nothing | TranslationKeyword (_TRS)
)

```

WordSpaceTerm :=

```

WordSpace (
    ResourceType, // ByteConstExpr (_RT), 0xC0 - 0xFF
    ResourceUsage, // Nothing (ResourceConsumer)| ResourceTypeKeyword
    Decode, // Nothing (PosDecode) | DecodeKeyword (_DEC)
    MinType, // Nothing (MinNotFixed) | MinKeyword (_MIF)
    MaxType, // Nothing (MaxNotFixed) | MaxKeyword (_MAF)
    TypeSpecificFlags, // ByteConstExpr (_TSF)
    AddressGranularity, // WordConstExpr (_GRA)
    MinAddress, // WordConstExpr (_MIN)
    MaxAddress, // WordConstExpr (_MAX)
    AddressTranslation, // WordConstExpr (_TRA)
    AddressLength, // WordConstExpr (_LEN)
    ResourceSourceIndex, // Nothing | ByteConstExpr
)

```

```

ResourceSource, // Nothing | StringData
DescriptorName // Nothing | NameString
)

```

19.3 ASL Concepts

This reference section is for developers who are writing ASL code while developing definition blocks for platforms.

19.3.1 ASL Names

This section describes how to encode object names using ASL.

The following table lists the characters legal in any position in an ASL object name. ASL names are not case-sensitive and will be converted to upper case.

Table 19.2: Named Object Reference Encodings

Value	Description	Title
0x41-0x5A, 0x5F, 0x61-0x7A	Lead character of name ('A'-'Z', '_', 'a'-'z')	LeadNameChar
0x30-0x39, 0x41-0x5A, 0x5F, 0x61-0x7A	Non-lead (trailing) character of name ('A'-'Z', '_', 'a'-'z', '0'-'9')	NameChar

The following table lists the name modifiers that can be prefixed to an ASL name.

Table 19.3: Definition Block Name Modifier Encodings

Value	Description	NamePrefix :=	Follows by...
0x5C	Namespace root ('')	RootPrefix	Name
0x5E	Parent namespace ('^')	ParentPrefix	ParentPrefix or Name

19.3.1.1 _T_x Reserved Object Names

The ACPI specification reserves object names with the prefix `_T_` for internal use by the ASL compiler. The ASL compiler may, for example, use these objects to store temporary values when implementing translation of complicated control structures into AML. The ASL compiler must declare `_T_x` objects normally (using `Name`) and must not define them more than once within the same scope.

19.3.2 ASL Literal Constants

This section describes how to encode integer and string constants using ASL.

19.3.2.1 Integers

```
DigitChar      := '0'-'9'
LeadDigitChar  := '1'-'9'
OctalDigitChar := '0'-'7'
HexDigitChar   := DigitChar \| 'A'-'F' \| 'a'-'f'
```

```
Integer        := DecimalConst \| OctalConst \| HexConst
DecimalConst    := LeadDigitChar \| <decimalconst digitchar>
OctalConst      := '0' \| <octalconst octaldigitchar>
HexConst        := <0x hexdigitchar> \| <0x hexdigitchar> \| <HexConst HexDigitChar>
ByteConst       := Integer => 0x00-0xFF
WordConst       := Integer => 0x0000-0xFFFF
DWordConst      := Integer => 0x00000000-0xFFFFFFFF
QWordConst      := Integer => 0x0000000000000000-0xFFFFFFFFFFFFFFFF
```

Numeric constants can be specified in decimal, octal, or hexadecimal. Octal constants are preceded by a leading zero (0), and hexadecimal constants are preceded by a leading zero and either a lower or upper case ‘x’. In some cases, the grammar specifies that the number must evaluate to an integer within a limited range, such as 0x00-0xFF, and so on.

19.3.2.2 Strings

```
String          := '"' Utf8CharList '"'
Utf8CharList    := Nothing | <escapesequence utf8charlist> | <Utf8Char Utf8CharList>
Utf8Char        := 0x01-0x21 |
                   0x23-0x5B |
                   0x5D-0x7F |
                   0xC2-0xDF 0x80-0xBF |
                   0xE0 0xA0-0xBF 0x80-0xBF |
                   0xE1-0xEC 0x80-0xBF 0x80-0xBF |
                   0xED 0x80-0x9F 0x80-0xBF |
                   0xEE-0xEF 0x80-0xBF 0x80-0xBF |
                   0xF0 0x90-0xBF 0x80-0xBF 0x80-0xBF |
                   0xF1-0xF3 0x80-0xBF 0x80-0xBF 0x80-0xBF |
                   0xF4 0x80-0x8F 0x80-0xBF 0x80-0xBF
EscapeSeq       := SimpleEscapeSeq | OctalEscapeSeq | HexEscapeSeq
SimpleEscapeSeq := \' | \" | \a | \b | \f | \n | \r | \t | \v | \\
OctalEscapeSeq  := \ OctalDigitChar |
                   \ OctalDigitChar OctalDigitChar |
                   \ OctalDigitChar OctalDigitChar OctalDigitChar
HexEscapeSeq     := \x HexDigitChar |
                   \x HexDigitChar HexDigitChar
NullChar        := 0x00
```

String literals consist of zero or more ASCII characters surrounded by double quotation marks (“”). A string literal represents a sequence of characters that, taken together, form a null-terminated string. After all adjacent strings in the constant have been concatenated, a null character is appended.

Strings in the source file may be encoded using the UTF-8 encoding scheme as defined in the Unicode 4.0 specification. UTF-8 is a byte-oriented encoding scheme, where some characters take a single byte and others take multiple bytes. The ASCII character values 0x01-0x7F take up exactly one byte.

However, only one operator currently supports UTF-8 strings: Unicode. Since string literals are defined to contain only

non-null character values, both Hex and Octal escape sequence values must be non-null values in the ASCII range 0x01 through 0xFF. For arbitrary byte data (outside the range of ASCII values), the **Buffer** object should be used instead.

Since the backslash is used as the escape character and also the namespace root prefix, any string literals that are to contain a fully qualified namepath from the root of the namespace must use the double backslash to indicate this:

```
Name (_EJD, "\\_SB.PCI0.DOCK1")
```

The double backslash is only required within quoted string literals.

Since double quotation marks are used close a string, a special escape sequence (") is used to allow quotation marks within strings. Other escape sequences are listed in the table below.

Table 19.4: ASL Escape Sequences

Escape Sequence	ASCII Character
\a	0x07 (BEL)
\b	0x08 (BS)
\f	0x0C (FF)
\n	0x0A (LF)
\r	0x0D (CR)
\t	0x09 (TAB)
\v	0x0B (VT)
\"	0x22 (")
\'	0x27 (')
\\	0x5C (\)

Since literal strings are read-only constants, the following ASL statement (for example) is not supported:

```
Store ("ABC", "DEF")
```

However, the following sequence of statements is supported:

```
Name (STR, "DEF")
...
Store ("ABC", STR)
```

19.3.3 ASL Resource Templates

ASL includes some macros for creating resource descriptors. The ResourceTemplate macro creates a Buffer in which resource descriptor macros can be listed. The ResourceTemplate macro automatically generates an End descriptor and calculates the checksum for the resource template. The format for the ResourceTemplate macro is as follows:

```
ResourceTemplate ()
{
    // List of resource macros
}
```

The following is an example of how these macros can be used to create a resource template that can be returned from a _PRS control method:

```
Name (PRS0, ResourceTemplate ()
{
```

(continues on next page)

(continued from previous page)

```

StartDependentFn (1, 1)
{
    IRQ (Level, ActiveLow, Shared) {10, 11}
    DMA (TypeF, NotBusMaster, Transfer16) {4}
    IO (Decode16, 0x1000, 0x2000, 0, 0x100)
    IO (Decode16, 0x5000, 0x6000, 0, 0x100, IO1)
}
StartDependentFn (1, 1)
{
    IRQ (Level, ActiveLow, Shared) {}
    DMA (TypeF, NotBusMaster, Transfer16){5}
    IO (Decode16, 0x3000, 0x4000, 0, 0x100)
    IO (Decode16, 0x5000, 0x6000, 0, 0x100, IO2)
}
EndDependentFn ()
})

```

Occasionally, it is necessary to change a parameter of a descriptor in an existing resource template at run-time (i.e., during a method execution.) To facilitate this, the descriptor macros optionally include a name declaration that can be used later to refer to the descriptor. When a name is declared with a descriptor, the ASL compiler will automatically create field names under the given name to refer to individual fields in the descriptor.

The offset returned by a reference to a resource descriptor field name is either in units of bytes (for 8-, 16-, 32-, and 64-bit field widths) or in bits (for all other field widths). In all cases, the returned offset is the integer offset (in either bytes or bits) of the name from the first byte (offset 0) of the parent resource template.

For example, given the above resource template, the following code changes the minimum and maximum addresses for the I/O descriptor named IO2:

```

CreateWordField (PRS0, IO2._MIN, IMIN)
Store (0xA000, IMIN)

CreateWordField (PRS0, IO2._MAX, IMAX)
Store (0xB000, IMAX)

```

The resource template macros for each of the resource descriptors are listed below, after the table that defines the resource descriptor. The resource template macros are formally defined in *ASL Macros for Resource Descriptors*

The reserved names (such as _MIN and _MAX) for the fields of each resource descriptor are defined in the appropriate table entry of the table that defines that resource descriptor.

19.3.4 ASL Macros

ASL compilers support built in macros to assist in various ASL coding operations. These macros do not have a corresponding AML opcode, but are instead fully processed by the compiler itself, and may result in the generation of AML opcodes for other ASL/AML operators. The following table lists some of the supported directives and an explanation of their function.

The ASL language provides a wide variety of data types and operators that manipulate data. It also provides mechanisms for both explicit and implicit conversion between the data types when used with ASL operators.

Each of the available ASL macros are described below.

EISAID (TextID)

Converts and compresses the 7-character text argument into its corresponding 4-byte numeric EISA ID encoding (Integer). This can be used when declaring IDs for devices that are EISA IDs.

The algorithm used to convert the TextID is as shown in the following example:

Starting with a seven character input string “PNP0303”, we want to create a DWordConst. This string contains a three character manufacturer code “PNP”, a three character hex product identifier “030”, and a one character revision identifier “3”.

The compressed manufacturer code is created as follows:

- 1) Find hex ASCII value for each letter
- 2) Subtract 40h from each ASCII value
- 3) Retain 5 least significant bits for each letter and discard remaining 0’s:

Byte 0:

Bit 7: reserved (0)

Bit 6-2: 1st character of compressed mfg code “P”

Bit 1-0: Upper 2 bits of 2nd character of mfg code “N”

Byte 1:

Bit 7-5: Lower 3 bits of 2nd character of mfg code “N”

Bit 4-0: 3rd character of mfg code “P”

Byte 2:

Bit 7-4: 1st hex digit of product number “0”

Bit 3-0: 2nd hex digit of product number “3”

Byte 3:

Bit 7-4: 3rd hex digit of product number “0”

Bit 3-0: 4th hex digit of product number “3”

For (Initialize, Predicate, Update) {TermList}

Implements a standard For() loop by converting the For() arguments and TermList into an AML While loop.

Fprintf (Target, FormatString, FormatArgs)

Converts a format string to a series of string Concatenate operations and stores the result to a Named Object (Target).

Printf (FormatString, FormatArgs)

Converts a format string to a series of string Concatenate operations and automatically stores the result to the Debug Object.

ResourceTemplate ()

Used to supply Plug and Play resource descriptor information in human readable form, which is then translated into the appropriate binary Plug and Play resource descriptor encodings in a Resource Template Buffer object. For more information about resource descriptor encodings, (See: *Resource Data Types for ACPI*).

ToPLD (PLDKeywordList)

Converts a PLD (Physical Location of Device) Keyword List into a _PLD Buffer object.

ToUUID (AsciiString)

Converts an ASCII UUID or GUID string to an encoded 128-bit Buffer object.

Unicode (StringData)

Converts a standard ASCII string to a Unicode string returned in a Buffer object.

19.3.5 ASL Data Types

ASL provides a wide variety of data types and operators that manipulate data. It also provides mechanisms for both explicit and implicit conversion between the data types when used with ASL operators.

The table below describes each of the available ASL data types.

Table 19.5: Summary of ASL Data Types

ASL Data Type	Description
[Uninitialized]	No assigned type or value. This is the type of all control method LocalX variables and unused ArgX variables at the beginning of method execution, as well as all uninitialized Package elements. Uninitialized objects must be initialized (via Store or CopyObject) before they may be used as source operands in ASL expressions.
Buffer	An array of bytes. Uninitialized elements are zero by default.
Buffer Field	Portion of a buffer created using CreateBitField, CreateByteField, CreateWordField, CreateQWordField, CreateField, or returned by the Index operator.
Debug Object	Debug output object. Formats an object and prints it to the system debug port. Has no effect if debugging is not active.
Device	Device or bus object
Event	Event synchronization object
Field Unit (within an Operation Region)	Portion of an address space, bit-aligned and of one-bit granularity. Created using Field, BankField, or IndexField.
Integer	An <i>n</i> -bit little-endian unsigned integer. In ACPI 1.0 this was 32 bits. In ACPI 2.0 and later, this is 64 bits. The Integer (DWORD) designation indicates that only the lower 32 bits have meaning and the upper 32 bits of 64-bit integers must be zero (masking of upper bits is not required).
Integer Constant	Created by the ASL terms “Zero”, “One”, “Ones”, and “Revision”.
Method	Control Method (Executable AML function)
Mutex	Mutex synchronization object
Object Reference	Reference to an object created using the RefOf, Index, or CondRefOf operators
Operation Region	Operation Region (A region within an Address Space)
Package	Collection of ASL objects with a fixed number of elements (up to 255).
Power Resource	Power Resource description object
Processor	Processor description object
RawDataBuffer	An array of bytes. Uninitialized elements are zero by default. RawDataBuffer does not contain any AML encoding bytes, only the raw bytes.
String	Null-terminated ASCII string.
Thermal Zone	Thermal Zone description object

Note

Compatibility Note: The ability to store and manipulate object references was introduced in ACPI 2.0. In ACPI 1.0 references could not be stored in variables, passed as parameters or returned from functions.

19.3.5.1 Data Type Conversion Overview

ASL provides two mechanisms to convert objects from one data type to another data type at run-time (during execution of the AML interpreter). The first mechanism, Explicit Data Type Conversion, allows the use of explicit ASL operators to convert an object to a different data type. The second mechanism, Implicit Data Type Conversion, is invoked by the AML interpreter when it is necessary to convert a data object to an expected data type before it is used or stored.

The following general rules apply to data type conversions:

- Input parameters are always subject to implicit data type conversion (also known as implicit source operand conversion) whenever the operand type does not match the expected input type.
- Output (target) parameters for all operators except the explicit data conversion operators are subject to implicit data type conversion (also known as implicit result object conversion) whenever the target is an existing named object or named field that is of a different type than the object to be stored.
- Output parameters for the explicit data conversion operators, as well as output parameters that refer to a method local or argument (LocalX or ArgX) are not subject to implicit type conversion.

Both of these mechanisms (explicit and implicit conversion) are described in detail in the sections that follow.

19.3.5.2 Explicit Data Type Conversions

The following ASL operators are provided to explicitly convert an object from one data type to another:

ToBuffer

Convert an Integer, String, or Buffer to an object of type Buffer

ToDecimalString

Convert an Integer, String, or Buffer to an object of type String. The string contains the ASCII representation of the decimal value of the source operand.

ToHexString

Convert an Integer, String, or Buffer to an object of type String. The string contains the ASCII representation of the hexadecimal value of the source operand.

ToInteger

Convert an Integer, String, or Buffer to an object of type Integer.

ToString

Copy directly and convert a Buffer to an object of type String.

The following ASL operator is provided to copy and transfer objects with an explicit result conversion of the type of the target to match the type of the source object:

CopyObject

Explicitly store a copy of the operand object to the target name. No implicit type conversion is performed. (This operator is used to avoid the implicit conversion inherent in the ASL Store operator.)

19.3.5.3 Implicit Data Type Conversions

Automatic or Implicit type conversions can take place at two different times during the execution of an ASL operator. First, it may be necessary to convert one or more of the source operands to the data type(s) expected by the ASL operator. Second, the result of the operation may require conversion before it is stored into the destination. (Many of the ASL operators can store their result optionally into an object specified by the last parameter. In these operators, if the destination is specified, the action is exactly as if a Store operator had been used to place the result in the destination.)

Such data conversions are performed by an AML interpreter during execution of AML code and are known collectively as Implicit Operand Conversions. As described briefly above, there are two different types of implicit operand conversion:

1. Conversion of a source operand from a mismatched data type to the correct data type required by an ASL operator, called Implicit Source Conversion. This conversion occurs when a source operand must be converted to the operand type expected by the operator. Any or all of the source operands may be converted in this manner before the execution of the ASL operator can proceed.
2. Conversion of the result of an operation to the existing type of a target operand before it is stored into the target operand, called Implicit Result Conversion. This conversion occurs when the target is a fixed type such as a named object or a field. When storing to a method Local or Arg, no conversion is performed or required because these data types are of variable type (the store simply overwrites any existing object and the existing type).

The following ASL operator is provided to copy and transfer objects with an implicit result conversion to the existing type of the target object:

Store

Store a copy of the operand object to the target name. Implicit result conversion is performed if the target name is of a fixed data type (see above). However, Stores to method locals and arguments do not perform implicit conversion and are therefore the same as using CopyObject.

19.3.5.4 Implicit Source Operand Conversion

During the execution of an ASL operator, each source operand is processed by the AML interpreter as follows:

- If the operand is of the type expected by the operator, no conversion is necessary.
- If the operand type is incorrect, attempt to convert it to the proper type.
- For the Concatenate operator and logical operators (LEqual, LGreater, LGreaterEqual, LLess, LLessEqual, and LNotEqual), the data type of the first operand dictates the required type of the second operand, and for Concatenate only, the type of the result object. (The second operand is implicitly converted, if necessary, to match the type of the first operand.)
- If conversion is impossible, abort the running control method and issue a fatal error.

An implicit source conversion will be attempted anytime a source operand contains a data type that is different than the type expected by the operator. For example:

```
Store ("5678", Local1)
Add (0x1234, Local1, BUF1)
```

In the Add statement above, Local1 contains a String object and must undergo conversion to an Integer object before the Add operation can proceed.

In some cases, the operator may take more than one type of operand (such as Integer and String). In this case, depending on the type of the operand, the highest priority conversion is applied. The table below describes the source operand conversions available. For example:

```
Store (Buffer (1) {}, Local0)
Name (ABCD, Buffer (10) {1, 2, 3, 4, 5, 6, 7, 8, 9, 0})
CreateDWordField (ABCD, 2, XYZ)
Name (MNOP, "1234")
Concatenate (XYZ, MNOP, Local0)
```

The Concatenate operator can take an Integer, Buffer or String for its first two parameters and the type of the first parameter determines how the second parameter will be converted. In this example, the first parameter is of type Buffer Field (from the CreateDWordField operator). What should it be converted to: Integer, Buffer or String? According to the table *Object Conversion Rules* the highest priority conversion is to Integer. Therefore, both of the following objects will be converted to Integers:

```
XYZ (0x05040302)
MNOP (0x31, 0x32, 0x33, 0x34)
```

And will then be joined together and the resulting type and value will be:

```
Buffer (0x02, 0x03, 0x04, 0x05, 0x31, 0x32, 0x33, 0x34)
```

19.3.5.5 Implicit Result Object Conversion

For all ASL operators that generate and store a result value (including the Store operator), the result object is processed and stored by the AML interpreter as follows:

- If the ASL operator is one of the explicit conversion operators (ToString, ToInteger, etc., and the CopyObject operator), no conversion is performed. (In other words, the result object is stored directly to the target and completely overwrites any existing object already stored at the target.)
- If the target is a method local or argument (LocalX or ArgX), no conversion is performed and the result is stored directly to the target.
- If the target is a fixed type such as a named object or field object, an attempt is made to convert the source to the existing target type before storing.
- If conversion is impossible, abort the running control method and issue a fatal error.

An implicit result conversion can occur anytime the result of an operator is stored into an object that is of a fixed type. For example:

```
Name (BUF1, Buffer (10))
Add (0x1234, 0x789A, BUF1)
```

Since BUF1 is a named object of fixed type *Buffer*, the Integer result of the Add operation must be converted to a Buffer before it is stored into BUF1.

19.3.5.6 Data Types and Type Conversions

The following table lists the available ASL data types and the available data type conversions (if any) for each. The entry for each data type is fully cross-referenced, showing both the types to which the object may be converted as well as all other types that may be converted to the data type.

The allowable conversions apply to both explicit and implicit conversions.

Table 19.6: Data Types and Type Conversions

ASL Data Type	Can be implicitly or explicitly converted to these Data Types (In priority order)	Can be implicitly or explicitly converted from these Data Types
[Uninitialized]	None. Causes a fatal error when used as a source operand in any ASL statement.	Integer, String, Buffer, Package, Object Reference
Buffer	Integer, String, Debug Object	Integer, String
Buffer Field	Integer, Buffer, String, Debug Object	Integer, Buffer, String
Debug Object	None. Causes a fatal error when used as a source operand in any ASL statement.	Integer, String, Buffer, Package, Field Unit, Buffer Field
Device	None	None
Event	None	None
Field Unit (within an Operation Region)	Integer, Buffer, String, Debug Object	Integer, Buffer, String
Integer	Buffer, Buffer Field, Field Unit, String, Debug Object	Buffer, String
Integer Constant	Integer, Debug Object	None. Also, storing any object to a constant is a no-op, not an error.
Method	None	None
Mutex	None	None
Object Reference	None	None
Operation Region	None	None
Package	Debug Object	None
String	Integer, Buffer, Debug Object	Integer, Buffer
Power Resource	None	None
RawDataBuffer	None	None
Thermal Zone	None	None

19.3.5.7 Data Type Conversion Rules

The following table presents the detailed data conversion rules for each of the allowable data type conversions. These conversion rules are implemented by the AML Interpreter and apply to all conversion types – explicit conversions, implicit source conversions, and implicit result conversions.

Table 19.7: Object Conversion Rules

To convert from an object of this Data Type	To an object of this Data Type	This action is performed by the AML Interpreter
Buffer	Buffer Field	The contents of the buffer are copied to the Buffer Field. If the buffer is smaller than the size of the buffer field, it is zero extended. If the buffer is larger than the size of the buffer field, the upper bits are truncated. Compatibility Note: This conversion was first introduced in ACPI 2.0. The behavior in ACPI 1.0 was undefined.
Buffer	Debug Object	Each buffer byte is displayed as a hexadecimal integer, delimited by spaces and/or commas.
Buffer	Field Unit	The entire contents of the buffer are copied to the Field Unit. If the buffer is larger (in bits) than the size of the Field Unit, it is broken into pieces and completely written to the Field Unit, lower chunks first. If the buffer (or the last piece of the buffer, if broken up) is smaller than the size of the Field Unit, it is zero extended before being written.

continues on next page

Table 19.7 – continued from previous page

To convert from an object of this Data Type	To an object of this Data Type	This action is performed by the AML Interpreter
Buffer	Integer	If no integer object exists, a new integer is created. The contents of the buffer are copied to the Integer, starting with the least-significant bit and continuing until the buffer has been completely copied — up to the maximum number of bits in an Integer. The size of an Integer is indicated by the Definition Block table header's Revision field. A Revision field value less than 2 indicates that the size of an Integer is 32 bits. A value greater than or equal to 2 signifies that the size of an Integer is 64 bits. If the buffer is smaller than the size of an integer, it is zero extended. If the buffer is larger than the size of an integer, it is truncated. Conversion of a zero-length buffer to an integer is not allowed.
Buffer	String	If no string object exists, a new string is created. If the string already exists, it is completely overwritten and truncated or extended to accommodate the converted buffer exactly. The entire contents of the buffer are converted to a string of two-character hexadecimal numbers, each separated by a space. A zero-length buffer will be converted to a null (zero-length) string.
Buffer Field	[See the Integer and Buffer Rules]	If the Buffer Field is smaller than or equal to the size of an Integer (in bits), it will be treated as an Integer. Otherwise, it will be treated as a buffer. The size of an Integer is indicated by the Definition Block table header's Revision field. A Revision field value less than 2 indicates that the size of an Integer is 32 bits. A value greater than or equal to 2 signifies that the size of an Integer is 64 bits. (See the conversion rules for the Integer and Buffer data types.)
Field Unit	[See the Integer and Buffer Rules]	If the Field Unit is smaller than or equal to the size of an Integer (in bits), it will be treated as an Integer. If the Field Unit is larger than the size of an Integer, it will be treated as a Buffer. The size of an Integer is indicated by the Definition Block table header's Revision field. A Revision field value less than 2 indicates that the size of an Integer is 32 bits. A value greater than or equal to 2 signifies that the size of an Integer is 64 bits. (See the conversion rules for the Integer and Buffer data types.)
Integer	Buffer	If no buffer object exists, a new buffer object is created based on the size of the integer (4 bytes for 32-bit integers and 8 bytes for 64-bit integers). If a buffer object already exists, the Integer overwrites the entire Buffer object. If the integer requires more bits than the size of the Buffer, then the integer is truncated before being copied to the Buffer. If the integer contains fewer bits than the size of the buffer, the Integer is zero-extended to fill the entire buffer.
Integer	Buffer Field	The Integer overwrites the entire Buffer Field. If the integer is smaller than the size of the buffer field, it is zero-extended. If the integer is larger than the size of the buffer field, the upper bits are truncated. Compatibility Note: This conversion was first introduced in ACPI 2.0. The behavior in ACPI 1.0 was undefined.
Integer	Debug Object	The integer is displayed as a hexadecimal value.
Integer	Field Unit	The Integer overwrites the entire Field Unit. If the integer is smaller than the size of the buffer field, it is zero-extended. If the integer is larger than the size of the buffer field, the upper bits are truncated.

continues on next page

Table 19.7 – continued from previous page

To convert from an object of this Data Type	To an object of this Data Type	This action is performed by the AML Interpreter
Integer	String	If no string object exists, a new string object is created based on the size of the integer (8 characters for 32-bit integers and 16 characters for 64-bit integers). If the string already exists, it is completely overwritten and truncated or extended to accommodate the converted integer exactly. In either case, the entire integer is converted to a string of hexadecimal ASCII characters.
Package	Package	If no package object exists, a new package object is created. If the package already exists, it is completely overwritten and truncated or extended to accommodate the source package exactly. Any and all existing valid (non-null) package elements of the target package are deleted, and the entire contents of the source package are copied into the target package.
Package	Debug Object	Each element of the package is displayed based on its type.
String	Buffer	If no buffer object exists, a new buffer object is created. If a buffer object already exists, it is completely overwritten. If the string is longer than the buffer, the string is truncated before copying. If the string is shorter than the buffer, the remaining buffer bytes are set to zero. In either case, the string is treated as a buffer, with each ASCII string character copied to one buffer byte, including the null terminator. A null (zero-length) string will be converted to a zero-length buffer.
String	Buffer Field	The string is treated as a buffer. If this buffer is smaller than the size of the buffer field, it is zero extended. If the buffer is larger than the size of the buffer field, the upper bits are truncated. Compatibility Note: This conversion was first introduced in ACPI 2.0. The behavior in ACPI 1.0 was undefined.
String	Debug Object	Each string character is displayed as an ASCII character.
String	Field Unit	Each character of the string is written, starting with the first, to the Field Unit. If the Field Unit is less than eight bits, then the upper bits of each character are lost. If the Field Unit is greater than eight bits, then the additional bits are zeroed.
String	Integer	If no integer object exists, a new integer is created. The integer is initialized to the value zero and the ASCII string is interpreted as a hexadecimal constant. Each string character is interpreted as a hexadecimal value ('0'-'9', 'A'-'F', 'a'-'f'), starting with the first character as the most significant digit, and ending with the first non-hexadecimal character, end-of-string, or when the size of an integer is reached (8 characters for 32-bit integers and 16 characters for 64-bit integers). Note: the first non-hex character terminates the conversion without error, and a "0x" prefix is not allowed. Conversion of a null (zero-length) string to an integer is not allowed.

19.3.5.8 Rules for Storing and Copying Objects

The table below lists the actions performed when storing objects to different types of named targets. ASL provides the following types of “store” operations:

- The Store operator is used to explicitly store an object to a location, with implicit conversion support of the source object.
- Many of the ASL operators can store their result optionally into an object specified by the last parameter. In these operators, if the destination is specified, the action is exactly as if a Store operator had been used to place the result in the destination.
- The CopyObject operator is used to explicitly store a copy of an object to a location, with no implicit conversion support.

Table 19.8: Object Storing and Copying Rules

When Storing an object of any datatype to this type of Target location	This action is performed by the Store operator or any ASL operator with a Target operand	This action is performed by the CopyObject operator
Method ArgX variable	The object is copied to the destination with no conversion applied, with one exception. If the ArgX contains an Object Reference, an automatic de-reference occurs and the object is copied to the target of the Object Reference instead of overwriting the contents of ArgX.	The object is copied to the destination with no conversion applied, with one exception. If the ArgX contains an Object Reference, an automatic de-reference occurs and the object is copied to the target of the Object Reference instead of overwriting the contents of ArgX.
Method LocalXvariable	The object is copied to the destination with no conversion applied. Even if LocalX contains an Object Reference, it is overwritten.	The object is copied to the destination with no conversion applied. Even if LocalX contains an Object Reference, it is overwritten.
Field Unit or BufferField	The object is copied to the destination after implicit result conversion is applied.	Fields permanently retain their type and cannot be changed. Therefore, CopyObject can only be used to copy an object of type Integer or Buffer to fields.
Named data object	The object is copied to the destination after implicit result conversion is applied to match the existing type of the named location.	The object and type are copied to the named location.

19.3.5.8.1 ArgX Objects

1. Read from ArgX parameters

- **ObjectReference** - Automatic dereference, return the target of the reference. Use of DerefOf returns the same.
- **Buffer** - Return the Buffer. Can create an Index, Field, or Reference to the buffer.
- **Package** - Return the Package. Can create an Index or Reference to the package.
- **All other object types** - Return the object.

Example method invocation for the table below:

```
MTHD (RefOf (Obj), Buf, Pkg, Obj)
```

Table 19.9: Reading from ArgX Objects

Parameter	MTHD ArgX Type	Read operation on ArgX	Result of read
RefOf (Obj),	Reference to object Obj	Store (Arg0, ...)	Obj
"	"	CopyObject (Arg0, ...)	Obj
"	"	DerefOf (Arg0)	Obj
Buf,	Buffer	Store (Arg1, ...)	Buf
"	"	CopyObject (Arg1, ...)	Buf
"	"	Index (Arg1, ...)	Index (Buf)
"	"	Field (Arg1, ...)	Field (Buf)
Pkg	Package	Store (Arg2, ...)	Pkg
"	"	CopyObject (Arg2, ...)	Pkg
"	"	Index (Arg2, ...)	Index (Pkg)
Obj	All other object types	Store (Arg3, ...)	Obj
"	"	CopyObject (Arg3, ...)	Obj

2. Store to ArgX parameters

- **ObjectReference objects** - Automatic dereference, copy the object and overwrite the final target.
- **All other object types** - Copy the object and overwrite the ArgX variable. (Direct writes to buffer or package ArgX parameters will also simply overwrite ArgX)

Table 19.10: Writing to ArgX Objects

Current type of ArgX	Object to be written	Write operation on ArgX	Result of write (in ArgX)
RefOf (OldObj)	Obj (any type)	Store (... , ArgX)	RefOf (copy of Obj)
"	"	CopyObject (... , ArgX)	RefOf (copy of Obj)
All other object types	Obj (Any type)	Store (... , ArgX)	Copy of Obj
"	"	CopyObject (... , ArgX)	Copy of Obj

Note

RefOf (ArgX) returns a reference to ArgX.

19.3.5.8.2 LocalX Objects

1. Read from LocalX variables

- **ObjectReference** - If performing a DerefOf return the target of the reference. Otherwise, return the reference.
- **All other object types** - Return a the object

Table 19.11: Reading from LocalX Objects

Current LocalX Type	Read operation on LocalX	Result of read
RefOf (Obj)	Store (LocalX, ...)	RefOf (Obj)
"	CopyObject (LocalX, ...)	RefOf (Obj)
"	DerefOf (LocalX)	Obj
Obj (All other types)	Store (LocalX, ...)	Obj

continues on next page

Table 19.11 – continued from previous page

Current LocalX Type	Read operation on LocalX	Result of read
"	CopyObject (LocalX, ...)	Obj

2. Store to LocalX variables

- **All object types** - Delete any existing object in LocalX first, then store a copy of the object.

Table 19.12: Writing to LocalX Objects

Current Type	LocalX-Object to be written	Write operation on LocalX	Result of write (in LocalX)
All object types	Obj (any type)	Store (... , LocalX)	Copy of Obj
"	"	CopyObject (... , LocalX)	Copy of Obj

19.3.5.8.3 Named Objects

1. Read from Named object

- **ObjectReference** - If performing a DerefOf return the target of the reference. Otherwise, return the reference.
- **All other object types** - Return the object

Table 19.13: Reading from Named Objects

Current NAME Type	Read operation on NAME	Result of read
RefOf (Obj)	Store (NAME, ...)	RefOf (Obj)
"	CopyObject (NAME, ...)	RefOf (Obj)
"	DerefOf (NAME)	Obj
Obj (All other types)	Store (NAME, ...)	Obj
"	CopyObject (NAME, ...)	Obj

2. Store to Named object

- **All object types** - Delete any existing object in NAME first, then store a copy of the object. The Store operator will perform an implicit conversion to the existing type in NAME. CopyObject does not perform an implicit store.

Table 19.14: Writing to Named Objects

Current Type	NAME	Object to be written	Write operation on NAME	Result of write (in NAME)
Any (Any Type)		Obj (Any type)	Store (... , NAME)	Copy of Obj (converted to match existing type of NAME)
"		"	CopyObject (... , NAME)	Copy of Obj (No conversion)

19.4 ASL Operators Summary

Table 19.15: ASL Operators Summary List

Operator Name	Description
AccessAs	Change Field Access
Acquire	Acquire a mutex
Add	Integer Add
Alias	Define a name alias
And	Integer Bitwise And
ArgX	Method argument data objects
BankField	Declare fields in a banked configuration object
Break	Continue following the innermost enclosing While
BreakPoint	Used for debugging, stops execution in the debugger
Buffer	Declare Buffer object
Case	Expression for conditional execution
Concatenate	Concatenate two strings, integers or buffers
ConcatenateResTemplate	Concatenate two resource templates
CondRefOf	Conditional reference to an object
Connection	Declare Field Connection Attributes
Continue	Continue innermost enclosing While loop
CopyObject	Copy and existing object
CreateBitField	Declare a bit field object of a buffer object
CreateByteField	Declare a byte field object of a buffer object
CreateDWordField	Declare a DWord field object of a buffer object
CreateField	Declare an arbitrary length bit field of a buffer object
CreateQWordField	Declare a QWord field object of a buffer object
CreateWordField	Declare a Word field object of a buffer object
DataTableRegion	Declare a Data Table Region
Debug	Debugger output
Decrement	Decrement an Integer
Default	Default execution path in Switch()
DefinitionBlock	Declare a Definition Block
DerefOf	Dereference an object reference
Device	Declare a bus/device object
Divide	Integer Divide
DMA	DMA Resource Descriptor macro
DWordIO	DWord IO Resource Descriptor macro
DWordMemory	DWord Memory Resource Descriptor macro
DWordSpace	DWord Space Resource Descriptor macro
EisaId	EISA ID String to Integer conversion macro
Else	Alternate conditional execution
ElseIf	Conditional execution
EndDependentFn	End Dependent Function
Event	Resource Descriptor macro
ExtendedIO	Declare an event synchronization object
ExtendedMemory	Extended IO Resource Descriptor macro
ExtendedSpace	Extended Space Resource Descriptor macro
External	Declare external objects

continues on next page

Table 19.15 – continued from previous page

Operator Name	Description
Fatal	Fatal error check
Field	Declare fields of an operation region object
FindSetLeftBit	Index of first least significant bit set
FindSetRightBit	Index of first most significant bit set
FixedDMA	Fixed DMA Resource Descriptor macro
FixedIO	Fixed I/O Resource Descriptor macro
Fprintf	Stores formatted string to a Named Object
FromBCD	Convert from BCD to numeric
Function	Declare control method
GpioInt	GPIO Interrupt Connection Resource Descriptor macro
GpioIo	GPIO I/O Connection Resource Descriptor macro
I2CSerialBusV2	I2C Serialbus Connection Resource Descriptor (Version 2) macro
If	Conditional execution
Include	Include another ASL file
Increment	Increment a Integer
Index	Indexed Reference to member object
IndexField	Declare Index/Data Fields
Interrupt	Interrupt Resource Descriptor macro
IO	IO Resource Descriptor macro
IRQ	Interrupt Resource Descriptor macro
IRQNoFlags	Short Interrupt Resource Descriptor macro
LAnd	Logical And
LEqual	Logical Equal
LGreater	Logical Greater
LGreaterEqual	Logical Not less
LLess	Logical Less
LLessEqual	Logical Not greater
LNot	Logical Not
LNotEqual	Logical Not equal
Load	Load differentiating definition block
LoadTable	Load Table from RSDT/XSDT
LocalX	Method local data objects
LOr	Logical Or
Match	Search for match in package array
Memory24	Memory Resource Descriptor macro
Memory32	Memory Resource Descriptor macro
Memory32Fixed	Memory Resource Descriptor macro
Method	Declare a control method
Mid	Return a portion of buffer or string
Mod	Integer Modulo
Multiply	Integer Multiply
Mutex	Declare a mutex synchronization object
Name	Declare a Named object
NAnd	Integer Bitwise Nand
NoOp	No operation
NOr	Integer Bitwise Nor
Not	Integer Bitwise Not
Notify	Notify Object of event
ObjectType	Type of object
Offset	Set Field Offset within operation range

continues on next page

Table 19.15 – continued from previous page

Operator Name	Description
One	Constant One Object (1)
Ones	Constant Ones Object (-1)
OperationRegion	Declare an operational region
Or	Integer Bitwise Or
Package	Declare a package object
PowerResource	Declare a power resource object
Printf	Stores formatted string to Debug Object
Processor	Declare a processor package
QWordIO	QWord IO Resource Descriptor macro
QWordMemory	QWord Memory Resource Descriptor macro
QWordSpace	QWord Space Resource Descriptor macro
RawDataBuffer	Declare a RawDataBuffer
RefOf	Create Reference to an object
Register	Generic register Resource Descriptor macro
Release	Release a synchronization object
Reset	Reset a synchronization object
ResourceTemplate	Resource to buffer conversion macro
Return	Return from method execution
Revision	Constant revision object
Scope	Open named scope
ShiftLeft	Integer shift value left
ShiftRight	Integer shift value right
Signal	Signal a synchronization object
SizeOf	Get the size of a buffer, string, or package
Sleep	Sleep n milliseconds (yields the processor)
SPISerialbusV2	SPI Serialbus Connection Resource Descriptor (Version 2) macro
Stall	Delay n microseconds (does not yield the processor)
StartDependentFn	Start Dependent Function Resource Descriptor macro
StartDependentFnNoPri	Start Dependent Function Resource Descriptor macro
Store	Store object Integer
Subtract	Subtract
Switch	Select code to execute based on expression value
ThermalZone	Declare a thermal zone package.
Timer	Get 64-bit timer value
ToBCD	Convert Integer to BCD
ToBuffer	Convert data type to buffer
ToDecimalString	Convert data type to decimal string
ToHexString	Convert data type to hexadecimal string
ToInteger	Convert data type to integer
ToPLD	Converts a PLD Keyword List into a _PLD buffer
ToString	Copy ASCII string from buffer
ToUUID	Convert ASCII string to UUID
Unicode	String to Unicode conversion macro
UARTSerialBusV2	UART SerialBus Connection Resource Descriptor (version2) macro
VendorLong	Vendor Resource Descriptor
VendorShort	Vendor Resource Descriptor
Wait	Wait on an Event
While	Conditional loop
WordBusNumber	Word Bus number Resource Descriptor macro
WordIO	Word IO Resource Descriptor macro

continues on next page

Table 19.15 – continued from previous page

Operator Name	Description
WordSpace	Word Space Resource Descriptor macro
Xor	Integer Bitwise Xor
Zero	Constant Zero object 0

19.5 ASL Operator Summary by Type

Table 19.16: ASL compiler controls

Operator Name	Description
External	Declare external objects
Include	Include another ASL file

Table 19.17: ACPI table management

Operator Name	Description
DefinitionBlock	Load definition block
Declare a Definition Block	LoadTable
Load	Load Table from RSDT/XSDT

Table 19.18: Miscellaneous named object creation

Operator Name	Description
Alias	Define a name alias
Buffer	Declare Buffer object
Device	Declare a bus/device object
Function	Declare a control method
Method	Declare a control method
Name	Declare a Named object
Package	Declare a package object
PowerResource	Declare a power resource object
Processor	Declare a processor package
RawDataBuffer	Declare a RawDataBuffer
Scope	Open named scope
ThermalZone	Declare a thermal zone package

Table 19.19: Operation Regions and Fields

Operator Name	Description
AccessAs	Change Field Access
BankField	Declare fields in a banked configuration object
Connection	Declare Field Connection Attributes
DataTableRegion	Declare a Data Table Region
Field	Declare fields of an operation region object
IndexField	Declare Index/Data Fields

continues on next page

Table 19.19 – continued from previous page

<i>Operator Name</i>	<i>Description</i>
Offset	Set Field offset within operation region
OperationRegion	Declare an operational region

Table 19.20: **Buffer Fields**

<i>Operator Name</i>	<i>Description</i>
CreateBitField	Declare a bit field object of a buffer object
CreateByteField	Declare a byte field object of a buffer object
CreateDWordField	Declare a DWord field object of a buffer object
CreateField	Declare an arbitrary length bit field of a buffer object
CreateQWordField	Declare a QWord field object of a buffer object
CreateWordField	Declare a Word field object of a buffer object

Table 19.21: **Synchronization**

<i>Operator Name</i>	<i>Description</i>
Acquire	Acquire a mutex
Event	Declare an event synchronization object
Mutex	Declare a mutex synchronization object
Notify	Notify Object of event
Release	Release a synchronization object
Reset	Reset a synchronization object
Signal	Signal a synchronization object
Wait	Wait on an Event

Table 19.22: **Object references**

<i>Operator Name</i>	<i>Description</i>
CondRefOf	Conditional reference to an object
DerefOf	Dereference an object reference
RefOf	Create Reference to an object

Table 19.23: **Integer arithmetic**

<i>Operator Name</i>	<i>Description</i>
Add	Integer Add
And	Integer Bitwise And
Decrement	Decrement an Integer
Divide	Integer Divide
FindSetLeftBit	Index of first least significant bit set
FindSetRightBit	Index of first most significant bit set
Increment	Increment a Integer
Mod	Integer Modulo
Multiply	Integer Multiply
NAnd	Integer Bitwise Nand
NOr	Integer Bitwise Nor

continues on next page

Table 19.23 – continued from previous page

<i>Operator Name</i>	<i>Description</i>
Not	Integer Bitwise Not
Or	Integer Bitwise Or
ShiftLeft	Integer shift value left
ShiftRight	Integer shift value right I
Subtract	Integer Subtract
Xor	Integer Bitwise Xor

Table 19.24: **Logical operators**

<i>Operator Name</i>	<i>Description</i>
LAnd	Logical And
LEqual	Logical Equal
LGreater	Logical Greater
LGreaterEqual	Logical Not less
LLess	Logical Less
LLessEqual	Logical Not greater
LNot	Logical Not
LNotEqual	Logical Not equal
LOr	Logical Or

Table 19.25: **Method execution control**

<i>Operator Name</i>	<i>Description</i>
Break	Continue following the innermost enclosing While
BreakPoint	Used for debugging, stops execution in the debugger
Case	Expression for conditional execution
Continue	Continue innermost enclosing While loop
Default	Default execution path in Switch()
Else	Alternate conditional execution
ElseIf	Conditional execution
Fatal	Fatal error check
If	Conditional execution
NoOp	No operation
Return	Return from method execution
Sleep	Sleep in milliseconds (yields the processor)
Stall	Delay in microseconds (does not yield the processor)
Switch	Select code to execute based on expression value
While	Conditional loop

Table 19.26: **Data type conversion and manipulation**

<i>Operator Name</i>	<i>Description</i>
Concatenate	Concatenate two strings, integers or buffers
CopyObject	Copy and existing object
Debug	Debugger output
EisaId	EISA ID String to Integer conversion macro
Fprintf	Stores formatted string to a Named Object

continues on next page

Table 19.26 – continued from previous page

<i>Operator Name</i>	<i>Description</i>
FromBCD	Convert from BCD to numeric
Index	Indexed Reference to member object
Match	Search for match in package array
Mid	Return a portion of buffer or string
ObjectType	Type of object
Printf	Stores formatted string to Debug Object
SizeOf	Get the size of a buffer, string, or package
Store	Store object
Timer	Get 64-bit timer value
ToBCD	Convert Integer to BCD
ToBuffer	Convert data type to buffer
ToDecimalString	Convert data type to decimal string
ToHexString	Convert data type to hexadecimal string
ToInteger	Convert data type to integer
ToPLD	Converts a PLD Keyword List into a _PLD buffer
ToString	Copy ASCII string from buffer
ToUUID	Convert ASCII string to UUID
Unicode	String to Unicode conversion macro

Table 19.27: Resource Descriptor macros

<i>Operator Name</i>	<i>Description</i>
ConcatenateResTemplate	Concatenate two resource templates
DMA	DMA Resource Descriptor macro
DWordIO	DWord IO Resource Descriptor macro
DWordMemory	DWord Memory Resource Descriptor macro
DWordSpace	DWord Space Resource Descriptor macro
EndDependentFn	End Dependent Function Resource Descriptor macro
ExtendedIO	Extended I/O Resource Descriptor macro
ExtendedMemory	Extended Memory Resource Descriptor macro
ExtendedSpace	Extended Space Resource Descriptor macro
FixedDMA	Fixed DMA resource Descriptor macro
FixedIO	Fixed I/O Resource Descriptor macro
GpioInt	GPIO Interrupt Connection Resource Descriptor macro
GpioIO	GPIO IO Connection Resource Descriptor macro
I2CSerialBusV2	I2C SerialBus Connection Resource Descriptor (Version 2) macro
Interrupt	Interrupt Resource Descriptor macro
IO	IO Resource Descriptor macro
IRQ	Interrupt Resource Descriptor macro
IRQNoFlags	Short Interrupt Resource Descriptor macro
Memory24	Memory Resource Descriptor macro
Memory32	Memory Resource Descriptor macro
Memory32Fixed	Memory Resource Descriptor macro
QWordIO	QWord IO Resource Descriptor macro
QWordMemory	QWord Memory Resource Descriptor macro
QWordSpace	QWord Space Resource Descriptor macro
Register	Generic register Resource Descriptor macro
ResourceTemplate	Resource to buffer conversion macro
SPISerialBusV2	SPI SerialBus Connection Resource Descriptor (Version 2) macro

continues on next page

Table 19.27 – continued from previous page

<i>Operator Name</i>	<i>Description</i>
StartDependentFn Start-DependentFnNoPri	Start Dependent Function Resource Descriptor macro
UARTSerialBusV2	UART SerialBus Connection Resource Descriptor (Version 2) macro
VendorLong	Vendor Resource Descriptor
VendorShort	Vendor Resource Descriptor
WordBusNumber	Word Bus number Resource Descriptor macro
WordIO	Word IO Resource Descriptor macro
WordSpace	Word Space Resource Descriptor macro

Table 19.28: **Constants**

<i>Operator Name</i>	<i>Description</i>
One	Constant One Object (1)
Ones	Constant Ones Object (-1)
Revision	Constant revision object
Zero	Constant Zero object (0)

Table 19.29: **Control method objects**

<i>Operator Name</i>	<i>Description</i>
ArgX	Method argument data objects
LocalX	Method local data objects

19.6 ASL Operator Reference

This section describes each of the ASL operators. The syntax for each operator is given, with a description of each argument and an overall description of the operator behavior. Example ASL code is provided for the more complex operators.

ASL operators can be categorized as follows:

- Named Object creation
- Method execution control (If, Else, While, etc.)
- Integer math
- Logical operators
- Resource Descriptor macros
- Object conversion
- Utility/Miscellaneous

19.6.1 AccessAs (Change Field Unit Access)

Syntax:

```
AccessAs (AccessType, AccessAttribute)
AccessAs (AccessType, AccessAttribute (AccessLength))
```

Arguments

AccessType is an *AccessTypeKeyword* that specifies the type of access desired (**ByteAcc**, **WordAcc**, etc.). *AccessAttribute* is an optional argument of type *AccessAttributeKeyword* that specifies additional protocols to be used, such as **AttribQuick**, **AttribSendReceive**, etc. *AccessLength* is a required argument for some of the Access Attributes.

Description

The **AccessAs** operator is used within a *FieldList* to specify the Access Type, Access Attributes, and Access Length for the remaining *FieldUnits* within the list (or until another **AccessAs** operator is encountered.) It allows FieldUnits to have different access types within a single Field definition.

Supported *AccessTypes*:

- AnyAcc
- ByteAcc
- WordAcc
- DWordAcc
- QWordAcc
- BufferAcc

Supported simple *AccessAttributes* (with SMBus synonyms):

- AttribQuick (*SMBQuick*)
- AttribSendReceive (*SMBSendReceive*)
- AttribByte (*SMBByte*)
- AttribWord (*SMBWord*)
- AttribBlock (*SMBBlock*)
- AttribProcessCall (*SMBProcessCall*)
- AttribBlockProcessCall (*SMBBlockProcessCall*)

Access Attributes that require an *AccessLength* argument:

- AttribBytes (*AccessLength*)
- AttribRawBytes (*AccessLength*)
- AttribRawProcessBytes (*AccessLength*)

19.6.2 Acquire (Acquire a Mutex)

Syntax:

```
Acquire (SyncObject, TimeoutValue) => Boolean
```

Arguments

SyncObject must be a mutex synchronization object. *TimeoutValue* is evaluated as an Integer.

Description

Ownership of the Mutex is obtained. If the Mutex is already owned by a different invocation, the current execution thread is suspended until the owner of the Mutex releases it or until at least *TimeoutValue* milliseconds have elapsed. A Mutex can be acquired more than once by the same invocation.

Note

For Mutex objects referenced by a `_DLM` object, the host OS may also contend for ownership.

This operation returns **True** if a timeout occurred and the mutex ownership was not acquired. A *TimeoutValue* of 0xFFFF (or greater) indicates that there is no timeout and the operation will wait indefinitely.

19.6.3 Add (Integer Add)

Syntax:

```
Add (Addend1, Addend2, Result) => Integer
Result = Addend1 + Addend2 => Integer
Result += Addend => Integer
```

Arguments

Addend1 and *Addend2* are evaluated as Integers.

Description

The operands are added and the result is optionally stored into *Result*. Overflow conditions are ignored and the result of overflows simply loses the most significant bits.

19.6.4 Alias (Declare Name Alias)

Syntax:

```
Alias (SourceObject, AliasObject)
```

Arguments

SourceObject is any named object. *AliasObject* is a NameString.

Description

Creates a new object named *AliasObject* that refers to and acts exactly the same as *SourceObject*.

AliasObject is created as an alias of *SourceObject* in the namespace. The *SourceObject* name must already exist in the namespace. If the alias is to a name within the same definition block, the *SourceObject* name must be logically ahead of this definition in the block.

Example:

The following example shows the use of an **Alias** term:

```
Alias (\SUS.SET.EVEN, SSE)
```

19.6.5 And (Integer Bitwise And)**Syntax:**

```
And (Source1, Source2, Result) => Integer
Result = Source1 & Source2 => Integer
Result &= Source => Integer
```

Arguments

Source1 and *Source2* are evaluated as Integers.

Description

A bitwise AND is performed and the result is optionally stored into *Result*.

19.6.6 Argx (Method Argument Data Objects)**Syntax:**

```
Arg0 | Arg1 | Arg2 | Arg3 | Arg4 | Arg5 | Arg6
```

Description

Up to 7 argument-object references can be passed to a control method. On entry to a control method, only the argument objects that are passed are usable.

19.6.7 BankField (Declare Bank/Data Field)**Syntax:**

```
BankField (RegionName, BankName, BankValue, AccessType, LockRule,
UpdateRule) {FieldUnitList}
```

Arguments

RegionName is evaluated as a NameString, and is the name of the host Operation Region.

BankName is evaluated as a NameString, and is the name of the bank selection register.

BankValue is the bank selection ID (Integer) that is written to the *BankName* register before the *FieldUnitList* is accessed.

The *AccessType*, *LockRule*, *UpdateRule*, and *FieldUnitList* are the same format as the Field operator.

Description

Accessing the contents of a banked field data object will occur automatically through the proper bank setting, with synchronization occurring on the operation region that contains the *BankName* data variable, and on the Global Lock if specified by the *LockRule*.

This operator creates data field objects. The contents of the created objects are obtained by a reference to a bank selection register.

This encoding is used to define named data field objects whose data values are fields within a larger object selected by a bank-selected register.

Example

The following is a block of ASL sample code using BankField:

- Creates a 4-bit bank-selected register in system I/O space.
- Creates overlapping fields in the same system I/O space that are selected via the bank register.

```
//
// Define a 256-byte operational region in SystemIO space
// and name it GIO0

OperationRegion (GIO0, SystemIO, 0x125, 0x100)

// Create some fields in GIO including a 4-bit bank select register

Field (GIO0, ByteAcc, NoLock, Preserve) {
    GLB1, 1,
    GLB2, 1,
    Offset (1),           // Move to offset for byte 1
    BNK1, 4
}

// Create FET0 & FET1 in bank 0 at byte offset 0x30

BankField (GIO0, BNK1, 0, ByteAcc, NoLock, Preserve) {
    Offset (0x30),
    FET0, 1,
    FET1, 1
}

// Create BLVL & BAC in bank 1 at the same offset

BankField (GIO0, BNK1, 1, ByteAcc, NoLock, Preserve) {
    Offset (0x30),
    BLVL, 7,
    BAC, 1
}
```

19.6.8 Break (Break from While)

Syntax:

```
Break
```

Description

Break causes execution to continue immediately following the innermost enclosing **While** or **Switch** scope, in the current Method. If there is no enclosing **While** or **Switch** within the current Method, a fatal error is generated.

Compatibility Note: In ACPI 1.0, the Break operator continued immediately following the innermost “code package.” Starting in ACPI 2.0, the Break operator was changed to exit the innermost “While” or “Switch” package. This should have no impact on existing code, since the ACPI 1.0 definition was, in practice, useless.

19.6.9 BreakPoint (Execution Break Point)

Syntax:

```
BreakPoint
```

Description

Used for debugging, the **Breakpoint** opcode stops the execution and enters the AML debugger. In the non-debug version of the AML interpreter, **BreakPoint** is equivalent to **Noop**.

19.6.10 Buffer (Declare Buffer Object)

Syntax:

```
Buffer (BufferSize) {Initializer} => Buffer
```

Arguments

Declares a Buffer of optional size BufferSize and an optional initial value of Initializer. The Initializer is must be either a ByteList or a String.

Description

The optional *BufferSize* argument specifies the size of the buffer and an optional initial value of the buffer is specified via the *Initializer*. The initial value can be either an ASCII String or a list of byte values separated by commas. Strings are automatically null terminated with a single zero byte.

The relationship between the BufferSize and the Initializer is summarized by the rules below.

In the typical case, the BufferSize is identical to the length of the Initializer:

```
Name (BUF0, Buffer(4) {0x01,0x02,0x03,0x04}) // Length = 4
```

If the BufferSize is not specified, the length of the Initializer is used as the buffer size:

```
Name (BUF1, Buffer() {0,1,2,3,4,5}) // Length = 6
Name (BUF2, Buffer() {"abcde"}) // Length = 6
```

If the BufferSize is larger than the length of the Initializer, the BufferSize is used as the final buffer size. At runtime, the AML interpreter will automatically pad zeros to the Initializer to match the BufferSize:

```
Name (BUF3, Buffer(1024) {4,5,6,7,8}) // Length = 1024
Name (BUF4, Buffer(1024) {"abcde"}) // Length = 1024
```

If the BufferSize is smaller than the length of the Initializer, the length of the Initializer is used as the buffer size:

```
Name (BUF5, Buffer(1) {5,4,3,2,1}) // Length = 5
```

If the Initializer is not specified, the AML interpreter creates a buffer containing all zeros, the length of which matches the BufferSize:

```
Name (BUF6, Buffer(32} {}}) // Length = 32
```

If neither the BufferSize nor the Initializer are specified, a buffer of zero length is created:

```
Name (BUF7, Buffer() {}}) // Length = 0
```

19.6.11 Case (Expression for Conditional Execution)

Syntax:

```
Case ( Value ) {TermList}
```

Arguments

Value specifies an Integer, Buffer, String or Package object. TermList is a sequence of executable ASL expressions.

Description

Execute code based upon the value of a Switch statement.

If the Case Value is an Integer, Buffer or String, then control passes to the statement that matches the value of the enclosing Switch (Value). If the Case value is a Package, then control passes if any member of the package matches the Switch (Value). The Switch CaseTermList can include any number of Case instances, but no two Case Values (or members of a Value, if Value is a Package) within the same Switch statement can contain the same value.

Execution of the statement body begins at the start of the TermList and proceeds until the end of the TermList body or until a Break or Continue operator transfers control out of the body.

19.6.12 Concatenate (Concatenate Data)

Syntax:

```
Concatenate ( Source1, Source2, Result ) => Buffer or String
```

Arguments

Source1 and Source2 must each evaluate to any valid ACPI object. For the basic data object types (Integer, String, or Buffer), the value of the object is used in the concatenation. For all other object types (see table below), a string object is created that contains the name (type) of the object. This string object is then concatenated according to the rules in *Concatenate Data Types*.

The data type of *Source1* dictates the required type of *Source2* and the type of the result object. *Source2* is implicitly converted if necessary (and possible) to match the type of *Source1*.

Description

Source2 is concatenated to Source1 and the result data is optionally stored into Result.

Table 19.30: Concatenate Data Types

Source1 Type	Data	Source2 Data Type (Converted Type)	Result Data Type
Integer		Integer/String/Buffer → Integer	Buffer
String		Integer/String/Buffer /All other types → String	String
Buffer		Integer/String/Buffer /All other types → Buffer	Buffer

continues on next page

Table 19.30 – continued from previous page

Source1 Type	Data	Source2 Data Type (Converted Type)	Result Data Type
All other types String	→	Integer/String/Buffer /All other types → String	String

For the *Source1* /Integer case, a String or Buffer that cannot be implicitly converted to an Integer will generate a fatal error.

Table 19.31: Concatenate Object Types

Data Object Type	Name	Resolved to Value
1	Integer	Integer value of the object
2	String	String value of the object
3	Buffer	Buffer value of the object
Other Object Types	Name	Resolved to String
0	Uninitialized	“[Uninitialized Object]”
4	Package	“[Package]”
5	Field Unit	“[Field]”
6	Device	“[Device]”
7	Event	“[Event]”
8	Control Method	“[Control Method]”
9	Mutex	“[Mutex]”
10	Operation Region	“[Operation Region]”
11	Power Resource	“[Power Resource]”
12	Processor	“[Processor]”
13	Thermal Zone	“[Thermal Zone]”
14	Buffer Field	“[Buffer Field]”
15	<i>Reserved</i>	
16	Debug Object	“[Debug Object]”

Examples:

```
Device (DEVX) {}
Name (PKGX, Package () {1,2,3,"Battery1"})

Method (MTHX, 2)
{
    Concatenate ("My Object: ", DEVX, Debug) // MyObject: Device

    Printf ("PKGX %o contains %o elements\n", PKGX, SizeOf (PKGX))

    Printf ("Arg0: %o\n", Arg0)
}
```

19.6.13 ConcatenateResTemplate (Concatenate Resource Templates)

Syntax:

```
ConcatenateResTemplate (Source1, Source2, Result) => Buffer
```

Arguments

Source1 and *Source2* are evaluated as Resource Template buffers.

Description

The resource descriptors from *Source2* are appended to the resource descriptors from *Source1*. Then a new end tag and checksum are appended and the result is stored in *Result*, if specified. If either *Source1* or *Source2* is exactly 1 byte in length, a run-time error occurs. An empty buffer is treated as a resource template with only an end tag.

19.6.14 CondRefOf (Create Object Reference Conditionally)

Syntax:

```
CondRefOf (Source, Result) => Boolean
```

Arguments

Attempts to create a reference to the *Source* object. The *Source* of this operation can be any object type (for example, data package, device object, and so on), and the result data is optionally stored into *Result*.

Description

On success, the *Destination* object is set to refer to *Source* and the execution result of this operation is the value *True*. On failure, *Destination* is unchanged and the execution result of this operation is the value *False*. This can be used to reference items in the namespace that may appear dynamically (for example, from a dynamically loaded definition block).

CondRefOf is equivalent to *RefOf* except that if the *Source* object does not exist, it is fatal for *RefOf* but not for *CondRefOf*.

19.6.15 Connection (Declare Field Connection Attributes)

Syntax:

```
Connection (ConnectionResourceObj)
```

Arguments

ConnectionResourceObj is a GPIO or Serial Bus Connection Descriptor depending on the Operation Region type, or a named object containing the Descriptor.

See [Section 6.4.3.8.2](#) and *Field (Declare Field Objects)* for more information.

Examples:

```
OperationRegion(TOP1, GenericSerialBus, 0x00, 0x100)
                                     // GenericSerialBus device at command value
↳offset zero

Name (I2C, ResourceTemplate){
```

(continues on next page)

(continued from previous page)

```

    I2CSerialBusV2(0x5a,,100000,, "\_SB.I2C",,,,RawDataBuffer(){1,6})
})

Field(TOP1, BufferAcc, NoLock, Preserve)
{
    Connection(I2C)                // Specify connection resource information
    AccessAs(BufferAcc, AttribWord) // Use the GenericSerialBus
                                   // Read/Write Word protocol
    FLD0, 8,                        // Virtual register at command value 0.
    FLD1, 8,                        // Virtual register at command value 1.

Field(TOP1, BufferAcc, NoLock, Preserve)
{
    Connection(I2CSerialBusV2(0x5b,,100000,, "\_SB.I2C",,,,RawDataBuffer(){3,9}))
    AccessAs(BufferAcc, AttribBytes (16))
    FLD2, 8                        // Virtual register at command value 0.
}

// Create the GenericSerialBus data buffer
Name(BUFF, Buffer(34){})          // Create GenericSerialBus data buffer as
↪ BUFF
CreateByteField(BUFF, 0x00, STAT)  // STAT = Status (Byte)
CreateWordField(BUFF, 0x02, DATA) // DATA = Data (Word)

```

Description

The Connection macro declares the connection attributes for subsequent fields defined within the Field declaration.

19.6.16 Continue (Continue Innermost Enclosing While)**Syntax:**

```
Continue
```

Description

Continue causes execution to continue at the start of the innermost enclosing While scope, in the currently executing Control Method, at the point where the condition is evaluated. If there is no enclosing While within the current Method, a fatal error is generated.

19.6.17 CopyObject (Copy and Store Object)**Syntax:**

```
CopyObject (Source, Destination) => DataRefObject
```

Arguments

Converts the contents of the Source to a DataRefObject using the conversion rules in [Section 19.3.5](#) and then copies the results without conversion to the object referred to by Destination.

Description

If Destination is already an initialized object of type DataRefObject, the original contents of Destination are discarded and replaced with Source. Otherwise, a fatal error is generated.

Note

Compatibility Note: The CopyObject operator was first introduced new in ACPI 2.0.

19.6.18 CreateBitField (Create 1-Bit Buffer Field)

Syntax:

```
CreateBitField (SourceBuffer, BitIndex, BitFieldName)
```

Arguments

SourceBuffer is evaluated as a buffer. *BitIndex* is evaluated as an integer. *BitFieldName* is a NameString.

Description

A new buffer field object named *BitFieldName* is created for the bit of *SourceBuffer* at the bit index of *BitIndex*. The bit-defined field within *SourceBuffer* must exist. *BitFieldName* is created for the bit of *SourceBuffer* at the bit index of *BitIndex*. The bit-defined field within *SourceBuffer* must exist.

19.6.19 CreateByteField (Create 8-Bit Buffer Field)

Syntax:

```
CreateByteField ( SourceBuffer, ByteIndex, ByteFieldName )
```

Arguments

SourceBuffer is evaluated as a buffer. *ByteIndex* is evaluated as an integer. *ByteFieldName* is a NameString.

Description

A new buffer field object named *ByteFieldName* is created for the byte of *SourceBuffer* at the byte index of *ByteIndex*. The byte-defined field within *SourceBuffer* must exist.

19.6.20 CreateDWordField (Create 32-Bit Buffer Field)

Syntax:

```
CreateDWordField ( SourceBuffer, ByteIndex, DWordFieldName )
```

Arguments

SourceBuffer is evaluated as a buffer. *ByteIndex* is evaluated as an integer. *DWordFieldName* is a NameString.

Description

A new buffer field object named *DWordFieldName* is created for the DWord of *SourceBuffer* at the byte index of *ByteIndex*. The DWord-defined field within *SourceBuffer* must exist.

19.6.21 CreateField (Create Arbitrary Length Buffer Field)

Syntax:

```
CreateField ( SourceBuffer, BitIndex, NumBits, FieldName )
```

Arguments

SourceBuffer is evaluated as a buffer. *BitIndex* and *NumBits* are evaluated as integers. *FieldName* is a *NameString*.

Description

A new buffer field object named *FieldName* is created for the bits of *SourceBuffer* at *BitIndex* for *NumBits*. The entire bit range of the defined field within *SourceBuffer* must exist. If *NumBits* evaluates to zero, a fatal exception is generated.

19.6.22 CreateQWordField (Create 64-Bit Buffer Field)

Syntax:

```
CreateQWordField ( SourceBuffer, ByteIndex, QWordFieldName )
```

Arguments

SourceBuffer is evaluated as a buffer. *ByteIndex* is evaluated as an integer. *QWordFieldName* is a *NameString*.

Description

A new buffer field object named *QWordFieldName* is created for the *QWord* of *SourceBuffer* at the byte index of *ByteIndex*. The *QWord*-defined field within *SourceBuffer* must exist.

19.6.23 CreateWordField (Create 16-Bit Buffer Field)

Syntax:

```
CreateWordField ( SourceBuffer, ByteIndex, WordFieldName )
```

Arguments

SourceBuffer is evaluated as a buffer. *ByteIndex* is evaluated as an integer. *WordFieldName* is a *NameString*.

Description

A new bufferfield object named *WordFieldName* is created for the word of *SourceBuffer* at the byte index of *ByteIndex*. The word-defined field within *SourceBuffer* must exist.

19.6.24 CSI2Bus (CSI-2 Serial Bus Connection Resource Descriptor Macro)

Syntax

CSI2Bus (*SlaveMode*, *PhyType*, *LocalPort*, *ResourceSource*, *ResourceSourceIndex*, *ResourceUsage*, *DescriptorName*, *VendorData*)

Arguments

SlaveMode is an optional argument and can be either *ControllerInitiated* or *DeviceInitiated*. *ControllerInitiated* is the default. The bit field name *_SLV* is automatically created to refer to this portion of the resource descriptor.

PhyType is an integer ranging from 0 to 3 specifying the value for the PHY Type. This value describes the PHY type used to connect this device to its receiver. _PHY is automatically created to refer to this portion of the resource descriptor.

LocalPort is an optional integer argument, that identifies the index of the local Port for this connection. The first Port instance is 0.

ResourceSource is a string which uniquely identifies the remote CSI-2 receiver referred to by this descriptor. ResourceSource can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.

ResourceSourceIndex is an integer specifying the Port index of the Device specified by ResourceSource. Port index values begin at 0.

ResourceUsage is an optional argument and is assumed to be ResourceConsumer for this revision.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

VendorData is an optional argument that specifies an object to be decoded by the OS driver. It is a RawDataBuffer. The bit field name _VEN is automatically created to refer to this portion of the resource descriptor.

Description

The CSI2Bus macro evaluates to a buffer that contains a CSI-2 resource descriptor. The macro is designed to be used inside of a ResourceTemplate (see [Section 19.3.4](#)).

19.6.25 DataTableRegion (Create Data Table Operation Region)

Syntax:

```
DataTableRegion ( RegionName, SignatureString, OemIDString, OemTableIDString )
```

Arguments

Creates a new region named RegionName. SignatureString, OemIDString and OemTableIDString are evaluated as strings.

Description

A Data Table Region is a special Operation Region whose RegionSpace is SystemMemory. Any table referenced by a Data Table Region must be in memory marked by AddressRangeReserved or AddressRangeNVS.

The memory referred to by the Data Table Region is the memory that is occupied by the table referenced in XSDT that is identified by SignatureString, OemIDString and OemTableIDString. Any Field object can reference RegionName

The base address of a Data Table region is the address of the first byte of the header of the table identified by SignatureString, OemIDString and OemTableIDString. The length of the region is the length of the table.

19.6.26 Debug (Debugger Output)

Syntax:

```
Debug
```

Description

The debug data object is a virtual data object. Writes to this object provide debugging information. On at least debug versions of the interpreter, any writes into this object are appropriately displayed on the system's native kernel debugger. All writes to the debug object are otherwise benign. If the system is in use without a kernel debugger, then writes to the debug object are ignored. The following table relates the ASL term types that can be written to the Debug object to the format of the information on the kernel debugger display.

Table 19.32: Debug Object Display Formats

ASL Term Type	Display Format
Numeric data object	All digits displayed in hexadecimal format.
String data object	String is displayed.
Object reference	Information about the object is displayed (for example, object type and object name), but the object is not evaluated.

The Debug object is a write-only object; attempting to read from the debug object is not supported.

19.6.27 Decrement (Integer Decrement)

Syntax:

```
Decrement (Minuend) => Integer
Minuend -- => Integer
```

Arguments

Minuend is evaluated as an Integer.

Description

This operation decrements the Minuend by one and the result is stored back to Minuend. Equivalent to Subtract (Minuend, 1, Minuend). Underflow conditions are ignored and the result is Ones.

19.6.28 Default (Default Execution Path in Switch)

Syntax:

```
Default {TermList}
```

Arguments

TermList is a sequence of executable ASL expressions.

Description

Within the body of a *Switch (Select Code To Execute Based On Expression)* statement, the statements specified by TermList will be executed if no *Case (Expression for Conditional Execution)* statement value matches the Switch statement value. If Default is omitted and no Case match is found, none of the statements in the Switch body are

executed. There can be at most one Default statement in the immediate scope of the parent Switch statement. The Default statement can appear anywhere in the body of the Switch statement.

19.6.29 DefinitionBlock (Declare Definition Block)

Syntax:

```
DefinitionBlock ( AMLFileName, TableSignature, ComplianceRevision, OEMID, TableID,
↪ OEMRevision) {TermList}
```

Arguments

AMLFileName is a string that specifies the desired name of the translated output AML file. If the *AMLFileName* is a NULL (zero length) string, the ASL compiler will automatically create the filename (typically generated from the input filename/pathname). *TableSignature* is a string that contains the 4-character ACPI signature. *ComplianceRevision* is an 8-bit value. *OEMID* is a 6-character string, *TableId* is an 8-character string, and *OEMRevision* is a 32-bit value. *TermList* is a sequence of executable ASL expressions.

If multiple *DefinitionBlocks* are defined in the same ASL file, the first *DefinitionBlock* defines the output *AMLFileName* as per the rule above.

Description

The *DefinitionBlock* term specifies the unit of data and/or AML code that the OS will load as part of the Differentiated Definition Block or as part of an additional Definition Block.

This unit of data and/or AML code describes either the base system or some large extension (such as a docking station). The entire *DefinitionBlock* will be loaded and compiled by the OS as a single unit.

System software loads a definition block by referencing the objects in the *TermList* package in order. The object list is encoded as *TermList*, so that rather than describing a static object list, it is possible to describe a dynamic object list according to the system settings. See [Section 5.4.2](#).

Note: For compatibility with ACPI versions before ACPI 2.0, the bit width of Integer objects is dependent on the *ComplianceRevision* of the DSDT. If the *ComplianceRevision* is less than 2, all integers are restricted to 32 bits. Otherwise, full 64-bit integers are used. The version of the DSDT sets the global integer width for all integers, including integers in SSDTs.

19.6.30 DerefOf (Dereference an Object Reference)

Syntax:

```
DerefOf (Source) => Object
```

Arguments

Returns the object referred by the *Source* object reference.

Description

If the *Source* evaluates to an object reference, the actual contents of the object referred to are returned. If the *Source* evaluates to a string, the string is evaluated as an ASL name (relative to the current scope) and the contents of that object are returned. If the object specified by *Source* does not exist then a fatal error is generated. If the object specified is a reference generated by the *Index()* operator and refers to an uninitialized package element, then a fatal error is generated.

Note

Compatibility Note: The use of a String with DerefOf was first introduced in ACPI 2.0.

19.6.31 Device (Declare Device Package)

Syntax:

```
Device (DeviceName) {TermList}
```

Arguments

Creates a Device object of name DeviceName, which represents a processor, a bus or a device, or any other similar hardware. Device opens a name scope.

Description

A Device Package is one of the basic ways the Differentiated Definition Block describes the hardware devices in the system to the operating software. Each Device Package is defined somewhere in the hierarchical namespace corresponding to that device's location in the system. Within the namespace of the device are other names that provide information and control of the device, along with any sub-devices that in turn describe sub-devices, and so on.

For any device, the platform runtime firmware provides only information that is added to the device in a non-hardware standard manner. This type of value-added function is expressible in the ACPI Definition Block such that operating software can use the function.

The platform runtime firmware supplies Device Objects only for devices that are obtaining some system-added function outside the device's normal capabilities and for any Device Object required to fill in the tree for such a device. For example, if the system includes a PCI device (integrated or otherwise) with no additional functions such as power management, the platform runtime firmware would not report such a device; however, if the system included an integrated ISA device below the integrated PCI device (device is an ISA bridge), then the system would include a Device Package for the ISA device with the minimum feature being added being the ISA device's ID and configuration information and the parent PCI device, because it is required to get the ISA Device Package placement in the namespace correct.

The device object list is encoded as *TermList*, so that rather than describing a static device object list, it is possible to describe a dynamic device object list according to the system settings. see [Section 5.4.2](#).

Example

The following block of ASL sample code shows a nested use of Device objects to describe an IDE controller connected to the root PCI bus:

```
Device (IDE0) { // primary controller
    Name (_ADR, 0) // put PCI Address (device/function) here

    // define region for IDE mode register

    OperationRegion (PCIC, PCI_Config, 0x50, 0x10)
    Field (PCIC, AnyAcc, NoLock, Preserve) {
        ...
    }
    Device (PRIM) { // Primary adapter
        Name (_ADR, 0) // Primary adapter = 0
        ...
        Method (_STM, 2) {
```

(continues on next page)

(continued from previous page)

```

    ...
}
Method (_GTM) {
    ...
}

Device (MSTR) { // master channel
    Name (_ADR, 0)
    Name (_PR0, Package () {0, PIDE})

    Name (_GTF) {
        ...
    }
}

Device (SLAV) {
    Name (_ADR, 1)
    Name (_PR0, Package () {0, PIDE})
    Name (_GTF) {
        ...
    }
}
}
}

```

19.6.32 Divide (Integer Divide)

Syntax:

```

Divide (Dividend, Divisor, Remainder, Result) => Integer
Result = Dividend / Divisor => Integer
Result /= Divisor => Integer

```

Arguments

Dividend and *Divisor* are evaluated as Integers.

Description

Dividend is divided by Divisor, then the resulting remainder is optionally stored into Remainder and the resulting quotient is optionally stored into Result. Divide-by-zero exceptions are fatal.

The function return value is the Result (quotient).

19.6.33 DMA (DMA Resource Descriptor Macro)

Syntax:

```

DMA ( DmaType , IsBusMaster , DmaTransferSize, DescriptorName )
{ DmaChannellist } => Buffer

```

Arguments

DmaType specifies the type of DMA cycle: ISA compatible (Compatibility), EISA Type A (TypeA), EISA Type B (TypeB) or EISA Type F (TypeF). The 2-bit field *DescriptorName._TYP* is automatically created to refer to this portion of the resource descriptor, where '0' is Compatibility, '1' is TypeA, '2' is TypeB and '3' is TypeF.

IsBusMaster specifies whether this device can generate DMA bus master cycles (BusMaster) or not (NotBusMaster). If nothing is specified, then BusMaster is assumed. The 1-bit field *DescriptorName._BM* is automatically created to refer to this portion of the resource descriptor, where '0' is NotBusMaster and '1' is BusMaster.

DmaTransferSize specifies the size of DMA cycles the device is capable of generating: 8-bit (Transfer8), 16-bit (Transfer16) or both 8 and 16-bit (Transfer8_16). The 2-bit field *DescriptorName._SIZ* is automatically created to refer to this portion of the resource descriptor, where '0' is Transfer8, '1' is Transfer8_16 and '2' is Transfer16.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

DmaChannelList is a comma-delimited list of integers in the range 0 through 7 that specify the DMA channels used by the device. There may be no duplicates in the list.

Description

The DMA macro evaluates to a buffer that contains a DMA resource descriptor. The format of this resource descriptor can be found in [DMA Descriptor](#). This macro is designed to be used inside of a [ResourceTemplate \(Resource To Buffer Conversion Macro\)](#).

19.6.34 DWordIO (DWord IO Resource Descriptor Macro)

Syntax:

```
DWordIO ( ResourceUsage, IsMinFixed, IsMaxFixed, Decode, ISARanges,
AddressGranularity, AddressMinimum, AddressMaximum, AddressTranslation,
RangeLength, ResourceSourceIndex, ResourceSource, DescriptorName,
TranslationType, TranslationDensity)
```

Arguments

ResourceUsage specifies whether the I/O range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

IsMinFixed specifies whether the minimum address of this I/O range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field *DescriptorName._MIF* is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this I/O range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field *DescriptorName._MAF* is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

Decode specifies whether or not the device decodes the I/O range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field *DescriptorName._DEC* is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

ISARanges specifies whether the I/O ranges specifies are limited to valid ISA I/O ranges (ISAOnly), valid non-ISA I/O ranges (NonISAOnly) or encompass the whole range without limitation (EntireRange). The 2-bit field *DescriptorName._RNG* is automatically created to refer to this portion of the resource descriptor, where '1' is NonISAOnly, '2' is ISAOnly and '0' is EntireRange.

AddressGranularity evaluates to a 32-bit integer that specifies the power-of-two boundary (- 1) on which the I/O range must be aligned. The 32-bit field *DescriptorName._GRA* is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 32-bit integer that specifies the lowest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate

addresses, this is the address on the secondary bus. The 32-bit field `DescriptorName._MIN` is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 32-bit integer that specifies the highest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in *AddressGranularity* is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 32-bit field `DescriptorName._MAX` is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 32-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 32-bit field `DescriptorName._TRA` is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 32-bit integer that specifies the total number of bytes decoded in the I/O range. The 32-bit field `DescriptorName._LEN` is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by *ResourceSource*. If this argument is specified, the *ResourceSource* argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the *ResourceSourceIndex* argument is not specified, a value of zero is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

TranslationType is an optional argument that specifies whether the resource type on the secondary side of the bus is different (*TypeTranslation*) from that on the primary side of the bus or the same (*TypeStatic*). If *TypeTranslation* is specified, then the primary side of the bus is Memory. If *TypeStatic* is specified, then the primary side of the bus is I/O. If nothing is specified, then *TypeStatic* is assumed. The 1-bit field `DescriptorName._TTP` is automatically created to refer to this portion of the resource descriptor, where '1' is *TypeTranslation* and '0' is *TypeStatic*. see [Table 6.49](#) for more information.

TranslationDensity is an optional argument that specifies whether or not the translation from the primary to secondary bus is sparse (*SparseTranslation*) or dense (*DenseTranslation*). It is only used when *TranslationType* is *TypeTranslation*. If nothing is specified, then *DenseTranslation* is assumed. The 1-bit field `DescriptorName._TRS` is automatically created to refer to this portion of the resource descriptor, where '1' is *SparseTranslation* and '0' is *DenseTranslation*. see [Table 6.50](#) for more information.

Description

The `DWordIO` macro evaluates to a buffer that contains a 32-bit I/O range resource descriptor. The format of the 32-bit I/O range resource descriptor can be found in [Table 6.46](#). This macro is designed to be used inside of a *ResourceTemplate* (*Resource To Buffer Conversion Macro*).

19.6.35 DWordMemory (DWord Memory Resource Descriptor Macro)

Syntax:

```
DWordMemory (ResourceUsage, Decode, IsMinFixed, IsMaxFixed, Cacheable,
ReadAndWrite, AddressGranularity, AddressMinimum, AddressMaximum,
AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource,
DescriptorName, MemoryRangeType, TranslationType)
```

Arguments

ResourceUsage specifies whether the Memory range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

Decode specifies whether or not the device decodes the Memory range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName._DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

IsMinFixed specifies whether the minimum address of this Memory range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName._MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this Memory range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName._MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

Cacheable specifies whether or not the memory region is cacheable (Cacheable), cacheable and write-combining (WriteCombining), cacheable and prefetchable (Prefetchable) or uncachable (NonCacheable). If nothing is specified, then NonCacheable is assumed. The 2-bit field DescriptorName._MEM is automatically created to refer to this portion of the resource descriptor, where '1' is Cacheable, '2' is WriteCombining, '3' is Prefetchable and '0' is NonCacheable.

ReadAndWrite specifies whether or not the memory region is read-only (ReadOnly) or read/write (ReadWrite). If nothing is specified, then ReadWrite is assumed. The 1-bit field DescriptorName._RW is automatically created to refer to this portion of the resource descriptor, where '1' is ReadWrite and '0' is ReadOnly.

AddressGranularity evaluates to a 32-bit integer that specifies the power-of-two boundary (- 1) on which the Memory range must be aligned. The 32-bit field DescriptorName._GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 32-bit integer that specifies the lowest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 32-bit field DescriptorName._MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 32-bit integer that specifies the highest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 32-bit field DescriptorName._MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 32-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 32-bit field DescriptorName._TRA is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 32-bit integer that specifies the total number of bytes decoded in the Memory range. The 32-bit field DescriptorName._LEN is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by ResourceSource. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this Memory range is allocated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

MemoryRangeType is an optional argument that specifies the memory usage. The memory can be marked as normal (AddressRangeMemory), used as ACPI NVS space (AddressRangeNVS), used as ACPI reclaimable space (Address-

RangeACPI) or as system reserved (AddressRangeReserved). If nothing is specified, then AddressRangeMemory is assumed. The 2-bit field DescriptorName._MTP is automatically created in order to refer to this portion of the resource descriptor, where '0' is AddressRangeMemory, '1' is AddressRangeReserved, '2' is AddressRangeACPI and '3' is AddressRangeNVS.

TranslationType is an optional argument that specifies whether the resource type on the secondary side of the bus is different (TypeTranslation) from that on the primary side of the bus or the same (TypeStatic). If TypeTranslation is specified, then the primary side of the bus is I/O. If TypeStatic is specified, then the primary side of the bus is Memory. If nothing is specified, then TypeStatic is assumed. The 1-bit field DescriptorName._TTP is automatically created to refer to this portion of the resource descriptor, where '1' is TypeTranslation and '0' is TypeStatic. see [Table 6.50](#) for more information.

Description

The DWordMemory macro evaluates to a buffer which contains a 32-bit memory resource descriptor. The format of the 32-bit memory resource descriptor can be found in *DWORD Address Space Descriptor Definition*. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.36 DWordPCC (DWordPCC Resource Descriptor Macro)

Syntax:

```
DWordPCC (PccChannel, ResourceSourceIndex, ResourceSource,
DescriptorName)
```

Arguments

PccChannel evaluates to an 8-bit integer that specifies the PCCT Index of the PCC Subspace consumed by this Resource.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by ResourceSource. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a value of zero is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The DWordPCC macro evaluates to a buffer that contains a 32-bit Address Space resource descriptor. The format of the 32-bit Address Space resource descriptor can be found in [Table 6.46](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*. The PccChannel field is used to populate the Address field in the resultant Resource Buffer.

19.6.37 DWordSpace (DWord Space Resource Descriptor Macro)

Syntax:

```
DWordSpace (ResourceType, ResourceUsage, Decode, IsMinFixed, IsMaxFixed,
TypeSpecificFlags, AddressGranularity, AddressMinimum, AddressMaximum,
AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource,
DescriptorName)
```

Arguments

ResourceType evaluates to an 8-bit integer that specifies the type of this resource. Acceptable values are 0xC0 through 0xFF.

ResourceUsage specifies whether the Memory range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

Decode specifies whether or not the device decodes the Memory range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName._DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

IsMinFixed specifies whether the minimum address of this Memory range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName._MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this Memory range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName._MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

TypeSpecificFlags evaluates to an 8-bit integer. The flags are specific to the ResourceType.

AddressGranularity evaluates to a 32-bit integer that specifies the power-of-two boundary (- 1) on which the Memory range must be aligned. The 32-bit field DescriptorName._GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 32-bit integer that specifies the lowest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 32-bit field DescriptorName._MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 32-bit integer that specifies the highest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 32-bit field DescriptorName._MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 32-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 32-bit field DescriptorName._TRA is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 32-bit integer that specifies the total number of bytes decoded in the Memory range. The 32-bit field DescriptorName._LEN is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by ResourceSource. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this Memory range is allocated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The DWordSpace macro evaluates to a buffer which contains a 32-bit Address Space resource descriptor. The format of this resource descriptor can be found in *DWORD Address Space Descriptor*. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.38 EISAID (EISA ID String To Integer Conversion Macro)

Syntax:

```
EISAID ( EisaIdString ) => DWordConst
```

Arguments

The EisaIdString must be a String object of the form “UUUNNNN”, where “U” is an uppercase letter and “N” is a hexadecimal digit. No asterisks or other characters are allowed in the string.

Description

Converts EisaIdString, a 7-character text string argument, into its corresponding 4-byte numeric EISA ID encoding. It can be used when declaring IDs for devices that have EISA IDs.

Example

```
EISAID ("PNP0C09") // This is a valid invocation of the macro.
```

19.6.39 Else (Alternate Execution)

Syntax:

```
Else {TermList}
```

Arguments

TermList is a sequence of executable ASL statements.

Description

If Predicate evaluates to 0 in an If statement, then control is transferred to the Else portion, which can consist of zero or more ElseIf statements followed by zero or one Else statements. If the Predicate of any ElseIf statement evaluates to non-zero, the statements in its term list are executed and then control is transferred past the end of the final Else term. If no Predicate evaluates to non-zero, then the statements in the Else term list are executed.

Example

The following example checks Local0 to be zero or non-zero. On non-zero, CNT is incremented; otherwise, CNT is decremented:

```
If (LGreater (Local0, 5)
{
    Increment (CNT)
}
```

(continues on next page)

(continued from previous page)

```

Else If (Local0) {
    Add (CNT, 5, CNT)
}
Else
{
    Decrement (CNT)
}

```

19.6.40 Elself (Alternate/Conditional Execution)

Syntax:

```
ElseIf ( Predicate ) {TermList}
```

Arguments

Predicate is evaluated as an Integer.

Description

If the Predicate of any ElseIf statement evaluates to non-zero, the statements in its term list are executed and then control is transferred past the end of the final Else. If no Predicate evaluates to non-zero, then the statements in the Else term list are executed.

Note

Compatibility Note: The ElseIf operator was first introduced in ACPI 2.0, but is backward compatible with the ACPI 1.0 specification. An ACPI 2.0 and later ASL compiler must synthesize ElseIf from the If and Else opcodes available in 1.0. For example:

```

If (predicate1)
{
    ...statements1...
}
ElseIf (predicate2)
{
    ...statements2...
}
Else
{
    ...statements3...
}

```

is translated to the following:

```

If (predicate1)
{
    ...statements1...
}
Else
{
    If (predicate2)
    {
        ...statements2...
    }
    Else

```

19.6.41 EndDependentFn (End Dependent Function Resource Descriptor Macro)

Syntax:

```
EndDependentFn ( ) => Buffer
```

Description

The EndDependentFn macro generates an end-of-dependent-function resource descriptor buffer inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*. This descriptor must be matched with a *StartDependentFn (Start Dependent Function Resource Descriptor Macro)* or a *StartDependentFnNoPri (Start Dependent Function Resource Descriptor Macro)*.

19.6.42 Event (Declare Event Synchronization Object)

Syntax:

```
Event ( EventName )
```

Arguments

Creates an event synchronization object named EventName.

Description

For more information about the uses of an event synchronization object, see the ASL definitions for the Wait, Signal, and Reset function operators.

19.6.43 ExtendedIO (Extended IO Resource Descriptor Macro)

Syntax:

```
ExtendedIO ( ResourceUsage, IsMinFixed , IsMaxFixed , Decode ,  
ISARanges , AddressGranularity, AddressMinimum, AddressMaximum ,  
AddressTranslation , RangeLength , TypeSpecificAttributes,  
DescriptorName, TranslationType, TranslationDensity)
```

Arguments

ResourceUsage specifies whether the Memory range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

IsMinFixed specifies whether the minimum address of this I/O range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName._MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this I/O range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName._MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

Decode specifies whether or not the device decodes the I/O range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName._DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

ISARanges specifies whether the I/O ranges specified are limited to valid ISA I/O ranges (ISAOnly), valid non-ISA I/O ranges (NonISAOnly) or encompass the whole range without limitation (EntireRange). The 2-bit field DescriptorName._RNG is automatically created to refer to this portion of the resource descriptor, where '1' is NonISAOnly, '2' is ISAOnly and '0' is EntireRange.

AddressGranularity evaluates to a 64-bit integer that specifies the power-of-two boundary (-1) on which the I/O range must be aligned. The 64-bit field DescriptorName._GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 64-bit integer that specifies the lowest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName._MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 64-bit integer that specifies the highest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName._MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 64-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 64-bit field DescriptorName._TRA is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 64-bit integer that specifies the total number of bytes decoded in the I/O range. The 64-bit field DescriptorName._LEN is automatically created to refer to this portion of the resource descriptor.

Type Specific Attributes is an optional argument that specifies attributes specific to this resource type.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operatorsDescription

The ExtendedIO macro evaluates to a buffer that contains a 64-bit I/O resource descriptor, which describes a range of I/O addresses. The format of this resource descriptor can be found in [Section 6.4.3.5.4](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

TranslationType is an optional argument that specifies whether the resource type on the secondary side of the bus is different (TypeTranslation) from that on the primary side of the bus or the same (TypeStatic). If TypeTranslation is specified, then the primary side of the bus is Memory. If TypeStatic is specified, then the primary side of the bus is I/O. If nothing is specified, then TypeStatic is assumed. The 1-bit field DescriptorName._TTP is automatically created to refer to this portion of the resource descriptor, where '1' is TypeTranslation and '0' is TypeStatic. See [Section 5.6.8](#) for more information.

TranslationDensity is an optional argument that specifies whether or not the translation from the primary to secondary bus is sparse (SparseTranslation) or dense (DenseTranslation). It is only used when TranslationType is TypeTranslation. If nothing is specified, then DenseTranslation is assumed. The 1-bit field DescriptorName._TRS is automatically created to refer to this portion of the resource descriptor, where '1' is SparseTranslation and '0' is DenseTranslation. See [Section 5.6.8](#) for more information.

19.6.44 ExtendedMemory (Extended Memory Resource Descriptor Macro)

Syntax:

```
ExtendedMemory ( ResourceUsage, Decode, IsMinFixed, IsMaxFixed,
Cacheable, ReadAndWrite, AddressGranularity, AddressMinimum,
AddressMaximum, AddressTranslation, RangeLength, TypeSpecificAttributes,
DescriptorName, MemoryRangeType, TranslationType)
```

Arguments

ResourceUsage specifies whether the Memory range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

Decode specifies whether or not the device decodes the Memory range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName._DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

IsMinFixed specifies whether the minimum address of this Memory range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName._MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this Memory range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName._MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

Cacheable specifies whether or not the memory region is cacheable (Cacheable), cacheable and write-combining (WriteCombining), cacheable and prefetchable (Prefetchable) or uncacheable (NonCacheable). If nothing is specified, then NonCacheable is assumed. The 2-bit field DescriptorName._MEM is automatically created to refer to this portion of the resource descriptor, where '1' is Cacheable, '2' is WriteCombining, '3' is Prefetchable and '0' is NonCacheable.

ReadAndWrite specifies whether or not the memory region is read-only (ReadOnly) or read/write (ReadWrite). If nothing is specified, then ReadWrite is assumed. The 1-bit field DescriptorName._RW is automatically created to refer to this portion of the resource descriptor, where '1' is ReadWrite and '0' is ReadOnly.

AddressGranularity evaluates to a 64-bit integer that specifies the power-of-two boundary (- 1) on which the Memory range must be aligned. The 64-bit field DescriptorName._GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 64-bit integer that specifies the lowest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName._MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 64-bit integer that specifies the highest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName._MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 64-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 64-bit field DescriptorName._TRA is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 64-bit integer that specifies the total number of bytes decoded in the Memory range. The 64-bit field DescriptorName._LEN is automatically created to refer to this portion of the resource descriptor.

Type Specific Attributes is an optional argument that specifies attributes specific to this resource type.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined

descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

MemoryRangeType is an optional argument that specifies the memory usage. The memory can be marked as normal (AddressRangeMemory), used as ACPI NVS space (AddressRangeNVS), used as ACPI reclaimable space (AddressRangeACPI) or as system reserved (AddressRangeReserved). If nothing is specified, then AddressRangeMemory is assumed. The 2-bit field DescriptorName. _MTP is automatically created in order to refer to this portion of the resource descriptor, where '0' is AddressRangeMemory, '1' is AddressRangeReserved, '2' is AddressRangeACPI and '3' is AddressRangeNVS.

TranslationType is an optional argument that specifies whether the resource type on the secondary side of the bus is different (TypeTranslation) from that on the primary side of the bus or the same (TypeStatic). If TypeTranslation is specified, then the primary side of the bus is I/O. If TypeStatic is specified, then the primary side of the bus is Memory. If nothing is specified, then TypeStatic is assumed. The 1-bit field DescriptorName. _TTP is automatically created to refer to this portion of the resource descriptor, where '1' is TypeTranslation and '0' is TypeStatic. See [Section 5.6.8](#) for more information.

Description

The ExtendedMemory macro evaluates to a buffer that contains a 64-bit memory resource descriptor, which describes a range of memory addresses. The format of this resource descriptor can be found in [Section 6.4.3.5.4](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.45 ExtendedSpace (Extended Address Space Resource Descriptor Macro)

Syntax:

```
ExtendedSpace (ResourceType, ResourceUsage, Decode, IsMinFixed,
IsMaxFixed, TypeSpecificFlags, AddressGranularity, AddressMinimum,
AddressMaximum, AddressTranslation, RangeLength, TypeSpecificAttributes,
DescriptorName)
```

Arguments

ResourceType evaluates to an 8-bit integer that specifies the type of this resource. Acceptable values are 0x00 through 0x03 and 0xC0 through 0xFF.

ResourceUsage specifies whether the Memory range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

Decode specifies whether or not the device decodes the Memory range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName. _DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

IsMinFixed specifies whether the minimum address of this Memory range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName. _MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this Memory range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName. _MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

TypeSpecificFlags evaluates to an 8-bit integer. The flags are specific to the ResourceType. This field is optional. If absent, it represents memory space of type AddressRangeMemory.

AddressGranularity evaluates to a 64-bit integer that specifies the power-of-two boundary (- 1) on which the Memory range must be aligned. The 64-bit field DescriptorName. _GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 64-bit integer that specifies the lowest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in *AddressGranularity* is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field *DescriptorName._MIN* is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 64-bit integer that specifies the highest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in *AddressGranularity* is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field *DescriptorName._MAX* is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 64-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 64-bit field *DescriptorName._TRA* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 64-bit integer that specifies the total number of bytes decoded in the Memory range. The 64-bit field *DescriptorName._LEN* is automatically created to refer to this portion of the resource descriptor.

Type Specific Attributes is an optional argument that specifies attributes specific to this resource type.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The *ExtendedSpace* macro evaluates to a buffer that contains a 64-bit Address Space resource descriptor, which describes a range of addresses. The format of the 64-bit AddressSpace descriptor can be found in [Section 6.4.3.5.4](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.46 External (Declare External Objects)

Syntax:

```
External ( ObjectName, ObjectType, ReturnType, ParameterTypes )
```

Arguments

ObjectName is a *NameString*.

ObjectType is an optional *ObjectTypeKeyword* (e.g. *IntObj*, *PkgObj*, etc.). If not specified, "UnknownObj" type is assumed.

ReturnType is optional. If the specified *ObjectType* is *MethodObj*, then this specifies the type or types of object returned by the method. If the method does not return an object, then nothing is specified or *UnknownObj* is specified. To specify a single return type, simply use the *ObjectTypeKeyword*. To specify multiple possible return types, enclose the comma-separated *ObjectTypeKeywords* with braces. For example: {*IntObj*, *BuffObj*}.

ParameterTypes is optional. If the specified *ObjectType* is *MethodObj*, this specifies both the number and type of the method parameters. It is a comma-separated, variable-length list of the expected object type or types for each of the method parameters, enclosed in braces. For each parameter, the parameter type consists of either an *ObjectTypeKeyword* or a comma-separated sub-list of *ObjectTypeKeywords* enclosed in braces. There can be no more than seven parameters in total.

The *External* directive informs the ASL compiler that the object is declared external to this table so that no errors will be generated for an undeclared object. The ASL compiler will create the external object at the specified place in the namespace (if a full path of the object is specified), or the object will be created at the current scope of the *External* term.

For external control methods, the ASL compiler can emit an External AML opcode that contains the name of the method and the number of required arguments. This information may be used by AML disassemblers to properly disassemble the AML to the correct ASL code.

External is especially useful for use in secondary SSDTs, when the required scopes and objects are declared in the main DSDT.

Example

This example shows the use of External in conjunction with Scope within an SSDT:

```
DefinitionBlock ("ssdt.aml", "SSDT", 2, "X", "Y", 0x00000001)
{
    External (\_SB.PCI0, DeviceObj)

    Scope (\_SB.PCI0)
    {
    }
}
```

19.6.47 Fatal (Fatal Error Check)

Syntax:

```
Fatal ( Type, Code, Arg )
```

Arguments

This operation is used to inform the OS that there has been an OEM-defined fatal error.

Description

In response, the OS must log the fatal event and perform a controlled OS shutdown in a timely fashion.

19.6.48 Field (Declare Field Objects)

Syntax:

```
Field ( RegionName, AccessType, LockRule, UpdateRule ) {FieldUnitList}
```

Arguments

RegionName is evaluated as a NameString that refers to the host operation region.

AccessType is optional and defines the default access width of the field definition and is any one of the following: AnyAcc, ByteAcc, WordAcc, DWordAcc, or QWordAcc. In general, accesses within the parent object are performed naturally aligned. If desired, AccessType set to a value other than AnyAcc can be used to force minimum access width. Notice that the parent object must be able to accommodate the AccessType width. For example, an access type of WordAcc cannot read the last byte of an odd-length operation region. The exceptions to natural alignment are the access types used for a non-linear SMBus device. These will be discussed in detail below. Not all access types are meaningful for every type of operational region. If not specified, the default is AnyAcc.

LockRule is optional and indicates whether the Global Lock is to be used when accessing this field and is one of the following: Lock or NoLock. If LockRule is set to Lock, accesses to modify the component data objects will acquire and release the Global Lock. If both types of locking occur, the Global Lock is acquired after the parent object Mutex. On Hardware-reduced ACPI platforms, Lock is not supported. If not specified, the default is NoLock.

UpdateRule is optional and specifies how the unmodified bits of a field are treated, and can be any one of the following: Preserve, WriteAsOnes, or WriteAsZeros. For example, if a field defines a component data object of 4 bits in the middle of a WordAcc region, when those 4 bits are modified the UpdateRule specifies how the other 12 bits are treated. If not specified, the default is Preserve.

FieldUnitList is a variable-length list of individual field unit definitions, separated by commas. Each entry in the field unit list is one of the following:

Table 19.33: Field Unit List Entries

FieldUnitName (BitLength)
Offset (<i>ByteOffset</i>)
AccessAs (<i>AccessType</i> , <i>AccessAttribute</i>)
Connection (<i>ConnectionResourceObj</i>)

FieldUnitName is the ACPI name for the field unit (1 to 4 characters), and BitLength is the length of the field unit in bits. Offset is used to specify the byte offset of the next defined field unit. This can be used instead of defining the bit lengths that need to be skipped. AccessAs is used to define the access type and attributes for the remaining field units within the list. Connection is used to identify the connection resource of the field access. This is necessary for GenericSerialBus and GeneralPurposeIO operation region address spaces only.

Description

Declares a series of named data objects whose data values are fields within a larger object. The fields are parts of the object named by RegionName, but their names appear in the same scope as the Field term.

For example, the field operator allows a larger operation region that represents a hardware register to be broken down into individual bit fields that can then be accessed by the bit field names. Extracting and combining the component field from its parent is done automatically when the field is accessed.

When reading from a FieldUnit, returned values are normalized (shifted and masked to the proper length.) The data type of an individual FieldUnit can be either a Buffer or an Integer, depending on the bit length of the FieldUnit. If the FieldUnit is smaller than or equal to the size of an Integer (in bits), it will be treated as an Integer. If the FieldUnit is larger than the size of an Integer, it will be treated as a Buffer. The size of an Integer is indicated by the DSDT header's Revision field. A revision less than 2 indicates that the size of an Integer is 32 bits. A value greater than or equal to 2 signifies that the size of an Integer is 64 bits. For more information about data types and FieldUnit type conversion rules, see [Section 19.3.5.7](#).

Accessing the contents of a field data object provides access to the corresponding field within the parent object. If the parent object supports Mutex synchronization, accesses to modify the component data objects will acquire and release ownership of the parent object around the modification.

The following table relates region types declared with an OperationRegion term to the different access types supported for each region.

Table 19.34: OperationRegion Address Spaces and Access Types

Address Space	Permitted Access Type(s)	Description
SystemMemory	ByteAcc, WordAcc, DWordAcc, QWordAcc, or AnyAcc	All access allowed
SystemIO	ByteAcc, WordAcc, DWordAcc, QWordAcc, or AnyAcc	All access allowed
PCI_Config	ByteAcc, WordAcc, DWordAcc, QWordAcc, or AnyAcc	All access allowed
EmbeddedControl	ByteAcc	Byte access only

continues on next page

Table 19.34 – continued from previous page

Address Space	Permitted Access Type(s)	Description
SMBus	BufferAcc	Reads and writes to this operation region involve the use of a region specific data buffer. (See below.)
SystemCMOS	ByteAcc	Byte access only
PciBarTarget	ByteAcc, WordAcc, DWordAcc, QWordAcc, or AnyAcc	All access allowed
IPMI	BufferAcc	Reads and writes to this operation region involve the use of a region specific data buffer. (See below.)
GeneralPurposeIO	ByteAcc	Byte access only
GenericSerialBus	BufferAcc	Reads and writes to this operation region involve the use of a region-specific data buffer. (See below.)
PCC	ByteAcc	Reads and writes to this operation region are performed in units of bytes.

The named FieldUnit data objects are provided in the FieldList as a series of names and bit widths. Bits assigned no name (or NULL) are skipped. The ASL compiler supports the Offset (ByteOffset) macro within a FieldList to skip to the bit position of the supplied byte offset, and the AccessAs macro to change access within the field list.

GenericSerialBus, SMBus and IPMI regions are inherently non-linear, where each offset within the respective address space represents a variable sized (0 to 32 bytes) field. Given this uniqueness, these operation regions include restrictions on their field definitions and require the use of a region-specific data buffer when initiating transactions. For more information on the SMBus data buffer format see [Section 13.2.5](#). For more information on the IPMI data buffer format, see [Section 5.5.2.4.5](#). For more information on the GenericSerialBus data buffer format, see [Section 5.5.2.4.7](#).

For restrictions on the use of Fields with GeneralPurposeIO OpRegions, see [Section 5.5.2.4.6](#).

Example:

```

OperationRegion (MIOC, PCI_Config, Zero, 0xFF)
Field (MIOC, AnyAcc, NoLock, Preserve)
{
    Offset (0x58),
    HXGB, 32,
    HXGT, 32,
    GAPE, 8,
    MR0A, 4,
    MR0B, 4
}

```

19.6.49 FindSetLeftBit (Find First Set Left Bit)

Syntax:

```
FindSetLeftBit ( Source, Result ) => Integer
```

Arguments

Source is evaluated as an Integer.

Description

The one-based bit location of the first MSb (most significant set bit) is optionally stored into Result. The result of 0 means no bit was set, 1 means the left-most bit set is the first bit, 2 means the left-most bit set is the second bit, and so on.

19.6.50 FindSetRightBit (Find First Set Right Bit)

Syntax:

```
FindSetRightBit ( Source, Result ) => Integer
```

Arguments

Source is evaluated as an Integer.

Description

The one-based bit location of the most LSb (least significant set bit) is optionally stored in Result. The result of 0 means no bit was set, 32 means the first bit set is the thirty-second bit, 31 means the first bit set is the thirty-first bit, and so on.

19.6.51 FixedDMA (DMA Resource Descriptor Macro)

Syntax:

```
FixedDMA ( DmaRequestLine, Channel, DmaTransferWidth, DescriptorName ) => Buffer
```

Arguments

DmaRequestLine is a system-relative number uniquely identifying the request line statically assigned to the device.. The bit field name *_DMA* is automatically created to refer to this portion of the resource descriptor.

Channel is a controller-relative number uniquely identifying the channel statically assigned to this DMARequestLine. Channels can be shared by reusing Channel numbers across descriptors. The bit field name *_TYP* is automatically created to refer to this portion of the resource descriptor.

DmaTransferWidth is an optional argument specifying the width of data transfer for which the device is configured. Valid values are Width8Bit, Width16Bit, Width32Bit, Width64Bit, Width 128Bit or Width256Bit. If not specified, Width32Bit is assumed. The bit field name *_SIZ* is automatically created to refer to this portion of the resource descriptor.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The FixedDMA macro evaluates to a buffer that contains a *Fixed DMA Descriptor*. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.52 FixedIO (Fixed IO Resource Descriptor Macro)

Syntax:

```
FixedIO ( AddressBase, RangeLength, DescriptorName ) => Buffer
```

Arguments

AddressBase evaluates to a 16-bit integer. It describes the starting address of the fixed I/O range. The field DescriptorName. *_BAS* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to an 8-bit integer. It describes the length of the fixed I/O range. The field DescriptorName. *_LEN* is automatically created to refer to this portion of the resource descriptor.

DescriptorName evaluates to a name string which refers to the entire resource descriptor.

Description

The FixedIO macro evaluates to a buffer that contains a fixed I/O resource descriptor. The format of this resource descriptor can be found in [Section 6.4.2.6](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.53 For (Conditional Loop)

Syntax:

```
For ( Initialize, Predicate, Update ) {TermList}
```

Arguments

Initialize. This optional expression is evaluated once before the loop is entered. If not specified, no initialization takes place.

Predicate. The list of terms within the TermList are executed until the predicate evaluates to zero (FALSE). If this argument is not specified, the For macro is equivalent to *While(1)*.

Update. This optional expression is evaluated once per execution of the loop, after all other terms within the TermList have been executed.

Description

For is a macro that creates a loop by converting the input arguments to the equivalent ASL *While* loop.

Note: Creation of a named object more than once in a given scope is not allowed. As such, unconditionally creating named objects within a For loop must be avoided. A fatal error will be generated on the second iteration of the loop, during the attempt to create the same named object a second time.

Example

The following example shows the use of the *For* macro to create a loop, followed by the equivalent *While* loop that is actually emitted by the ASL compiler:

```
for (local0 = 0, local0 < 8, local0++)
{
}

Local0 = 0
While (Local0 < 8)
{
    Local0++
}
```

19.6.54 Fprintf (Create and Store formatted string)

Syntax:

```
Fprintf ( Destination, FormatString, FormatArgs ) => String
```

Arguments

Fprintf is a macro that converts the evaluated *FormatString* into a series of string *Concatenate* operations, storing the result in *Destination*.

FormatString is a string literal which may contain one or more uses of the format specifier, *%o*, to indicate locations in the string where an object may be inserted. *%o* is the only format specifier supported since the resulting object is a string and type conversion is handled automatically by *Concatenate*.

FormatArgs is a comma separated list of Named Objects, Locals, or Args that can be evaluated to a string. Each argument is added to the *FormatString* using the *Concatenate* operation at the location specified by *%o* in order of appearance.

Description

Fprintf is a macro that converts the evaluated *FormatString* into a series of string *Concatenate* operations, storing the result in *Destination*.

Example

The following ASL example uses *Fprintf* to write a formatted string of Arg0 and Arg1 to the Named Object STR1:

```
Fprintf (STR1, "%o: %o Successful", Arg1, Arg0)
```

This *Fprintf* macro expression evaluates to the following ASL operation.

```
Store (Concatenate (Concatenate (Concatenate (Concatenate ("", Arg1), ": "), Arg0), "Successful"), STR1)
```

19.6.55 FromBCD (Convert BCD To Integer)

Syntax:

```
FromBCD ( BCDValue, Result ) => Integer
```

Arguments

BCDValue is evaluated as an Integer in Binary Coded Decimal format.

Description

The *FromBCD* operation *converts BCDValue to a numeric format, and optionally stores the numeric value into Result.*

19.6.56 Function (Declare Control Method)

Syntax:

```
Function ( FunctionName, ReturnType, ParameterTypes ) {TermList}
```

Arguments

ReturnType is optional and specifies the type(s) of the object(s) returned by the method. If the method does not return an object, then nothing is specified or *UnknownObj* is specified. To specify a single return type, simply use the *ObjectTypeKeyword* (e.g. *IntObj*, *PkgObj*, etc.). To specify multiple possible return types, enclose the comma-separated *ObjectTypeKeywords* with braces. For example:

```
{IntObj, BuffObj}.
```

ParameterTypes is optional and specifies both the number and type of the method parameters. It is a comma-separated, variable-length list of the expected object type or types for each of the method parameters, enclosed in braces. For each parameter, the parameter type consists of either an *ObjectTypeKeyword* or a comma-separated sub-list of *ObjectTypeKeywords* enclosed in braces. There can be no more than seven parameters in total.

Description

Function declares a named package containing a series of terms that collectively represent a control method. A control method is a procedure that can be invoked to perform computation. Function opens a name scope.

System software executes a control method by executing the terms in the package in order. For more information on method execution, see [Section 5.5.2](#).

The current namespace location used during name creation is adjusted to be the current location on the namespace tree. Any names created within this scope are “below” the name of this package. The current namespace location is assigned to the method package, and all namespace references that occur during control method execution for this package are relative to that location.

Functions are equivalent to a Method that specifies *NotSerialized*. As such, a function should not create any named objects, since a second thread that might re-enter the function will cause a fatal error if an attempt is made to create the same named object twice.

Note

Compatibility Note: New for ACPI 3.0

Example

The following block of ASL sample code shows the use of Function for defining a control method:

```
Function (EXAM, IntObj, {StrObj, {IntObj, StrObj}})
{
    Name (Temp, "")
    Store (Arg0, Temp)           // could have used Arg1
    Return (SizeOf (Concatenate (Arg1, Temp)))
}
```

This declaration is equivalent to:

```
Method (EXAM, 2, NotSerialized, 0, IntObj, {StrObj, {IntObj, StrObj}})
{
```

(continues on next page)

(continued from previous page)

```
...
}
```

19.6.57 GpioInt (GPIO Interrupt Connection Resource Descriptor Macro)

Syntax:

```
GpioInt (EdgeLevel, ActiveLevel, Shared, PinConfig, DebounceTimeout,
ResourceSource, ResourceSourceIndex, ResourceUsage, DescriptorName,
VendorData) {PinList}
```

Arguments

EdgeLevel can be either Edge or Level. The bit field name `_MOD` is automatically created to refer to this portion of the resource descriptor.

ActiveLevel can be one of ActiveHigh, ActiveLow or ActiveBoth. ActiveBoth can be specified only if EdgeLevel is Edge. The bit field name `_POL` is automatically created to refer to this portion of the resource descriptor.

Shared is an optional argument and can be one of Shared, Exclusive, SharedAndWake or ExclusiveAndWake. If not specified, Exclusive is assumed. The “Wake” designation indicates that the interrupt is capable of waking the system from a low-power idle state or a system sleep state. The bit field name `_SHR` is automatically created to refer to this portion of the resource descriptor.

PinConfig can be one of PullDefault, PullUp, PullDown, PullNone or a vendor-supplied value in the range 128-255. The bit field name `_PPI` is automatically created to refer to this portion of the resource descriptor.

DebounceTimeout is an optional argument specifying the debounce wait time, in hundredths of milliseconds. The bit field name `_DBT` is automatically created to refer to this portion of the resource descriptor.

ResourceSource is a string which uniquely identifies the GPIO controller referred to by this descriptor. ResourceSource can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.

ResourceSourceIndex is an optional argument and is assumed to be 0 for this revision.

ResourceUsage is an optional argument and is assumed to be ResourceConsumer for this revision.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

VendorData is an optional argument that specifies a RawDataBuffer containing vendor-defined byte data to be decoded by the OS driver. The bit field name `_VEN` is automatically created to refer to this portion of the resource descriptor.

PinList is a list of (zero-based) pin numbers on the ResourceSource that are described by this descriptor. For interrupt pin descriptors, only one pin is allowed. The bit field name `_PIN` is automatically created to refer to this portion of the resource descriptor.

Description

The *GpioInt* macro evaluates to a buffer that contains a GPIO Interrupt Connection resource descriptor. The format of this resource descriptor can be found in [Section 6.4.3.8.1](#). This macro is designed to be used inside of a *ResourceTemplate* (*Resource To Buffer Conversion Macro*).

19.6.58 GpioIo (GPIO Connection IO Resource Descriptor Macro)

Syntax:

```
GpioIo (Shared, PinConfig, DebounceTimeout, DriveStrength,
IORestriction, ResourceSource, ResourceSourceIndex, ResourceUsage,
DescriptorName, VendorData) {PinList}
```

Arguments

Shared is an optional argument and can be either Shared or Exclusive. If not specified, Exclusive is assumed. The bit field name `_SHR` is automatically created to refer to this portion of the resource descriptor.

PinConfig can be one of PullDefault, PullUp, PullDown, PullNone or a vendor-supplied value in the range 128-255. The bit field name `_PPI` is automatically created to refer to this portion of the resource descriptor.

DebounceTimeout is an optional argument specifying the hardware debounce wait time, in hundredths of milliseconds. The bit field name `_DBT` is automatically created to refer to this portion of the resource descriptor.

DriveStrength is an optional argument specifying the output drive capability of the pin, in hundredths of milliamperes. The bit field name `_DRS` is automatically created to refer to this portion of the resource descriptor.

IORestriction is an optional argument and can be IoRestrictionInputOnly, IoRestrictionOutputOnly, IoRestrictionNone, or IoRestrictionNoneAndPreserve. IORestrictions limit the mode in which the pin can be accessed (Input or Output). They also ensure that the pin configuration is preserved during periods when the driver is unloaded or the resource has been disconnected by the driver. If not specified, IoRestrictionNone is assumed. The bit field name `_IOR` is automatically created to refer to this portion of the resource descriptor.

ResourceSource is a string which uniquely identifies the GPIO controller referred to by this descriptor. ResourceSource can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.

ResourceSourceIndex is an optional argument and is always 0 for this revision.

ResourceUsage is an optional argument and is always ResourceConsumer for this revision.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

VendorData is an optional argument that specifies a RawDataBuffer containing vendor-defined byte data to be decoded by the OS driver. The bit field name `_VEN` is automatically created to refer to this portion of the resource descriptor.

PinList is a list of pin numbers on the ResourceSource that are described by this descriptor. The bit field name `_PIN` is automatically created to refer to this portion of the resource descriptor.

Description

The GpioIo macro evaluates to a buffer that contains a GPIO IO Connection resource descriptor. The format of this resource descriptor can be found in *GPIO Connection Descriptor*. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*

19.6.59 I2CSerialBusV2 (I2C Serial Bus Connection Resource Descriptor (Version 2) Macro)

Syntax:

```
I2CSerialBusV2 (SlaveAddress, SlaveMode, ConnectionSpeed,
AddressingMode, ResourceSource, ResourceSourceIndex, ResourceUsage,
DescriptorName, Shared, VendorData)
```

Arguments

SlaveAddress is the I2C bus address for this connection. The bit field name `_ADR` is automatically created to refer to this portion of the resource descriptor.

SlaveMode is an optional argument and can be either `ControllerInitiated` or `DeviceInitiated`. `ControllerInitiated` is the default. The bit field name `_SLV` is automatically created to refer to this portion of the resource descriptor.

ConnectionSpeed is the maximum connection speed supported by this connection, in hertz. The bit field name `_SPE` is automatically created to refer to this portion of the resource descriptor.

AddressingMode is an optional argument and can be either `AddressingMode7Bit` or `AddressingMode10Bit`. `AddressingMode7Bit` is the default. The bit field name `_MOD` is automatically created to refer to this portion of the resource descriptor.

ResourceSource is a string which uniquely identifies the I2C bus controller referred to by this descriptor. `ResourceSource` can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.

ResourceSourceIndex is an optional argument and is assumed to be 0 for this revision.

ResourceUsage is an optional argument and is assumed to be `ResourceConsumer` for this revision.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Shared is an optional argument and can be either **Shared** or **Exclusive**. If not specified, **Exclusive** is assumed. The bit field name `_SHR` is automatically created to refer to this portion of the resource descriptor. This was added in ACPI 6.1, and changed `I2CSerialBus` to `I2CSerialBusV2`.

VendorData is an optional argument that specifies an object to be decoded by the OS driver. It is a `RawDataBuffer`. The bit field name `_VEN` is automatically created to refer to this portion of the resource descriptor.

Description

The `I2CSerialBusV2` macro evaluates to a buffer that contains an *I2C Serial Bus Connection Resource Descriptor*. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.60 If (Conditional Execution)

Syntax:

```
If ( Predicate ) {TermList}
```

Arguments

Predicate is evaluated as an Integer.

Description

If the Predicate is non-zero, the term list of the If term is executed.

Example

The following examples all check for bit 3 in Local0 being set, and clear it if set:

```
// example 1
If (And (Local0, 4))
{
    XOr (Local0, 4, Local0)
}

// example 2
Store (4, Local2)
If (And (Local0, Local2))
{
    XOr (Local0, Local2, Local0)
}
```

19.6.61 Include (Include Additional ASL File)**Syntax:**

```
Include (FilePathName)
```

Arguments

FilePathName is a StringData data type that contains the full OS file system path.

Description

Include another file that contains ASL terms to be inserted in the current file of ASL terms. The file must contain elements that are grammatically correct in the current scope.

Example:

```
Include ("dataobj.asl")
```

19.6.62 Increment (Integer Increment)**Syntax:**

```
Increment (Addend) => Integer
```

Destination = Source [Index] => ObjectReference

Addend ++ => Integer

Arguments

Addend is evaluated as an Integer.

Description

Add one to the Addend and place the result back in Addend. Equivalent to Add (Addend, 1, Addend). Overflow conditions are ignored and the result of an overflow is zero.

19.6.63 Index (Indexed Reference To Member Object)

Syntax:

```
Index (Source, Index, Destination) => ObjectReference
```

Destination = Source [Index] => ObjectReference

Arguments

Source is evaluated to a buffer, string, or package data type. *Index* is evaluated to an integer. The reference to the *n*th object (where *n* = *Index*) within *Source* is optionally stored as a reference into *Destination*.

Description

When *Source* evaluates to a Buffer, *Index* returns a reference to a Buffer Field containing the *n*th byte in the buffer. When *Source* evaluates to a String, *Index* returns a reference to a Buffer Field containing the *n*th character in the string. When *Source* evaluates to a Package, *Index* returns a reference to the *n*th object in the package.

19.6.63.1 Index with Packages

The following example ASL code shows a way to use the *Index* term to store into a local variable the sixth element of the first package of a set of nested packages:

```
Name (I00D, Package () {
    Package () {
        0x01, 0x03F8, 0x03F8, 0x01, 0x08, 0x01, 0x25, 0xFF, 0xFE, 0x00, 0x00
    },
    Package () {
        0x01, 0x02F8, 0x02F8, 0x01, 0x08, 0x01, 0x25, 0xFF, 0xBE, 0x00, 0x00
    },
    Package () {
        0x01, 0x03E8, 0x03E8, 0x01, 0x08, 0x01, 0x25, 0xFF, 0xFA, 0x00, 0x00
    },
    Package () {
        0x01, 0x02E8, 0x02E8, 0x01, 0x08, 0x01, 0x25, 0xFF, 0xBA, 0x00, 0x00
    },
    Package() {
        0x01, 0x0100, 0x03F8, 0x08, 0x08, 0x02, 0x25, 0x20, 0x7F, 0x00, 0x00
    }
})

// Get the 6th element of the first package

Store (DerefOf (Index (DerefOf (Index (I00D, 0)), 5)), Local0)
```

Note

DerefOf is necessary in the first operand of the *Store* operator in order to get the actual object, rather than just a reference to the object. If *DerefOf* were not used, then *Local0* would contain an object reference to the sixth element in the first package rather than the number 1.

19.6.63.2 Index with Buffers

The following example ASL code shows a way to store into the third byte of a buffer:

```
Name (BUFF, Buffer () {0x01, 0x02, 0x03, 0x04, 0x05})
// Store 0x55 into the third byte of the buffer
Store (0x55, Index (BUFF, 2))
```

The Index operator returns a reference to an 8-bit Buffer Field (similar to that created using CreateByteField).

If Source is evaluated to a buffer data type, the ObjectReference refers to the byte at Index within Source. If Source is evaluated to a buffer data type, a Store operation will only change the byte at Index within Source.

The following example ASL code shows the results of a series of Store operations:

```
Name (SRCB, Buffer () {0x10, 0x20, 0x30, 0x40})
Name (BUFF, Buffer () {0x1, 0x2, 0x3, 0x4})
```

The following will store 0x78 into the 3rd byte of the destination buffer:

```
Store (0x12345678, Index (BUFF, 2))
```

The following will store 0x10 into the 2nd byte of the destination buffer:

```
Store (SRCB, Index (BUFF, 1))
```

The following will store 0x41 (an 'A') into the 4th byte of the destination buffer:

```
Store ("ABCDEFGH", Index (BUFF, 3))
```

Note

Compatibility Note: First introduced in ACPI 2.0. In ACPI 1.0, the behavior of storing data larger than 8-bits into a buffer using Index was undefined.

19.6.63.3 Index with Strings

The following example ASL code shows a way to store into the 3rd character in a string:

```
Name (STR, "ABCDEFGHIJKL")
// Store 'H' (0x48) into the third character to the string
Store ("H", Index (STR, 2))
```

The Index operator returns a reference to an 8-bit Buffer Field (similar to that created using CreateByteField).

Note

Compatibility Note: First introduced in ACPI 2.0.

19.6.64 IndexField (Declare Index/Data Fields)

Syntax:

```
IndexField (IndexName, DataName, AccessType, LockRule, UpdateRule) {FieldUnitList}
```

Arguments

IndexName is evaluated as a NameString and refers to a Field Unit object.

DataName is evaluated as a NameString and refers to a Field Unit object.

AccessType, *LockRule*, *UpdateRule*, and *FieldList* are the same format as the Field term.

Description

Creates a series of named data objects whose data values are fields within a larger object accessed by an index/data-style reference to *IndexName* and *DataName*.

This encoding is used to define named data objects whose data values are fields within an index/data register pair. This provides a simple way to declare register variables that occur behind a typical index and data register pair.

Accessing the contents of an indexed field data object will automatically occur through the *DataName* object by using an *IndexName* object aligned on an *AccessType* boundary, with synchronization occurring on the operation region that contains the index data variable, and on the Global Lock if specified by *LockRule*.

The value written to the *IndexName* register is defined to be a byte offset that is aligned on an *AccessType* boundary. For example, if *AccessType* is *DWordAcc*, valid index values are 0, 4, 8, etc. This value is always a byte offset and is independent of the width or access type of the *DataName* register.

Example

The following example contains a block of ASL sample code using *IndexField*, that:

- (a) Creates an index/data register in system I/O space made up of 8-bit registers.
- (b) Creates a FET0 field within the indexed range.

```
Method (EX1) {
    // Define a 256-byte operational region in SystemIO space
    // and name it GI00

    OperationRegion (GI00, 1, 0x125, 0x100)

    // Create a field named Preserve structured as a sequence
    // of index and data bytes

    Field (GI00, ByteAcc, NoLock, WriteAsZeros) {
        IDX0, 8,
        DAT0, 8,
        .
        .
        .
    }
    // Create an IndexField within IDX0 & DAT0 which has
    // FETs in the first two bits of indexed offset 0,
    // and another 2 FETs in the high bit on indexed
    // 2F and the low bit of indexed offset 30
```

(continues on next page)

(continued from previous page)

```

IndexField (IDX0, DAT0, ByteAcc, NoLock, Preserve) {
    FET0, 1,
    FET1, 1,
    Offset (0x2f),           // skip to byte offset 2f
    , 7,                     // skip another 7 bits
    FET3, 1,
    FET4, 1
}

// Clear FET3 (index 2F, bit 7)

Store (Zero, FET3)

} // End EX1

```

19.6.65 Interrupt (Interrupt Resource Descriptor Macro)

Syntax:

```

Interrupt ( ResourceUsage, EdgeLevel, ActiveLevel, Shared,
ResourceSourceIndex, ResourceSource, DescriptorName ) { InterruptList
} => Buffer

```

Arguments

ResourceUsage describes whether the device consumes the specified interrupt (ResourceConsumer) or produces it for use by a child device (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

EdgeLevel describes whether the interrupt is edge triggered (Edge) or level triggered (Level). The field *DescriptorName*._HE is automatically created to refer to this portion of the resource descriptor, where ‘1’ is Edge and ‘0’ is Level.

ActiveLevel describes whether the interrupt is active-high (ActiveHigh) or active-low (ActiveLow). The field *DescriptorName*._LL is automatically created to refer to this portion of the resource descriptor, where ‘1’ is ActiveLow and ‘0’ is ActiveHigh.

Shared describes whether the interrupt can be shared with other devices (Shared) or not (Exclusive), and whether it is capable of waking the system from a low-power idle or system sleep state (SharedAndWake or ExclusiveAndWake). The field *DescriptorName*._SHR is automatically created to refer to this portion of the resource descriptor, where ‘1’ is Shared and ‘0’ is Exclusive. If nothing is specified, then Exclusive is assumed.

ResourceSourceIndex evaluates to an integer between 0x00 and 0xFF and describes the resource source index. If it is not specified, then it is not generated. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource evaluates to a string which uniquely identifies the resource source. If it is not specified, it is not generated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a zero value is assumed.

DescriptorName evaluates to a name string which refers to the entire resource descriptor.

InterruptList is a comma-delimited list on integers, at least one value is required. Each integer represents a 32-bit interrupt number. At least one interrupt must be defined, and there may be no duplicates in the list. The field “DescriptorName._INT” is automatically created to refer to this portion of the resource descriptor.

Description

The Interrupt macro evaluates to a buffer that contains an interrupt resource descriptor. The format of this descriptor can be found in [Section 6.4.3.6](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

The interrupt macro uses the *ResourceUsage* field to distinguish two types of devices, a *ResourceProducer* and a *ResourceConsumer*.

A **ResourceProducer** represents a device that can forward interrupts from one or more devices to processors under the OSPM. Usage of ResourceProducer within interrupt macros is undefined and will be ignored by the OSPM. Declaring interrupt macros as ResourceProducer is not recommended.

A **ResourceConsumer** is a device that consumes the interrupts declared in the InterruptList. Most devices fall under this category and use this method to declare the interrupts that can be generated by that device. The interrupt descriptors declared as ResourceConsumer, are generated by either the main interrupt controller described in the MADT or by a device that acts as an “interrupt producer”. The *ResourceSource* field is used to make this distinction. If this is omitted, the interrupt numbers in the InterruptList identify Global System Interrupts, GSIs, and these interrupts target the main interrupt controller described in the MADT (see [Section 5.2.12](#)). The ResourceSource field may also provide the name of a device that is an “interrupt producer”. In this case the interrupt numbers in the InterruptList refer to the private interrupt number space of the indicated an interrupt set of the “interrupt producer” device.

The **ResourceSourceIndex** parameter is reserved. If a platform specifies “Interrupt ResourceSource support” in the Platform-Wide _OSC (bit 13 in [Table 6.13](#)), the ResourceSourceIndex parameter must be zero.

The following example illustrates how to specify consumption of a “secondary interrupt”. In this example, the device SDC0 consumes a secondary interrupt from MUX0, which multiplexes a group of secondary interrupts lines and generates a single summary interrupt (also referred to as an “interrupt producer”). The device driver for MUX0 is expected to generate a specific software based secondary interrupt based on implementation defined details of that device:

```
Scope(\_SB) {

    Device(MUX0){
        Name(_HID, "ACME0F0F") // vendor specific interrupt combiner
        Name(_UID, 0)
        Name(_CRS, ResourceTemplate () {
            //Register Interface
            MEMORY32FIXED(ReadWrite, 0x30000000, 0x200, )
            //Summary Interrupt line (GSI 51)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Exclusive) {51}
        })
    }

    Device(SDC0){
        Name(_HID, EISAID("PNP0D40")) // SDA Standard Compliant SD Host Controller
        Name(_UID, 0)
        Name(_CRS, ResourceTemplate() {
            //Register Interface
            MEMORY32FIXED(ReadWrite, 0xFF000000, 0x200, )
            // Secondary Interrupt 10 from interrupt combiner MUX0
            Interrupt(ResourceConsumer, Edge, ActiveHigh, Exclusive, 0, "\\_SB.MUX0"){10}
        })
    }
}
```

19.6.66 IO (IO Resource Descriptor Macro)

Syntax:

```
IO (Decode , AddressMin, AddressMax, AddressAlignment, RangeLength, DescriptorName) => Buffer
```

Argument

Decode describes whether the I/O range uses 10-bit decode (Decode10) or 16-bit decode (Decode16). The field *DescriptorName*.*_DEC* is automatically created to refer to this portion of the resource descriptor, where ‘1’ is Decode16 and ‘0’ is Decode10.

AddressMin evaluates to a 16-bit integer that specifies the minimum acceptable starting address for the I/O range. It must be an even multiple of *AddressAlignment*. The field *DescriptorName*.*_MIN* is automatically created to refer to this portion of the resource descriptor.

AddressMax evaluates to a 16-bit integer that specifies the maximum acceptable starting address for the I/O range. It must be an even multiple of *AddressAlignment*. The field *DescriptorName*.*_MAX* is automatically created to refer to this portion of the resource descriptor.

AddressAlignment evaluates to an 8-bit integer that specifies the alignment granularity for the I/O address assigned. The field *DescriptorName*.*_ALN* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to an 8-bit integer that specifies the number of bytes in the I/O range. The field *DescriptorName*.*_LEN* is automatically created to refer to this portion of the resource descriptor.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The IO macro evaluates to a buffer that contains an IO resource descriptor. The format of the IO descriptor can be found in [Section 6.4.2.5](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.67 IRQ (Interrupt Resource Descriptor Macro)

Syntax:

```
IRQ ( EdgeLevel, ActiveLevel, Shared, DescriptorName ) {InterruptList} => Buffer
```

Arguments

EdgeLevel describes whether the interrupt is edge triggered (Edge) or level triggered (Level). The field *DescriptorName*.*_HE* is automatically created to refer to this portion of the resource descriptor, where ‘1’ is Edge and ‘0’ is Level.

ActiveLevel describes whether the interrupt is active-high (ActiveHigh) or active-low (ActiveLow). The field *DescriptorName*.*_LL* is automatically created to refer to this portion of the resource descriptor, where ‘1’ is ActiveLow and ‘0’ is ActiveHigh.

Shared describes whether the interrupt can be shared with other devices (Shared) or not (Exclusive), and whether it is capable of waking the system from a low-power idle or system sleep state (SharedAndWake or ExclusiveAndWake). The field *DescriptorName*.*_SHR* is automatically created to refer to this portion of the resource descriptor, where ‘1’ is Shared and ‘0’ is Exclusive. If nothing is specified, then Exclusive is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

InterruptList is a comma-delimited list of integers in the range 0 through 15, at least one value is required. There may be no duplicates in the list.

Description

The IRQ macro evaluates to a buffer that contains an IRQ resource descriptor. The format of the IRQ descriptor can be found in [Section 6.4.2.1](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.68 IRQNoFlags (Interrupt Resource Descriptor Macro)

Syntax:

```
IRQNoFlags ( DescriptorName ) { InterruptList } => Buffer
```

Arguments

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer.

InterruptList is a comma-delimited list of integers in the range 0 through 15, at least one value is required. There may be no duplicates in the list

The IRQNoFlags macro evaluates to a buffer that contains an active-high, edge-triggered IRQ resource descriptor. The format of the IRQ descriptor can be found in [Section 6.4.2.1](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.69 LAnd (Logical And)

Syntax:

```
LAnd ( Source1, Source2 ) => Boolean
```

Source1 && *Source2* => Boolean

Arguments

Source1 and *Source2* are evaluated as integers.

Description

If both values are non-zero, True is returned: otherwise, False is returned.

19.6.70 LEqual (Logical Equal)

Syntax:

```
LEqual (Source1, Source2) => Boolean
```

Source1 == *Source2* => Boolean

Arguments

Source1 and *Source2* must each evaluate to an integer, a string, or a buffer. The data type of *Source1* dictates the required type of *Source2*. *Source2* is implicitly converted if necessary to match the type of *Source1*.

Description

If the values are equal, True is returned; otherwise, False is returned. For integers, a numeric compare is performed. For strings and buffers, True is returned only if both lengths are the same and the result of a byte-wise compare indicates exact equality.

19.6.71 LGreater (Logical Greater)

Syntax:

```
LLGreater (Source1, Source2) => Boolean
```

Source1 > *Source2* => Boolean

Arguments

Source1 and *Source2* must each evaluate to an integer, a string, or a buffer. The data type of *Source1* dictates the required type of *Source2*. *Source2* is implicitly converted if necessary to match the type of *Source1*.

Description

If *Source1* is greater than *Source2*, True is returned; otherwise, False is returned. For integers, a numeric comparison is performed. For strings and buffers, a lexicographic comparison is performed. True is returned if a byte-wise (unsigned) compare discovers at least one byte in *Source1* that is numerically greater than the corresponding byte in *Source2*. False is returned if at least one byte in *Source1* is numerically less than the corresponding byte in *Source2*. In the case of byte-wise equality, True is returned if the length of *Source1* is greater than *Source2*, False is returned if the length of *Source1* is less than or equal to *Source2*.

19.6.72 LGreaterEqual (Logical Greater Than Or Equal)

Syntax:

```
LLGreaterEqual (Source1, Source2) => Boolean
```

Source1 >= *Source2* => Boolean

Arguments

Source1 and *Source2* must each evaluate to an integer, a string, or a buffer. The data type of *Source1* dictates the required type of *Source2*. *Source2* is implicitly converted if necessary to match the type of *Source1*.

Description

If *Source1* is greater than or equal to *Source2*, True is returned; otherwise, False is returned. Equivalent to LNot(LLess()). See the description of the LLess operator.

19.6.73 LLess (Logical Less)

Syntax:

```
LLess (Source1, Source2) => Boolean
Source1 < source2 => Boolean
```

Arguments

Source1 and *Source2* must each evaluate to an integer, a string, or a buffer. The data type of *Source1* dictates the required type of *Source2*. *Source2* is implicitly converted if necessary to match the type of *Source1*.

Description

If *Source1* is less than *Source2*, True is returned; otherwise, False is returned. For integers, a numeric comparison is performed. For strings and buffers, a lexicographic comparison is performed. True is returned if a byte-wise (unsigned) compare discovers at least one byte in *Source1* that is numerically less than the corresponding byte in *Source2*. False is returned if at least one byte in *Source1* is numerically greater than the corresponding byte in *Source2*. In the case of byte-wise equality, True is returned if the length of *Source1* is less than *Source2*, False is returned if the length of *Source1* is greater than or equal to *Source2*.

19.6.74 LLessEqual (Logical Less Than Or Equal)

Syntax:

```
LLessEqual (Source1, Source2) => Boolean
Source1 <= source2 => Boolean
```

Arguments

Source1 and *Source2* must each evaluate to an integer, a string, or a buffer. The data type of *Source1* dictates the required type of *Source2*. *Source2* is implicitly converted if necessary to match the type of *Source1*.

Description

If *Source1* is less than or equal to *Source2*, True is returned; otherwise False is returned. Equivalent to LNot(LGreater()). See the description of the LGreater operator.

19.6.75 LNot (Logical Not)

Syntax:

```
LNot (Source) => Boolean
```

! *Source* => Boolean

Arguments

Source is evaluated as an integer.

Description

If the value is zero True is returned; otherwise, False is returned.

19.6.76 LNotEqual (Logical Not Equal)

Syntax:

```
LNotEqual ( Source1, Source2 ) => Boolean
Source1 != Source2 => Boolean
```

Arguments

Source1 and Source2 must each evaluate to an integer, a string, or a buffer. The data type of Source1 dictates the required type of Source2. Source2 is implicitly converted if necessary to match the type of Source1.

Description

If Source1 is not equal to Source2, True is returned; otherwise False is returned. Equivalent to LNot(LEqual()). See the description of the LEqual operator.

19.6.77 Load (Load Definition Block)

Syntax:

```
Load (Object, Result) => Boolean
```

Arguments

The Object parameter can refer to one of the following object types:

1. An operation region field
2. An operation region directly
3. An ASL Buffer object

If the object is an operation region, the operation region must be in SystemMemory space. The Definition Block should contain an ACPI DESCRIPTION_HEADER of type SSDT. The Definition Block must be totally contained within the supplied operation region, operation region field, or Buffer object. OSPM reads this table into memory, the checksum is verified, and then it is loaded into the ACPI namespace.

Result is optional and is a Boolean indicating the status of the operation. A value of zero (false) means the operation failed. Any other value means that the operation was successful. Also, this value is always returned as the function return value.

Description

Performs a run-time load of a Definition Block. Any table loaded via an operation region must be in memory marked as AddressRangeReserved or AddressRangeNVS. The OS can also check the OEM Table ID and Revision ID against a database for a newer revision Definition Block of the same OEM Table ID and load it instead.

The default namespace location to load the Definition Block is relative to the root of the namespace. The new Definition Block can override this by specifying absolute names or by adjusting the namespace location using the Scope operator.

Loading a Definition Block is a synchronous operation. Upon completion of the operation, the Definition Block has been loaded. The control methods defined in the Definition Block are not executed during load time.

19.6.78 LoadTable (Load Definition Block From XSDT)

Syntax:

```
LoadTable ( SignatureString, OEMIDString, OEMTableIDString,
RootPathString, ParameterPathString, ParameterData ) => Boolean
```

Arguments

The XSDT is searched for a table where the Signature field matches SignatureString, the OEM ID field matches OEMIDString, and the OEM Table ID matches OEMTableIDString. All comparisons are case sensitive. If the SignatureString is greater than four characters, the OEMIDString is greater than six characters, or the OEMTableID is greater than eight characters, a run-time error is generated. The OS can also check the OEM Table ID and Revision ID against a database for a newer revision Definition Block of the same OEM Table ID and load it instead.

The RootPathString specifies the root of the Definition Block. It is evaluated using normal scoping rules, assuming that the scope of the LoadTable instruction is the current scope. The new Definition Block can override this by specifying absolute names or by adjusting the namespace location using the Scope operator. If RootPathString is not specified, "" is assumed.

If ParameterPathString and ParameterData are specified, the data object specified by ParameterData is stored into the object specified by ParameterPathString after the table has been added into the namespace. If the first character of ParameterPathString is a backslash (') or caret (^) character, then the path of the object is ParameterPathString. Otherwise, it is RootPathString.ParameterPathString. If the specified object does not exist, a run-time error is generated.

The status of the operation is returned as a Boolean. A value of zero (false) means the operation failed. Any other value means that the operation was successful.

Description

Performs a run-time load of a Definition Block from the XSDT. Any table referenced by LoadTable must be in memory marked by AddressRangeReserved or AddressRangeNVS.

Note: OSPM loads the DSDT and all SSDTs during initialization. As such, Definition Blocks to be conditionally loaded via LoadTable must contain signatures other than "SSDT".

Loading a Definition Block is a synchronous operation. Upon completion of the operation, the Definition Block has been loaded. The control methods defined in the Definition Block are not executed during load time.

Example:

```
Store (LoadTable ("OEM1", "MYOEM", "TABLE1", "\\_SB.PCI0", "MYD", Package () {0, "\\_SB.
↪PCI0"}), Local0)
```

This operation would search through the RSDT or XSDT for a table with the signature "OEM1," the OEM ID of "MYOEM," and the table ID of "TABLE1." If not found, it would store Zero in Local0. Otherwise, it will store a package containing 0 and "_SB.PCI0" into the variable at _SB.PCI0.MYD.

19.6.79 LocalX (Method Local Data Objects)

Syntax:

```
Local0 | Local1 | Local2 | Local3 | Local4 | Local5 | Local6 | Local7
```

Description

Up to 8 local objects can be referenced in a control method. On entry to a control method, these objects are uninitialized and cannot be used until some value or reference is stored into the object. Once initialized, these objects are preserved in the scope of execution for that control method.

19.6.80 LOr (Logical Or)

Syntax:

```
LOr ( Source1, Source2 ) => Boolean
Source1 || Source2 => Boolean
```

Arguments

Source1 and *Source2* are evaluated as integers.

Description

If either value is non-zero, True is returned; otherwise, False is returned.

19.6.81 Match (Find Object Match)

Syntax:

```
Match ( SearchPackage, Op1, MatchObject1, Op2, MatchObject2, StartIndex ) => Ones \Integer
```

Arguments

SearchPackage is evaluated to a package object and is treated as a one-dimension array. Each package element must evaluate to either an integer, a string, or a buffer. Uninitialized package elements and elements that do not evaluate to integers, strings, or buffers are ignored. *Op1* and *Op2* are match operators. *MatchObject1* and *MatchObject2* are the objects to be matched and must each evaluate to either an integer, a string, or a buffer. *StartIndex* is the starting index within the *SearchPackage*.

Description

A comparison is performed for each element of the package, starting with the index value indicated by *StartIndex* (0 is the first element). If the element of *SearchPackage* being compared against is called *P[i]*, then the comparison is:

```
If (P[i] Op1 MatchObject1) and (P[i] Op2 MatchObject2) then Match => i is returned.
```

If the comparison succeeds, the index of the element that succeeded is returned; otherwise, the constant object *Ones* is returned. The data type of the *MatchObject* dictates the required type of the package element. If necessary, the package element is implicitly converted to match the type of the *MatchObject*. If the implicit conversion fails for any reason, the package element is ignored (no match.)

Op1 and *Op2* have the values and meanings listed in the following table.

Table 19.35: Match Term Operator Meanings

Operator	Encoding	Macro
TRUE - A don't care, always returns TRUE	0	MTR
EQ - Returns TRUE if <i>P[i]</i> == <i>MatchObject</i>	1	MEQ
LE - Returns TRUE if <i>P[i]</i> <= <i>MatchObject</i>	2	MLE
LT - Returns TRUE if <i>P[i]</i> < <i>MatchObject</i>	3	MLT
GE - Returns TRUE if <i>P[i]</i> >= <i>MatchObject</i>	4	MGE
GT - Returns TRUE if <i>P[i]</i> > <i>MatchObject</i>	5	MGT

Example

Following are some example uses of *Match*:

```

Name (P1,
Package () {1981, 1983, 1985, 1987, 1989, 1990, 1991, 1993, 1995, 1997, 1999, 2001}
)

// match 1993 == P1[i]
Match (P1, MEQ, 1993, MTR, 0, 0) // -> 7, since P1[7] == 1993

// match 1984 == P1[i]
Match (P1, MEQ, 1984, MTR, 0, 0) // -> ONES (not found)

// match P1[i] > 1984 and P1[i] <= 2000
Match (P1, MGT, 1984, MLE, 2000, 0) // -> 2, since P1[2]>1984 and P1[2]<=2000

// match P1[i] > 1984 and P1[i] <= 2000, starting with 3rd element
Match (P1, MGT, 1984, MLE, 2000, 3) // -> 3, first match at or past Start

```

19.6.82 Memory24 (Memory Resource Descriptor Macro)

Syntax:

```

Memory24 (ReadAndWrite, AddressMinimum, AddressMaximum, AddressAlignment, RangeLength,
↳DescriptorName)

```

Arguments

ReadAndWrite specifies whether or not the memory region is read-only (ReadOnly) or read/write (ReadWrite). If nothing is specified, then ReadWrite is assumed. The 1-bit field *DescriptorName._RW* is automatically created to refer to this portion of the resource descriptor, where '1' is ReadWrite and '0' is ReadOnly.

AddressMinimum evaluates to a 16-bit integer that specifies bits [8:23] of the lowest possible base address of the memory range. All other bits are assumed to be zero. The value must be an even multiple of *AddressAlignment*. The 16-bit field *DescriptorName._MIN* is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 16-bit integer that specifies bits [8:23] of the highest possible base address of the memory range. All other bits are assumed to be zero. The value must be an even multiple of *AddressAlignment*. The 16-bit field *DescriptorName._MAX* is automatically created to refer to this portion of the resource descriptor.

AddressAlignment evaluates to a 16-bit integer that specifies bits [0:15] of the required alignment for the memory range. All other bits are assumed to be zero. The address selected must be an even multiple of this value. The 16-bit field *DescriptorName._ALN* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 16-bit integer that specifies the total number of bytes decoded in the memory range. The 16-bit field *DescriptorName._LEN* is automatically created to refer to this portion of the resource descriptor. The range length provides the length of the memory range in 256 byte blocks.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The *Memory24* macro evaluates to a buffer that contains an 24-bit memory descriptor. The format of this descriptor can be found in [Table 6.40](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

Note

The use of Memory24 is deprecated and should not be used in new designs.

19.6.83 Memory32 (Memory Resource Descriptor Macro)

Syntax:

```
Memory32 (ReadAndWrite, AddressMinimum, AddressMaximum, AddressAlignment, RangeLength, _DescriptorName)
```

Arguments

ReadAndWrite specifies whether or not the memory region is read-only (ReadOnly) or read/write (ReadWrite). If nothing is specified, then ReadWrite is assumed. The 1-bit field DescriptorName._RW is automatically created to refer to this portion of the resource descriptor, where '1' is ReadWrite and '0' is ReadOnly.

AddressMinimum evaluates to a 32-bit integer that specifies the lowest possible base address of the memory range. The value must be an even multiple of AddressAlignment. The 32-bit field DescriptorName._MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 32-bit integer that specifies the highest possible base address of the memory range. The value must be an even multiple of AddressAlignment. The 32-bit field DescriptorName._MAX is automatically created to refer to this portion of the resource descriptor.

AddressAlignment evaluates to a 32-bit integer that specifies the required alignment for the memory range. The address selected must be an even multiple of this value. The 32-bit field DescriptorName._ALN is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 32-bit integer that specifies the total number of bytes decoded in the memory range. The 32-bit field DescriptorName._LEN is automatically created to refer to this portion of the resource descriptor. The range length provides the length of the memory range in 1 byte blocks.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The Memory32 macro evaluates to a buffer that contains a 32-bit memory descriptor, which describes a memory range with a minimum, a maximum and an alignment. The format of this descriptor can be found in [Section 6.4.3.3](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.84 Memory32Fixed (Memory Resource Descriptor Macro)

Syntax:

```
Memory32Fixed (ReadAndWrite, AddressBase, RangeLength, DescriptorName)
```

Arguments

ReadAndWrite specifies whether or not the memory region is read-only (ReadOnly) or read/write (ReadWrite). If nothing is specified, then ReadWrite is assumed. The 1-bit field DescriptorName._RW is automatically created to refer to this portion of the resource descriptor, where '1' is ReadWrite and '0' is ReadOnly.

AddressBase evaluates to a 32-bit integer that specifies the base address of the memory range. The 32-bit field *DescriptorName*. *_BAS* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 32-bit integer that specifies the total number of bytes decoded in the memory range. The 32-bit field *DescriptorName*. *_LEN* is automatically created to refer to this portion of the resource descriptor.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The `Memory32Fixed` macro evaluates to a buffer that contains a 32-bit memory descriptor, which describes a fixed range of memory addresses. The format of this memory descriptor can be found in [Section 6.4.3.4](#). This macro is designed to be used inside of a *ResourceTemplate* (*Resource To Buffer Conversion Macro*).

19.6.85 Method (Declare Control Method)

Syntax:

```
Method ( MethodName, NumArgs, SerializeRule, SyncLevel, ReturnType, ParameterTypes )
↪ {TermList}
```

Arguments

MethodName is evaluated as a `NameString` data type.

NumArgs is optional and is the required number of arguments to be passed to the method, evaluated as an `Integer` data type. If not specified, the default value is zero arguments. Up to 7 arguments may be passed to a method. These arguments may be referenced from within the method as `Arg0` through `Arg6`.

SerializeRule is optional and is a flag that defines whether the method is serialized or not and is one of the following: `Serialized` or `NotSerialized`. A method that is serialized cannot be reentered by additional threads. If not specified, the default is `NotSerialized`.

SyncLevel is optional and specifies the synchronization level for the method (0 - 15). If not specified, the default sync level is zero.

ReturnType is optional and specifies the type(s) of the object(s) returned by the method. If the method does not return an object, then nothing is specified or `UnknownObj` is specified. To specify a single return type, simply use the `ObjectTypeKeyword` (e.g. `IntObj`, `PkgObj`, etc.). To specify multiple possible return types, enclose the comma-separated `ObjectTypeKeywords` with braces. For example: `{IntObj, BuffObj}`.

ParameterTypes is optional and specifies the type of the method parameters. It is a comma-separated, variable-length list of the expected object type or types for each of the method parameters, enclosed in braces. For each parameter, the parameter type consists of either an `ObjectTypeKeyword` or a comma-separated sub-list of `ObjectTypeKeywords` enclosed in braces. If *ParameterTypes* is specified, the number of parameters must match *NumArgs*.

TermList is a variable-length list of executable ASL statements representing the body of the control method.

Description

Creates a new control method of name *MethodName*. This is a named package containing a series of object references that collectively represent a control method, which is a procedure that can be invoked to perform computation. Method opens a name scope.

System software executes a control method by referencing the objects in the package in order. For more information on method execution, see [Section 5.5.2](#)

The current namespace location used during name creation is adjusted to be the current location on the namespace tree. Any names created within this scope are “below” the name of this package. The current namespace location is assigned to the method package, and all namespace references that occur during control method execution for this package are relative to that location.

If a method is declared as Serialized, an implicit mutex associated with the method object is acquired at the specified SyncLevel. If no SyncLevel is specified, SyncLevel 0 is assumed. The serialize rule can be used to prevent reentering of a method. This is especially useful if the method creates namespace objects. Without the serialize rule, the reentering of a method will fail when it attempts to create the same namespace object.

There are eight local variables automatically available for each method, referenced as Local0 through Local7. These locals may be used to store any type of ASL object.

Also notice that all namespace objects created by a method have temporary lifetime. When method execution exits, the created objects will be destroyed.

Examples

The following block of ASL sample code shows a use of Method for defining a control method that turns on a power resource.

```
Method (_ON) {
    Store (One, GPIO.IDEP) // assert power
    Sleep (10) // wait 10ms
    Store (One, GPIO.IDER) // de-assert reset#
    Stall (10) // wait 10us
    Store (Zero, GPIO.IDEI) // de-assert isolation
}
```

This method is an implementation of _SRS (Set Resources). It shows the use of a method argument and two method locals:

```
Method (_SRS, 1, NotSerialized)
{
    CreateWordField (Arg0, One, IRQW)
    Store (\_SB.PCI0.PID1.IENA, Local1)
    Or (IRQW, Local1, Local1)
    Store (Local1, \_SB.PCI0.PID1.IENA)
    FindSetRightBit (IRQW, Local0)
    If (Local0)
    {
        Decrement (Local0)
        Store (Local0, \_SB.PCI0.PID1.IN01)
    }
}
```

19.6.86 Mid (Extract Portion of Buffer or String)

Syntax:

```
Mid ( Source, Index, Length, Result ) => Buffer or String
```

Arguments

Source is evaluated as either a Buffer or String. *Index* and *Length* are evaluated as Integers.

Description

If *Source* is a buffer, then *Length* bytes, starting with the *Index*th byte (zero-based) are optionally copied into *Result*. If *Index* is greater than or equal to the length of the buffer, then the result is an empty buffer. Otherwise, if *Index* + *Length* is greater than or equal to the length of the buffer, then only bytes up to and including the last byte are included in the result.

If *Source* is a string, then *Length* characters, starting with the *Index*th character (zero-based) are optionally copied into *Result*. If *Index* is greater than or equal to the length of the buffer, then the result is an empty string. Otherwise, if *Index* + *Length* is greater than or equal to the length of the string, then only bytes up to an including the last character are included in the result.

19.6.87 Mod (Integer Modulo)

Syntax:

```
Mod ( Dividend, Divisor, Result ) => Integer
Result = Dividend % Divisor => Integer
Result %= Divisor => Integer
```

Arguments

Dividend and *Divisor* are evaluated as Integers.

Description

The *Dividend* is divided by *Divisor*, and then the resulting remainder is optionally stored into *Result*. If *Divisor* evaluates to zero, a fatal exception is generated.

19.6.88 Multiply (Integer Multiply)

Syntax:

```
Multiply ( Multiplicand, Multiplier, Result ) => Integer
Result = Multiplicand \ Multiplier => Integer
Result \= Multiplier => Integer
```

Arguments

Multiplicand and *Multiplier* are evaluated as Integers.

Description

The *Multiplicand* is multiplied by *Multiplier* and the result is optionally stored into *Result*. Overflow conditions are ignored and results are undefined.

19.6.89 Mutex (Declare Synchronization/Mutex Object)

Syntax:

```
Mutex ( MutexName, SyncLevel )
```

Arguments

The MutexName is evaluated as a NameString data type.

The SyncLevel is optional and specifies the logical nesting level of the Mutex synchronization object. The current sync level is maintained internally for a thread, and represents the greatest SyncLevel among mutex objects that are currently acquired by the thread. The SyncLevel of a thread, before acquiring any mutexes, is zero. The SyncLevel of the Global Lock (_GL) is zero. If not specified, the default sync level value is zero.

Description

Creates a data mutex synchronization object named MutexName, with a synchronization level from 0 to 15 as specified by the Integer SyncLevel.

A mutex synchronization object provides a control method with a mechanism for waiting for certain events. To prevent deadlocks, wherever more than one synchronization object must be owned, the synchronization objects must always be released in the order opposite the order in which they were acquired.

The SyncLevel parameter declares the logical nesting level of the synchronization object. The current sync level is maintained internally for a thread, and represents the greatest SyncLevel among mutex objects that are currently acquired by the thread. The SyncLevel of a thread before acquiring any mutexes is zero. The SyncLevel of the Global Lock (_GL) is zero.

All Acquire terms must refer to a synchronization object with a SyncLevel that is equal or greater than the current level, and all Release terms must refer to a synchronization object with a SyncLevel that is equal to the current level.

Mutex synchronization provides the means for mutually exclusive ownership. Ownership is acquired using an Acquire term and is released using a Release term. Ownership of a Mutex must be relinquished before completion of any invocation. For example, the top-level control method cannot exit while still holding ownership of a Mutex. Acquiring ownership of a Mutex can be nested (can be acquired multiple times by the same thread).

19.6.90 Name (Declare Named Object)

Syntax:

```
Name ( ObjectName, Object )
```

Arguments

Creates a new object named ObjectName. Attaches Object to ObjectName in the Global ACPI namespace.

Description

Creates ObjectName in the namespace, which references the Object.

Example

The following example creates the name PTTX in the root of the namespace that references a package.

```
Name (\PTTX, // Port to Port Translate Table
    Package () {Package () {0x43, 0x59}, Package () {0x90, 0xFF}}
)
```

The following example creates the name CNT in the root of the namespace that references an integer data object with the value 5:

```
Name (\CNT, 5)
```

19.6.91 NAnd (Integer Bitwise Nand)

Syntax:

```
NAnd (Source1, Source2, Result) => Integer
```

Arguments

Source1 and *Source2* are evaluated as Integers.

Description

A bitwise NAND is performed and the result is optionally stored in Result.

19.6.92 NoOp Code (No Operation)

Syntax:

```
NoOp
```

Description

This operation has no effect.

19.6.93 NOr (Integer Bitwise Nor)

Syntax:

```
NOr (Source1, Source2, Result) => Integer
```

Arguments

Source1 and *Source2* are evaluated as Integers.

Description

A bitwise NOR is performed and the result is optionally stored in Result.

19.6.94 Not (Integer Bitwise Not)

Syntax:

```
Not (Source, Result) => Integer
```

Result = $\sim$ *Source* => Integer

Arguments

Source is evaluated as an integer data type.

Description

A bitwise NOT is performed and the result is optionally stored in Result.

19.6.95 Notify (Notify Object of Event)

Syntax:

```
Notify (Object, NotificationValue)
```

Arguments

Notifies the OS that the NotificationValue for the Object has occurred. Object must be a reference to a device, processor, or thermal zone object.

Description

Object type determines the notification values. For example, the notification values for a thermal zone object are different from the notification values used for a device object. Undefined notification values are treated as reserved and are ignored by the OS.

For lists of defined Notification values, see [Section 5.6.6](#)

19.6.96 Offset (Change Current Field Unit Offset)

Syntax:

```
Offset (ByteOffset)
```

Arguments

ByteOffset is the new offset (in bytes) for the next FieldUnit within a FieldList.

Description

The Offset operator is used within a FieldList to specify the byteOffset of the next defined field within its parent operation region. This can be used instead of defining the bit lengths that need to be skipped. All offsets are defined starting from zero, based at the starting address of the parent region.

19.6.97 ObjectType (Get Object Type)

Syntax:

```
ObjectType (Object) => Integer
```

Arguments

Object is any valid object.

Description

The execution result of this operation is an integer that has the numeric value of the object type for Object.

The object type codes are listed in the following table. Note that if this operation is performed on an object reference such as one produced by the Alias, Index, or RefOf statements, the object type of the base object is returned. For typeless objects such as predefined scope names (in other words, _SB, _GPE, etc.), the type value 0 (Uninitialized) is returned.

Table 19.36: Values Returned By the ObjectType Operator

Value	Object
0	Uninitialized
1	Integer
2	String
3	Buffer
4	Package
5	Field Unit
6	Device
7	Event
8	Method
9	Mutex
10	Operation Region
11	Power Resource
12	<i>Reserved</i>
13	Thermal Zone
14	Buffer Field
15	<i>Reserved</i>
16	Debug Object
>16	<i>Reserved</i>

19.6.98 One (Constant One Integer)

Syntax:

```
One => Integer
```

Description

The One operator returns an Integer with the value 1. Writes to this object are not allowed. The use of this operator can reduce AML code size, since it is represented by a one-byte AML opcode.

19.6.99 Ones (Constant Ones Integer)

Syntax:

```
Ones => Integer
```

Description

The Ones operator returns an Integer with all bits set to 1. Writes to this object are not allowed. The use of this operator can reduce AML code size, since it is represented by a one-byte AML opcode.

Note: The actual value of the integer returned by the Ones operator depends on the integer width of the DSDT. If the revision of the DSDT is 1 or less, the integer width is 32 bits and Ones returns 0xFFFFFFFF. If the revision of the DSDT is 2 or greater, the integer width is 64 bits and Ones returns 0xFFFFFFFFFFFFFFFF. This difference must be considered when performing comparisons against the Ones Integer.

19.6.100 OperationRegion (Declare Operation Region)

Syntax:

```
OperationRegion (RegionName, RegionSpace, Offset, Length)
```

Arguments

Declares an operation region named *RegionName*. *Offset* is the offset within the selected *RegionSpace* at which the region starts (byte-granular), and *Length* is the length of the region in bytes.

Description

An Operation Region is a type of data object where read or write operations to the data object are performed in some hardware space. For example, the Definition Block can define an Operation Region within a bus, or system I/O space. Any reads or writes to the named object will result in accesses to the I/O space.

Operation regions are regions in some space that contain hardware registers for exclusive use by ACPI control methods. In general, no hardware register (at least byte-granular) within the operation region accessed by an ACPI control method can be shared with any accesses from any other source, with the exception of using the Global Lock to share a region with the firmware. The entire Operation Region can be allocated for exclusive use to the ACPI subsystem in the host OS.

Operation Regions that are defined within the scope of a method are the exception to this rule. These Operation Regions are known as “Dynamic” since the OS has no idea that they exist or what registers they use until the control method is executed. Using a Dynamic SystemIO or SystemMemory Operation Region is not recommended since the OS cannot guarantee exclusive access. All other types of Operation Regions may be Dynamic.

Operation Regions define the overall base address and length of a hardware region, but they cannot be accessed directly by AML code. A Field object containing one or more FieldUnits is used to overlay the Operation Region in order to access individual areas of the Region. An individual FieldUnit within an Operation Region may be as small as one bit, or as large as the length of the entire Region. FieldUnit values are normalized (shifted and masked to the proper length.) The data type of a FieldUnit can be either a Buffer or an Integer, depending on the bit length of the FieldUnit. If the FieldUnit is smaller than or equal to the size of an Integer (in bits), it will be treated as an Integer. If the FieldUnit is larger than the size of an Integer, it will be treated as a Buffer. The size of an Integer is indicated by the DSDT header’s Revision field. A revision less than 2 indicates that the size of an Integer is 32 bits. A value greater than or equal to 2 signifies that the size of an Integer is 64 bits. For more information about data types and FieldUnit type conversion rules, see [Section 19.3.5.7](#).

An Operation Region object implicitly supports Mutex synchronization. Updates to the object, or a Field data object for the region, will automatically synchronize on the Operation Region object; however, a control method may also explicitly synchronize to a region to prevent other accesses to the region (from other control methods). Notice that according to the control method execution model, control method execution is non-preemptive. Because of this, explicit synchronization to an Operation Region needs to be done only in cases where a control method blocks or yields execution and where the type of register usage requires such synchronization.

The predefined Operation Region types specified in ACPI are shown in [Table 5.221](#)

Example

The following example ASL code shows the use of OperationRegion combined with Field to describe IDE 0 and 1 controlled through general I/O space, using one FET:

```
OperationRegion (GIO, SystemIO, 0x125, 0x1)
Field (GIO, ByteAcc, NoLock, Preserve) {
    IDEI, 1,          // IDEISO_EN - isolation buffer
    IDEP, 1,          // IDE_PWR_EN - power
    IDER, 1           // IDERST#_EN - reset#
}
```

19.6.101 Or (Integer Bitwise Or)

Syntax:

```
Or (Source1, Source2, Result) => Integer
*Result* = *Source1* \| *Source2* => Integer
*Result* \|= *Source1* => Integer
```

Arguments

Source1 and *Source2* are evaluated as Integers.

Description

A bitwise OR is performed and the result is optionally stored in *Result*.

19.6.102 Package (Declare Package Object)

Syntax:

```
Package (NumElements) {PackageList} => Package
```

Arguments

NumElements is evaluated as an Integer. *PackageList* is an initializer list of objects.

Description

Declares an unnamed aggregation of named data items, constants, and/or references to non-data namespace objects. The size of the package is *NumElements*. The *PackageList* contains the data items, constants, and/or object references used to initialize the package.

If *NumElements* is absent, it is automatically set by the ASL compiler to match the number of elements in the *PackageList*. If *NumElements* is present and greater than the number of elements in the *PackageList*, the default entry of type Uninitialized (see *ObjectType*) is used to initialize the package elements beyond those initialized from the *PackageList*.

There are three types of package elements allowed in the *PackageList*: *ConstantData Objects* (Integers, Strings, Buffers, and Packages), named references that resolve to *Data Objects* (Integers, Strings, Buffers, and Packages), and named references to objects other than *Data Objects*.

These constant terms are resolved at ASL compile time:

- Integer Constant
- String Constant
- Buffer Constant
- Package Constant

These Named References to *Data Objects* are resolved to actual data by the AML Interpreter at runtime:

- Integer reference
- String reference
- Buffer reference
- Buffer Field reference
- Field Unit reference

- Package reference

These Named References to non-Data Objects cannot be resolved to values. They are instead returned in the package as references:

- Device reference
- Event reference
- Method reference
- Mutex reference
- Operation Region reference
- Power Resource reference
- Processor reference
- Thermal Zone reference

Note

For Package elements of type Package (defining a subpackage), individual elements of the subpackage are resolved according to the rules above, both compile-time and runtime.

Evaluating an uninitialized element will yield a runtime error, but elements can be assigned values at runtime to define them (via the Index operator). It is a compile time error for *NumElements* to be less than the number of elements defined in the *PackageList*.

The ASL compiler can emit two different AML opcodes for a Package declaration, either *PackageOp* or *VarPackageOp*. For small, fixed-length packages, the *PackageOp* is used and this opcode is compatible with ACPI 1.0. A *VarPackageOp* will be emitted if any of the following conditions are true:

- The *NumElements* argument is a TermArg that can only be resolved at runtime.
- At compile time, *NumElements* resolves to a constant that is larger than 255.
- The *PackageList* contains more than 255 initializer elements.

Example:

```
Name (INT1, 0x1234)
Processor (CPU0, 0, 0x1010, 6) {}
PowerResource (PWR1, 0, 0) {}

Name (PKG1, Package () {
    0x3400,           // Integer Constant, resolved at compile time
    "Processor "      // String Constant, resolved at compile time
    \INT1             // Integer Reference, resolved to value at
                    // runtime
    \CPU0             // Object Reference, returned as a reference
                    // object
    Package () {      // Package Constant. Elements are resolved at
                    // both compile time and runtime
        0x4321,       // Integer Constant, resolved at compile time
        \INT1,        // Integer Reference, resolved to value at
                    // runtime
        \PWR1
    })
})
```

The runtime values of the parent package and subpackages are:

```
Package [Contains 0x05 Elements] {
  (00) Integer 0x00000000000003400
  (01) String [0x09] "Processor"
  (02) Integer 0x00000000000001234
  (03) Reference [Named Object] [CPU0] Processor
  (04) Package [Contains 0x03 Elements]
    (00) Integer 0x00000000000004321
    (01) Integer 0x00000000000001234
    (02) Reference [Named Object] [PWR1] Power
}
```

19.6.103 PinConfig (Pin Configuration Descriptor Macro)

Syntax:

Macro:

```
PinConfig (Shared/Exclusive, PinConfigType, PinConfigValue, ResourceSource,
ResourceSourceIndex, ResourceUsage, DescriptorName, VendorData) {Pin List}
```

Arguments

- *Shared* is an optional argument and can be either Shared or Exclusive. If not specified, Exclusive is assumed. The bit field name `_SHR` is automatically created to refer to this portion of the resource descriptor.
- *PinConfigType* can be one of the configuration types described below in *Pin Configuration Types and Values* . The bit field `_TYP` is automatically created to refer to this portion of the resource descriptor.
- *PinConfigValue* is one of the configurations values described below in *Pin Configuration Types and Values* . The bit field `_VAL` is automatically created to refer to this portion of the resource descriptor.
- *ResourceSource* is a string which uniquely identifies the pin controller referred to by this descriptor. ResourceSource can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.
- *ResourceSourceIndex* is an optional argument and is assumed to be 0 for this revision.
- *ResourceUsage* is an optional argument and is assumed to be ResourceConsumer for this revision.
- *DescriptorName* is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.
- *VendorData* is an optional argument that specifies a RawDataBuffer containing vendor-defined byte data to be decoded by the OS driver. The bit field name `_VEN` is automatically created to refer to this portion of the resource descriptor.
- *PinList* is a list of pin numbers on the ResourceSource that are described by this descriptor. The bit field name `_PIN` is automatically created to refer to this portion of the resource descriptor.

Table 19.37: Pin Configuration Types and Values

Pin Type	Configuration	Pin Value	Description
0x00 = Default		N/A	Default configuration. No configuration is applied.
0x01 = Bias Pull-Up		Pull up resistance, in Ohms.	This means the pin is pulled up with a certain number of Ohms to an implicitly supplied VDD rail.
0x02 = Bias Pull-down		Pull down resistance, in Ohms.	This means the pin is pulled down with a certain number of Ohms, toward the GND rail.
0x03 = Bias Default		N/A	If the silicon has a default biasing mode, reset the pin to this mode.
0x04 = Bias Disable		N/A	Any software-selectable bias settings on the pin will be disabled.
0x05 = Bias High Impedance		N/A	This means that the pin is configured into a high impedance mode and essentially shut off from the outside world. It will not influence the signal state if a rail is connected to the pin, hence a good default mode.
0x06 = Bias Bus Hold		N/A	This will make the pin in a weak latch state where it weakly drives the last value on a tristate bus.
0x07 = Drive Open Drain		N/A	This will configure the pin into open drain (open collector) state.
0x08 = Drive Open Source		N/A	This will configure the pin into open source (open emitter) state.
0x09 = Drive Push Pull		N/A	This will configure the pin into explicit push-pull state. This is useful if the power-on default state is e.g. open drain or high impedance state.
0x0A = Drive Strength		Drive strength in milliamperes	This will set the output driver of the pin to supply a certain number of milliamperes, usually by activating several driver stages.
0x0B = Slew Rate		Custom format	This controls the slew rate of the pin, affecting speed but also sharpness of edges and thus noisiness on the board. The hardware-specific argument tells what slew rate to configure
0x0C = Input Debounce		Debounce time in microseconds.	This will enable debouncing (for e.g. key inputs) of the pin signal.
0x0D = Input Schmitt Trigger		Enabled = 1, Disabled = 0	This will enable Schmitt trigger support for the line.
0x0E - 0x7F = Reserved		Reserved	Reserved
0x80 - 0xFF = Vendor defined values		Custom base	From this point, vendor and Hardware-specific configurations are listed.

Description

The PinConfig macro evaluates to a buffer that contains a Pin Configuration resource descriptor, as described in [Section 6.4.3.10](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

Note: There is some overlap between the properties set by GpioIo/GpioInt/ PinFunction and PinConfig descriptors. For example, both are setting properties such as pull-ups. If the same property is specified by multiple descriptors for the same pins, the order in which these properties are applied is undetermined. To avoid any conflicts, GpioInt/GpioIo/PinFunction should provide a default value for these properties when PinConfig is used. If PinConfig is used to set pin bias, PullDefault should be used for GpioIo/GpioInt/ PinFunction. If PinConfig is used to set debounce timeout, 0 should be used for GpioIo/GpioInt. If PinConfig is used to set drive strength, 0 should be used for GpioIo.*

Example:

```

//
// Description: GPIO
//
Device (GPIO)
{
    Name (_HID, "PNPFFFE")
    Name (_UID, 0x0)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4FE00000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x54}
        })
        Return(RBUF)
    }
}

//
// Description: I2C controller 1
//
Device (I2C1)
{
    Name (_HID, "PNPFFFF")
    Name (_UID, 0x0)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4F800000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x55}
            PinFunction(Exclusive, PullDefault, 0x5, "\\_SB.GPIO", 0, ResourceConsumer, )
↪{2, 3}
            // Configure 10k Pull up for I2C SDA/SCL pins
            PinConfig(Exclusive, 0x01, 10000, "\\_SB.GPIO", 0, ResourceConsumer, ) {2, 3}
        })
        Return(RBUF)
    }
}

//
// Description: Physical display panel
//
Device (SDIO)
{

```

(continues on next page)

(continued from previous page)

```

Name (_HID, "PNPFFFD")
Name (_UID, 0x0)
Method (_STA)
{
    Return(0xf)
}
Method (_CRS, 0x0, NotSerialized)
{
    Name (RBUF, ResourceTemplate()
    {
        Memory32Fixed(ReadWrite, 0x4F900000, 0x20)
        Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x57}
        GpioIo(Shared, PullDefault, 0, 0, IoRestrictionNone, "\\_SB.GPIO",) {2, 3}
        // Configure 20k Pull down
        PinConfig(Exclusive, 0x02, 20000, "\\_SB.GPIO", 0, ResourceConsumer, ) {2, 3}
        // Enable Schmitt-trigger
        PinConfig(Exclusive, 0x0D, 1, "\\_SB.GPIO", 0, ResourceConsumer, ) {2, 3}
        // Set slew rate to custom value 3
        PinConfig(Exclusive, 0x0B, 3, "\\_SB.GPIO", 0, ResourceConsumer, ) {2, 3}
    })
    Return(RBUF)
}
}

```

19.6.104 PinFunction (Pin Function Descriptor Macro)

Syntax:

Macro:

```
PinFunction (Shared/Exclusive, PinPullConfiguration, FunctionNumber, ResourceSource,
ResourceSourceIndex, ResourceUsage, DescriptorName, VendorData) {Pin List}
```

Arguments

- *Shared* is an optional argument and can be one of Shared, Exclusive. If not specified, Exclusive is assumed. The bit field name *_SHR* is automatically created to refer to this portion of the resource descriptor.
- *PinPullConfiguration* can be one of PullDefault, PullUp, PullDown, PullNone or a vendor-supplied value in the range 128-255.
- *FunctionNumber* is a provider-specific integer that designates which function is being described.
- *ResourceSource* is a string which uniquely identifies the GPIO controller referred to by this descriptor. ResourceSource can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.
- *ResourceSourceIndex* is an optional argument and is assumed to be 0 for this revision.
- *ResourceUsage* is an optional argument and is assumed to be ResourceConsumer for this revision.
- *DescriptorName* is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

- *VendorData* is an optional argument that specifies a *RawDataBuffer* containing vendor-defined byte data to be decoded by the OS driver. The bit field name *_VEN* is automatically created to refer to this portion of the resource descriptor.
- *PinList* is a non-empty list of (zero-based) pin numbers on the *ResourceSource* that are described by this descriptor. The bit field name *_PIN* is automatically created to refer to this portion of the resource descriptor.

Description

The *PinFunction* macro evaluates to a buffer that contains a Pin Function resource descriptor, as described in this section. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

Note: PinFunction macro allows for maximum flexibility to define the desired function of each pin individually. It is the responsibility of the firmware writer to take into account any platform-level restrictions where pin function must be applied at a coarser granularity. Thus, if the platform design requires the functions for a set of pins to be configured as group, the firmware writer must ensure this is done in the corresponding PinFunction description by specifying all relevant pins in a single PinFunction. In the multi-pin scenario, the OSPM must honor the PinFunction requirements for all of the specified pins on an “all-or-nothing” basis.

*Note: The Pin Function descriptor is intended for scenarios where non-GPIO functions are desired. For GPIO-based functionalities, the firmware should always specify the appropriate *GpioIo* or *GpioInt* descriptor.*

Example:

```
//
// Description: GPIO
//

Device (GPIO)
{
    Name (_HID, "PNPFFFE")
    Name (_UID, 0x0)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4FE00000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x54}
        })
        Return(RBUF)
    }
}

//
// Description: I2C controller 1
//

Device (I2C1)
{
    Name (_HID, "PNPFFFF")
    Name (_UID, 0x0)
    Method (_STA)
    {
```

(continues on next page)

(continued from previous page)

```

        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4F800000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x55}
            PinFunction(Exclusive, PullUp, 0x5, "\\_SB.GPI0", 0, ResourceConsumer,) {2, 3}
        })
        Return(RBUF)
    }
}

//
// Description: I2C controller 2
//

Device (I2C2)
{
    Name (_HID, "PNPFFFF")
    Name (_UID, 0x1)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4F900000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x56}
            PinFunction(Exclusive, PullUp, 0x0, 0x4, "\\_SB.GPI0", 0, ResourceConsumer, )
→ {2, 3}
        })
        Return(RBUF)
    }
}

//
// Description: Physical display panel
//

Device (DISP)
{
    Name (_HID, "PNPFFFD")
    Name (_UID, 0x0)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)

```

(continues on next page)

(continued from previous page)

```

{
    Name (RBUF, ResourceTemplate()
    {
        Memory32Fixed(ReadWrite, 0x4F900000, 0x20)
        Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x57}
        GpioIo(Shared, PullDefault, 0, 0, IoRestrictionNone, "\\_SB.GPI0",) {2, 3}
    })
    Return(RBUF)
}
}

```

19.6.105 PinGroup (Pin Group Descriptor Macro)

Syntax:

Macro:

```
PinGroup (ResourceLabel, ResourceUsage, DescriptorName, VendorData) {Pin List }
```

Arguments

- *ResourceUsage* is an optional argument and is assumed to be ResourceProducer for this revision.
- *ResourceLabel* is an arbitrary, non-empty string that uniquely identifies this particular PinGroup resource from others within a resource template buffer. This label is used by resource consumers to refer to this resource.
- *DescriptorName* is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.
- *VendorData* is an optional argument that specifies a RawDataBuffer containing vendor-defined byte data to be decoded by the OS driver. The bit field name `_VEN` is automatically created to refer to this portion of the resource descriptor.
- *PinList* is a non-empty list of (zero-based) pin numbers on the ResourceSource that are described by this descriptor. The bit field name `_PIN` is automatically created to refer to this portion of the resource descriptor.

Description

The PinGroup macro evaluates to a buffer that contains a Pin Group Configuration resource descriptor. The format of the Pin Group Configuration resource descriptor can be found in [Section 6.4.3.13](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

PinGroup resource descriptors must be declared within the scope of the pin controller device to which the pins belong.

19.6.106 PinGroupConfig (Pin Group Configuration Descriptor Macro)

Syntax:

Macro:

```
PinGroupConfig (Shared/Exclusive, PinConfigType, PinConfigValue, ResourceSource,
ResourceSourceIndex, ResourceSourceLabel, ResourceUsage, DescriptorName, VendorData)
```

Arguments:

- *Shared* is an optional argument and can be either Shared or Exclusive. If not specified, Exclusive is assumed. The bit field name `_SHR` is automatically created to refer to this portion of the resource descriptor.
- *PinConfigType* can be one of the configuration types described below in *Pin Group Configuration Types and Values*. The bit field name `_TYP` is automatically created to refer to this portion of the resource descriptor.
- *PinConfigValue* is one of the configurations values described below in *Pin Group Configuration Types and Values*. The bit field name `_VAL` is automatically created to refer to this portion of the resource descriptor.
- *ResourceSource* is a string that uniquely identifies the GPIO controller which includes the PinGroup resource referenced by this descriptor. ResourceSource can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.
- *ResourceSourceLabel* is a non-empty string argument that matches ResourceLabel of the PinGroup resource in the current resource template buffer of the GPIO controller referenced in ResourceSource.
- *DescriptorName* is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.
- *ResourceSourceIndex* is an optional argument and is assumed to be 0 for this revision.
- *ResourceUsage* is an optional argument and is assumed to be ResourceConsumer for this revision.
- *VendorData* is an optional argument that specifies a RawDataBuffer containing vendor-defined byte data to be decoded by the OS driver. The bit field name `_VEN` is automatically created to refer to this portion of the resource descriptor.

Table 19.38: Pin Group Configuration Types and Values

Pin Type	Configuration	Pin Value	Description
0x00 = Default		N/A	Default configuration. No configuration is applied).
0x01 = Bias Pull-Up		Pull up resistance, in Ohms.	This means the pin is pulled up with a certain number of Ohms to an implicitly supplied VDD rail.
0x02 = Bias Pull-down		Pull down resistance, in Ohms.	This means the pin is pulled down with a certain number of Ohms, toward the GND rail.
0x03 = Bias Default		N/A	If the silicon has a default biasing mode, reset the pin to this mode.
0x04 = Bias Disable		N/A	Any software-selectable bias settings on the pin will be disabled.
0x05 = Bias High Impedance		N/A	This means that the pin is configured into a high impedance mode and essentially shut off from the outside world. It will not influence the signal state if a rail is connected to the pin, hence a good default mode.
0x06 = Bias Bus Hold		N/A	This will make the pin in a weak latch state where it weakly drives the last value on a tristate bus.
0x07 = Drive Open Drain		N/A	This will configure the pin into open drain (open collector) state.
0x08 = Drive Open Source		N/A	This will configure the pin into open source (open emitter) state.
0x09 = Drive Push Pull		N/A	This will configure the pin into explicit push-pull state. This is useful if the power-on default state is e.g. open drain or high impedance state.

continues on next page

Table 19.38 – continued from previous page

Pin Type	Configuration	Pin Value	Description
0x0A = Drive Strength		Drive strength in milliamperes	This will set the output driver of the pin to supply a certain number of milliamperes, usually by activating several driver stages.
0x0B = Slew Rate		Custom format	This controls the slew rate of the pin, affecting speed but also sharpness of edges and thus noisiness on the board. The hardware-specific argument tells what slew rate to configure
0x0C = Input Debounce		Debounce time in microseconds.	This will enable debouncing (for e.g. key inputs) of the pin signal.
0x0D = Input Schmitt Trigger		Enabled = 1, Disabled = 0	This will enable Schmitt trigger support for the line.
0x0E - 0x7F = Reserved		Reserved	Reserved
0x80 - 0xFF = Vendor defined values		Custom base	From this point, vendor and Hardware-specific configurations are listed.

Description

The PinGroupConfig macro evaluates to a buffer that contains a Pin Group Configuration resource descriptor. The format of the Pin Group Configuration resource descriptor can be found in [Section 6.4.3.13](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

Example:

```
//
// Description: GPIO
//

Device (GPIO)
{
    Name (_HID, "PNPFFFE")
    Name (_UID, 0x0)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4FE00000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x54}
            PinGroup("group1", ResourceProducer) {2, 3}
        })
        Return(RBUF)
    }
}

//
// Description: I2C controller 1
//
```

(continues on next page)

(continued from previous page)

```

Device (I2C1)
{
    Name (_HID, "PNPFFFF")
    Name (_UID, 0x0)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4F800000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x55}
            // Set function I2C1 for SDA/SCL pins
            PinGroupFunction(Exclusive, 0x5, "\\_SB.GPI0 ", 0, "group1", ResourceConsumer, )
            // Configure 10k Pull up for SDA/SCL pins
            PinGroupConfig(Exclusive, 0x01, 10000, "\\_SB.GPI0 ", 0, "group1",
↪ResourceConsumer, )
        })
        Return(RBUF)
    }
}

//
// Description: I2C controller 2
//

Device (I2C2)
{
    Name (_HID, "PNPFFFF")
    Name (_UID, 0x1)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4F900000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x56}
            // Set function I2C2 for SDA/SCL pins
            PinGroupFunction(Exclusive, 0x4, "\\_SB.GPI0 ", 0, "group1", ResourceConsumer, )
            // Configure 10k Pull up for SDA/SCL pins
            PinGroupConfig(Exclusive, 0x01, 10000, "\\_SB.GPI0 ", 0, "group1",
↪ResourceConsumer, )
        })
        Return(RBUF)
    }
}

```

(continues on next page)

(continued from previous page)

```
//
// Description: Physical display panel
//

Device (DISP)
{
    Name (_HID, "PNPFFFD")
    Name (_UID, 0x0)
    Method (_STA)
    {
        Return(0xf)
    }
    Method (_CRS, 0x0, NotSerialized)
    {
        Name (RBUF, ResourceTemplate()
        {
            Memory32Fixed(ReadWrite, 0x4F900000, 0x20)
            Interrupt(ResourceConsumer, Level, ActiveHigh, Shared) {0x57}
            // Set function GPIO for pin group group1
            PinGroupFunction(Exclusive, 0x1, "\\_SB.GPIO ", 0, "group1", ResourceConsumer, )
            // Configure 20k Pull down
            PinGroupConfig (Exclusive, 0x02, 20000, "\\_SB.GPIO ", 0, "group1",
↪ResourceConsumer, )
            //Enable Schmitt-trigger
            PinGroupConfig (Exclusive, 0x0D, 1, "\\_SB.GPIO ", 0, "group1",
↪ResourceConsumer, )
            //Set slew rate to custom value 3
            PinGroupConfig (Exclusive, 0x0B, 3, "\\_SB.GPIO ", 0, "group1",
↪ResourceConsumer, )
        })
        Return(RBUF)}
    }
}
```

19.6.107 PinGroupFunction (Pin Group Function Configuration Descriptor Macro)

Syntax:

Macro:

```
PinGroupFunction (Shared/Exclusive, FunctionNumber, ResourceSource, ResourceSourceIndex,
ResourceSourceLabel, ResourceUsage, DescriptorName, VendorData)
```

Arguments

- *Shared* is an optional argument and can be one of Shared, Exclusive. If not specified, Exclusive is assumed. The bit field name `_SHR` is automatically created to refer to this portion of the resource descriptor.
- *FunctionNumber* is a provider-specific integer which designates which function is being described. The bit field name `_FUN` is automatically created to refer to this portion of the resource descriptor.
- *ResourceSource* is a string that uniquely identifies the GPIO controller which includes the PinGroup resource referenced by this descriptor. ResourceSource can be a fully-qualified name, a relative name or a name segment

that utilizes the namespace search rules.

- *ResourceSourceLabel* is a non-empty string argument that matches *ResourceLabel* of a *PinGroup* resource in the current resource template buffer of the GPIO controller referenced in *ResourceSource*.
- *DescriptorName* is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.
- *ResourceSourceIndex* is an optional argument and is assumed to be 0 for this revision.
- *ResourceUsage* is an optional argument and is assumed to be *ResourceConsumer* for this revision.
- *VendorData* is an optional argument that specifies a *RawDataBuffer* containing vendor-defined byte data to be decoded by the OS driver. The bit field name *_VEN* is automatically created to refer to this portion of the resource descriptor.

Description

The *PinGroupFunction* macro evaluates to a buffer that contains a Pin Function resource descriptor. The format of the Pin Function resource descriptor can be found in *Pin Function Descriptor*. This macro is designed to be used inside of a *ResourceTemplate* (*Resource To Buffer Conversion Macro*).

19.6.108 PowerResource (Declare Power Resource)

Syntax:

```
PowerResource (ResourceName, SystemLevel, ResourceOrder) {TermList}
```

Arguments

Declares a power resource named *ResourceName*. *PowerResource* opens a name scope.

Description

For a definition of the *PowerResource* term, see [Section 7.2](#)

The power management object list is encoded as *TermList*, so that rather than describing a static power management object list, it is possible to describe a dynamic power management object list according to the system settings. See “*Definition Block Loading*”:

19.6.109 Printf (Create and Store formatted string)

Syntax:

```
Printf (FormatString, FormatArgs) => String
```

Arguments

Printf is a macro that converts the evaluated *FormatString* into a series of string *Concatenate* operations, storing the result in the Debug object.

FormatString is a string literal which may contain one or more uses of the format specifier, *%o*, to indicate locations in the string where an object may be inserted. *%o* is the only format specifier supported since the resulting object is a string and type conversion is handled automatically by *Concatenate*.

FormatArgs is a comma separated list of Named Objects, Locals, or Args that can be evaluated to a string. Each argument is added to the *FormatString* using the *Concatenate* operation at the location specified by *%o* in order of appearance.

Description

The *Printf* macro converts a format string into a series of cascading string *Concatenate* operations, and stores the result in the Debug object

Example

The following ASL example uses *Printf* to write a formatted string with the values of Arg0, Arg1, Arg2, and Arg3 to the Debug Object:

```
Printf ("%o: Unexpected value for %o, %o at line %o", Arg0, Arg1, Arg2, Arg3)
```

This *Printf* macro expression evaluates to the following ASL operation:

```
Store (Concatenate (Concatenate (Concatenate (Concatenate
    (Concatenate (Concatenate (Concatenate ("", Arg0),
        ": Unexpected value for "), Arg1), ", "), Arg2),
    " at line "), Arg3), Debug)
```

19.6.110 QWordIO (QWord IO Resource Descriptor Macro)**Syntax:**

```
QWordIO ( ResourceUsage, IsMinFixed, IsMaxFixed, Decode, ISARanges, AddressGranularity, ↵
↵AddressMinimum, AddressMaximum,
AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource, DescriptorName, ↵
↵TranslationType, TranslationDensity)
```

Arguments

ResourceUsage specifies whether the I/O range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

IsMinFixed specifies whether the minimum address of this I/O range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName. _MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this I/O range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName. _MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

Decode specifies whether or not the device decodes the I/O range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName. _DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

ISARanges specifies whether the I/O ranges specifies are limited to valid ISA I/O ranges (ISAOnly), valid non-ISA I/O ranges (NonISAOnly) or encompass the whole range without limitation (EntireRange). The 2-bit field DescriptorName. _RNG is automatically created to refer to this portion of the resource descriptor, where '1' is NonISAOnly, '2' is ISAOnly and '0' is EntireRange.

AddressGranularity evaluates to a 64-bit integer that specifies the power-of-two boundary (- 1) on which the I/O range must be aligned. The 64-bit field DescriptorName. _GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 64-bit integer that specifies the lowest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName. _MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 64-bit integer that specifies the highest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in *AddressGranularity* is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field *DescriptorName._MAX* is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 64-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 64-bit field *DescriptorName._TRA* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 64-bit integer that specifies the total number of bytes decoded in the I/O range. The 64-bit field *DescriptorName._LEN* is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by *ResourceSource*. If this argument is specified, the *ResourceSource* argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the *ResourceSourceIndex* argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

TranslationType is an optional argument that specifies whether the resource type on the secondary side of the bus is different (*TypeTranslation*) from that on the primary side of the bus or the same (*TypeStatic*). If *TypeTranslation* is specified, then the primary side of the bus is Memory. If *TypeStatic* is specified, then the primary side of the bus is I/O. If nothing is specified, then *TypeStatic* is assumed. The 1-bit field *DescriptorName._TTP* is automatically created to refer to this portion of the resource descriptor, where '1' is *TypeTranslation* and '0' is *TypeStatic*. See *_TTP* for more information.

TranslationDensity is an optional argument that specifies whether or not the translation from the primary to secondary bus is sparse (*SparseTranslation*) or dense (*DenseTranslation*). It is only used when *TranslationType* is *TypeTranslation*. If nothing is specified, then *DenseTranslation* is assumed. The 1-bit field *DescriptorName._TRS* is automatically created to refer to this portion of the resource descriptor, where '1' is *SparseTranslation* and '0' is *DenseTranslation*. See *_TRS* for more information.

Description

The *QWordIO* macro evaluates to a buffer that contains a 64-bit I/O resource descriptor, which describes a range of I/O addresses. The format of the 64-bit I/O resource descriptor can be found in [Section 6.4.3.5.1](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.111 QWordMemory (QWord Memory Resource Descriptor Macro)

Syntax:

```
QWordMemory (ResourceUsage, Decode, IsMinFixed, IsMaxFixed, Cacheable, ReadAndWrite,
↪AddressGranularity, AddressMinimum,
AddressMaximum, AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource,
↪DescriptorName, MemoryRangeType, TranslationType)
```

Arguments

ResourceUsage specifies whether the Memory range is consumed by this device (*ResourceConsumer*) or passed on to child devices (*ResourceProducer*). If nothing is specified, then *ResourceConsumer* is assumed.

Decode specifies whether or not the device decodes the Memory range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName. _DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

IsMinFixed specifies whether the minimum address of this Memory range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName. _MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this Memory range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName. _MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

Cacheable specifies whether or not the memory region is cacheable (Cacheable), cacheable and write-combining (WriteCombining), cacheable and prefetchable (Prefetchable) or uncachable (NonCacheable). If nothing is specified, then NonCacheable is assumed. The 2-bit field DescriptorName. _MEM is automatically created to refer to this portion of the resource descriptor, where '1' is Cacheable, '2' is WriteCombining, '3' is Prefetchable and '0' is NonCacheable.

ReadAndWrite specifies whether or not the memory region is read-only (ReadOnly) or read/write (ReadWrite). If nothing is specified, then ReadWrite is assumed. The 1-bit field DescriptorName. _RW is automatically created to refer to this portion of the resource descriptor, where '1' is ReadWrite and '0' is ReadOnly.

AddressGranularity evaluates to a 64-bit integer that specifies the power-of-two boundary (- 1) on which the Memory range must be aligned. The 64-bit field DescriptorName. _GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 64-bit integer that specifies the lowest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName. _MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 64-bit integer that specifies the highest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName. _MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 64-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 64-bit field DescriptorName. _TRA is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 64-bit integer that specifies the total number of bytes decoded in the Memory range. The 64-bit field DescriptorName. _LEN is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by ResourceSource. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this Memory range is allocated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

MemoryRangeType is an optional argument that specifies the memory usage. The memory can be marked as normal (AddressRangeMemory), used as ACPI NVS space (AddressRangeNVS), used as ACPI reclaimable space (AddressRangeACPI) or as system reserved (AddressRangeReserved). If nothing is specified, then AddressRangeMemory is

assumed. The 2-bit field `DescriptorName`. `_MTP` is automatically created in order to refer to this portion of the resource descriptor, where '0' is `AddressRangeMemory`, '1' is `AddressRangeReserved`, '2' is `AddressRangeACPI` and '3' is `AddressRangeNVS`.

TranslationType is an optional argument that specifies whether the resource type on the secondary side of the bus is different (`TypeTranslation`) from that on the primary side of the bus or the same (`TypeStatic`). If `TypeTranslation` is specified, then the primary side of the bus is I/O. If `TypeStatic` is specified, then the primary side of the bus is Memory. If nothing is specified, then `TypeStatic` is assumed. The 1-bit field `DescriptorName`. `_TTP` is automatically created to refer to this portion of the resource descriptor, where '1' is `TypeTranslation` and '0' is `TypeStatic`. See *_TTP* for more information.

Description

The `QWordMemory` macro evaluates to a buffer that contains a 64-bit memory resource descriptor, which describes a range of memory addresses. The format of the 64-bit memory resource descriptor can be found in [Table 6.45](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.112 QWordPCC (QWordPCC Resource Descriptor Macro)

Syntax:

```
QWordPCC (PccChannel, ResourceSourceIndex, ResourceSource,
DescriptorName)
```

Arguments

PccChannel evaluates to an 8-bit integer that specifies the PCCT Index of the PCC Subspace consumed by this Resource.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by *ResourceSource*. If this argument is specified, the *ResourceSource* argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the *ResourceSourceIndex* argument is not specified, a value of zero is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The `QWordPCC` macro evaluates to a buffer that contains a 64-bit Address Space resource descriptor. The format of the 64-bit Address Space resource descriptor can be found in [Table 6.45](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*. The *PccChannel* field is used to populate the Address field in the resultant Resource Buffer.

19.6.113 QWordSpace (QWord Space Resource Descriptor Macro)

Syntax:

```
QWordSpace (ResourceType, ResourceUsage, Decode, IsMinFixed, IsMaxFixed,
↪TypeSpecificFlags, AddressGranularity, AddressMinimum,
AddressMaximum, AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource,
↪DescriptorName)*
```

Arguments

ResourceType evaluates to an 8-bit integer that specifies the type of this resource. Acceptable values are 0xC0 through 0xFF.

ResourceUsage specifies whether the Memory range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

Decode specifies whether or not the device decodes the Memory range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName. _DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

IsMinFixed specifies whether the minimum address of this Memory range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName. _MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this Memory range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName. _MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

TypeSpecificFlags evaluates to an 8-bit integer. The flags are specific to the ResourceType.

AddressGranularity evaluates to a 64-bit integer that specifies the power-of-two boundary (- 1) on which the Memory range must be aligned. The 64-bit field DescriptorName. _GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 64-bit integer that specifies the lowest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName. _MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 64-bit integer that specifies the highest possible base address of the Memory range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 64-bit field DescriptorName. _MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 64-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 64-bit field DescriptorName. _TRA is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 64-bit integer that specifies the total number of bytes decoded in the Memory range. The 64-bit field DescriptorName. _LEN is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by ResourceSource. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this Memory range is allocated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The QWordSpace macro evaluates to a buffer which contains a 64-bit Address Space resource descriptor, which describes a range of addresses. The format of the 64-bit AddressSpace descriptor can be found in [Table 6.45](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.114 RawDataBuffer (Raw Data Buffer)

Syntax:

```
RawDataBuffer (RDBufferSize) {ByteList} => RawDataBuffer
```

Arguments

Declares a RawDataBuffer of size RDBufferSize and optional initial value of ByteList.

Description

The optional RDBufferSize parameter specifies the size of the buffer and must be a word constant. The initial value is specified in Initializer ByteList. If RDBufferSize is not specified, it defaults to the size of initializer. If the count is too small to hold the value specified by initializer, the initializer size is used.

Note that a RawDataBuffer is not encoded as a Buffer (Opcode, Package length bytes, etc), but rather contains only the raw bytes specified.

19.6.115 RefOf (Create Object Reference)

Syntax:

```
RefOf (Object) => ObjectReference
```

Arguments

Object can be any object type (for example, a package, a device object, and so on).

Description

Returns an object reference to Object. If the Object does not exist, the result of a RefOf operation is fatal. Use the CondRefOf term in cases where the Object might not exist.

The primary purpose of RefOf() is to allow an object to be passed to a method as an argument to the method without the object being evaluated at the time the method was loaded.

19.6.116 Register (Generic Register Resource Descriptor Macro)

Syntax:

```
Register (AddressSpaceKeyword, RegisterBitWidth, RegisterBitOffset, RegisterAddress, ↵
↵AccessSize, DescriptorName)
```

Arguments

AddressSpaceKeyword specifies the address space where the register exists. The register can be one of the following:

- I/O space (SystemIO)
- System Memory (SystemMemory)
- PCI configuration space (PCI_Config)
- Embedded controller space (EmbeddedControl)
- SMBus (SMBus)
- CMOS (SystemCMOS)
- PCI Bar target (PciBarTarget)
- IPMI (IPMI)
- General purpose I/O (GeneralPurposeIO)
- Generic serial bus (GenericSerialBus)
- Platform Communications Channel (PCC)
- Fixed-feature hardware (FFixedHW)

The 8-bit field *DescriptorName*. *_ASI* is automatically created in order to refer to this portion of the resource descriptor. See the Address Space ID definition in *Table: Generic Register Descriptor Definition* for more information, including a list of valid values and their meanings.

RegisterBitWidth evaluates to an 8-bit integer that specifies the number of bits in the register. The 8-bit field *DescriptorName*. *_RBW* is automatically created in order to refer to this portion of the resource descriptor. See the *_RBW* definition in *Table: Generic Register Descriptor Definition* for more information.

RegisterBitOffset evaluates to an 8-bit integer that specifies the offset in bits from the start of the register indicated by *RegisterAddress*. The 8-bit field *DescriptorName*. *_RBO* is automatically created in order to refer to this portion of the resource descriptor. See the *_RBO* definition in *Table: Generic Register Descriptor Definition* for more information.

RegisterAddress evaluates to a 64-bit integer that specifies the register address. The 64-bit field *DescriptorName*. *_ADR* is automatically created in order to refer to this portion of the resource descriptor. See the *_ADR* definition in *Table: Generic Register Descriptor Definition* for more information.

AccessSize evaluates to an 8-bit integer that specifies the size of data values used when accessing the address space as follows:

- 0 - Undefined (legacy)
- 1 - Byte access
- 2 - Word access
- 3 - DWord access
- 4 - QWord access

The 8-bit field *DescriptorName*. *_ASZ* is automatically created in order to refer to this portion of the resource descriptor. See the *_ASZ* definition in the *Generic Register Resource Descriptor* for more information. For backwards

compatibility, the `AccessSize` parameter is optional when invoking the `Register` macro. If the `AccessSize` parameter is not supplied then the `AccessSize` field will be set to zero. In this case, OSPM will assume the access size.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the `Buffer Field` operators.

Description

The `Register` macro evaluates to a buffer that contains a generic register resource descriptor. For the format of the buffer see [Section 19.6.116](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.117 Release (Release a Mutex Synchronization Object)

Syntax:

```
Release (SyncObject)
```

Arguments

SyncObject must be a mutex synchronization object.

Description

If the mutex object is owned by the current invocation, ownership for the Mutex is released once. It is fatal to release ownership on a Mutex unless it is currently owned. A Mutex must be totally released before an invocation completes.

19.6.118 Reset (Reset an Event Synchronization Object)

Syntax:

```
Reset (SyncObject)
```

Arguments

SyncObject must be an Event synchronization object.

Description

This operator is used to reset an event synchronization object to a non-signaled state. See also the `Wait` and `Signal` function operator definitions.

19.6.119 ResourceTemplate (Resource To Buffer Conversion Macro)

Syntax:

```
ResourceTemplate() {ResourceMacroList} => Buffer
```

Description

For a full definition of the `ResourceTemplateTerm` macro, see [Section 19.3.3](#).

19.6.120 Return (Return from Method Execution)

Syntax:

```
Return
Return ()
Return (Arg)
```

Arguments

Arg is optional and can be any valid object or reference.

Description

Returns control to the invoking control method, optionally returning a copy of the object named in Arg. If no Arg object is specified, a Return(Zero) is generated by the ASL compiler.

Note

In the absence of an explicit Return () statement, the return value to the caller is undefined.

19.6.121 Revision (Constant Revision Integer)

Syntax:

```
Revision => Integer
```

Description

The Revision operator returns an Integer containing the current revision of the AML interpreter. Writes to this object are not allowed.

19.6.122 Scope (Open Named Scope)

Syntax:

```
Scope (Location) {ObjectList}
```

Arguments

Opens and assigns a base namespace scope to a collection of objects. All object names defined within the scope are created relative to Location. Note that Location does not have to be below the surrounding scope, but can refer to any location within the namespace. The Scope term itself does not create objects, but only locates objects within the namespace; the actual objects are created by other ASL terms.

Description

The object referred to by Location must already exist in the namespace and be one of the following object types that has a namespace scope associated with it:

- A predefined scope such as: (root), _SB, GPE, _PR, _TZ, etc.
- Device
- Processor
- Thermal Zone

- Power Resource

The Scope term alters the current namespace location to the existing Location. This causes the defined objects within TermList to be created relative to this new location in the namespace.

The object list is encoded as TermList, so that rather than describing a static object list, it is possible to describe a dynamic object list according to the system settings. See “*Definition Block Loading*”.

Note

When creating secondary SSDTs, it is often required to use the Scope operator to change the namespace location in order create objects within some part of the namespace that has been defined by the main DSDT. Use the External operator to declare the scope location so that the ASL compiler will not issue an error for an undefined Location.

Examples

The following example ASL code uses the Scope operator and creates several objects:

```
Scope (\PCI0)
{
    Name (X, 3)
    Scope (\)
    {
        Method (RQ) {Return (0)}
    }
    Name (^Y, 4)
}
```

The created objects are placed in the ACPI namespace as shown:

```
\\PCI0.X
\\RQ
\\Y
```

The following example shows the use of External in conjunction with Scope within an SSDT:

```
DefinitionBlock ("ssdt.aml", "SSDT", 2, "X", "Y", 0x00000001)
{
    External (\_SB.PCI0, DeviceObj)
    Scope (\_SB.PCI0)
    {
    }
}
```

19.6.123 ShiftLeft (Integer Shift Left)

Syntax:

```
ShiftLeft (Source, ShiftCount, Result) => Integer
Result = Source << shiftcount => Integer
Result <<= shiftcount => Integer
```

Arguments

Source and *ShiftCount* are evaluated as Integers.

Description

Source is shifted left with the least significant bit zeroed ShiftCount times. The result is optionally stored into Result.

19.6.124 ShiftRight (Integer Shift Right)

Syntax:

```
ShiftRight (Source, ShiftCount, Result) => Integer
Result = Source >> ShiftCount => Integer
Result >>= ShiftCount => Integer
```

Arguments

Source and *ShiftCount* are evaluated as Integers.

Description

Source is shifted right with the most significant bit zeroed ShiftCount times. The result is optionally stored into Result.

19.6.125 Signal (Signal a Synchronization Event)

Syntax:

```
Signal (SyncObject)
```

Arguments

SyncObject must be an Event synchronization object.

Description

The Event object is signaled once, allowing one invocation to acquire the event.

19.6.126 SizeOf (Get Data Object Size)

Syntax:

```
SizeOf (ObjectName) => Integer
```

Arguments

ObjectName must be a buffer, string or package object.

Description

Returns the size of a buffer, string, or package data object.

For a buffer, it returns the size in bytes of the data. For a string, it returns the size in bytes of the string, not counting the trailing NULL. For a package, it returns the number of elements. For an object reference, the size of the referenced object is returned. Other data types cause a fatal run-time error.

19.6.127 Sleep (Milliseconds Sleep)

Syntax:

```
Sleep (MilliSeconds)
```

Arguments

The Sleep term is used to implement long-term timing requirements. Execution is delayed for at least the required number of milliseconds.

Description

The implementation of Sleep is to round the request up to the closest sleep time supported by the OS and relinquish the processor.

19.6.128 SPISerialBusV2 (SPI Serial Bus Connection Resource Descriptor (Version 2) Macro)

Syntax:

```
SPISerialBusV2 (DeviceSelection, DeviceSelectionPolarity, WireMode, DataBitLength, ↵  
↵SlaveMode, ConnectionSpeed,  
ClockPolarity, ClockPhase, ResourceSource, ResourceSourceIndex, ResourceUsage, ↵  
↵DescriptorName, Shared, VendorData)
```

Arguments

DeviceSelection is the device selection value. This value may refer to a chip-select line, GPIO line or other line selection mechanism. *_ADR* is automatically created to refer to this portion of the resource descriptor.

DeviceSelectionPolarity is an optional argument and can be either *PolarityHigh* or *PolarityLow* to indicate that the device is active. *PolarityLow* is the default. The bit field *_DPL* is automatically created to refer to this portion of the resource descriptor.

WireMode is an optional argument and can be either *ThreeWireMode* or *FourWireMode*. *FourWireMode* is the default. The bit field name *_MOD* is automatically created to refer to this portion of the resource descriptor.

DataBitLength is the size, in bits, of the smallest transfer unit for this connection. *_LEN* is automatically created to refer to this portion of the resource descriptor.

SlaveMode is an optional argument and can be either *ControllerInitiated* or *DeviceInitiated*. *ControllerInitiated* is the default. The bit field name *_SLV* is automatically created to refer to this portion of the resource descriptor.

ConnectionSpeed is the maximum connection speed supported by this connection, in hertz. The bit field name *_SPE* is automatically created to refer to this portion of the resource descriptor.

ClockPolarity can be either *ClockPolarityLow* or *ClockPolarityHigh*. *_POL* is automatically created to refer to this portion of the resource descriptor.

ClockPhase can be either *ClockPhaseFirst* or *ClockPhaseSecond*. *_PHA* is automatically created to refer to this portion of the resource descriptor.

ResourceSource is a string which uniquely identifies the SPI bus controller referred to by this descriptor. *ResourceSource* can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.

ResourceSourceIndex is an optional argument and is assumed to be 0 for this revision.

ResourceUsage is an optional argument and is assumed to be *ResourceConsumer* for this revision. *DescriptorName* is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains

the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Shared is an optional argument and can be either *Shared* or *Exclusive*. If not specified, *Exclusive* is assumed. The bit field name `_SHR` is automatically created to refer to this portion of the resource descriptor.

VendorData is an optional argument that specifies an object to be decoded by the OS driver. It is a `RawDataBuffer`. The bit field name `_VEN` is automatically created to refer to this portion of the resource descriptor.

Description

The *SPISerialBusV2* macro evaluates to a buffer that contains an *SPI Serial Bus Connection Resource Descriptor*. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.129 Stall (Stall for a Short Time)

Syntax:

```
Stall (MicroSeconds)
```

Arguments

The Stall term is used to implement short-term timing requirements. Execution is delayed for at least the required number of microseconds.

Description

The implementation of Stall is OS-specific, but must not relinquish control of the processor. Because of this, delays longer than 100 microseconds must use Sleep instead of Stall.

19.6.130 StartDependentFn (Start Dependent Function Resource Descriptor Macro)

Syntax:

```
StartDependentFn (CompatibilityPriority, PerformancePriority) {ResourceList}
```

Arguments

CompatibilityPriority indicates the relative compatibility of the configuration specified by ResourceList relative to the PC/AT. 0 = Good, 1 = Acceptable, 2 = Sub-optimal.

PerformancePriority indicates the relative performance of the configuration specified by ResourceList relative to the other configurations. 0 = Good, 1 = Acceptable, 2 = Sub-optimal.

ResourceList is a list of resources descriptors which must be selected together for this configuration.

Description

The StartDependentFn macro evaluates to a buffer that contains a start dependent function resource descriptor, which describes a group of resources which must be selected together. Each subsequent StartDependentFn or StartDependentFnNoPri resource descriptor introduces a new choice of resources for configuring the device, with the last choice terminated with an EndDependentFn resource descriptor. The format of the start dependent function resource descriptor can be found in Section 6.4.2.3. This macro generates the two-byte form of the resource descriptor, and is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.131 StartDependentFnNoPri (Start Dependent Function Resource Descriptor Macro)

Syntax:

```
StartDependentFnNoPri() {ResourceList}
```

Description

The StartDependentFnNoPri macro evaluates to a buffer that contains a start dependent function resource descriptor, which describes a group of resources which must be selected together. Each subsequent StartDependentFn or StartDependentFnNoPri resource descriptor introduces a new choice of resources for configuring the device, with the last choice terminated with an EndDependentFn resource descriptor. The format of the start dependent function resource descriptor can be found in [Section 6.4.2.3](#). This macro generates the one-byte form of the resource descriptor, and is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

This is similar to *StartDependentFn* with both CompatibilityPriority and PerformancePriority set to 1, but is one byte shorter.

19.6.132 Store (Store an Object)

Syntax:

```
Store (Source, Destination) => DataRefObject Destination = Source => DataRefObject
```

Arguments

This operation evaluates Source, converts it to the data type of Destination, and writes the result into Destination. For information on automatic data-type conversion, see [Section 19.3.5](#)

Description

Stores to OperationRegion Field data types may relinquish the processor depending on the address space.

All stores (of any type) to the constant Zero, constant One, or constant Ones object are not allowed. Stores to read-only objects are fatal. The execution result of the operation depends on the type of Destination. For any type other than an operation region field, the execution result is the same as the data written to Destination. For operation region fields with an AccessType of ByteAcc, WordAcc, DWordAcc, QWordAcc or AnyAcc, the execution result is the same as the data written to Destination as in the normal case, but when the AccessType is BufferAcc, the operation region handler may modify the data when it is written to the Destination so that the execution result contains modified data.

Example

The following example creates the name CNT that references an integer data object with the value 5 and then stores CNT to Local0. After the Store operation, Local0 is an integer object with the value 5:

```
Name (CNT, 5)
Store (CNT, Local0)
```

19.6.133 Subtract (Integer Subtract)

Syntax:

```
Subtract ( Minuend, Subtrahend, Result ) => Integer
Result = Minuend - Subtrahend => Integer
Result -= Subtrahend => Integer
```

Arguments

Minuend and Subtrahend are evaluated as Integers.

Description

Subtrahend is subtracted from Minuend, and the result is optionally stored into Result. Underflow conditions are ignored and the result simply loses the most significant bits.

19.6.134 Switch (Select Code To Execute Based On Expression)

Syntax:

```
Switch (Expression) {CaseTermList}
```

Arguments

Expression is an ASL expression that evaluates to an Integer, String or Buffer.

Description

The Switch, Case and Default statements help simplify the creation of conditional and branching code. The Switch statement transfers control to a statement within the enclosed body of executable ASL code

If the Case Value is an Integer, Buffer or String, then control passes to the statement that matches the value of Switch (Expression). If the Case value is a Package, then control passes if any member of the package matches the Switch (Value) The Switch CaseTermList can include any number of Case instances, but no two Case Values (or members of a Value, if Value is a Package) within the same Switch statement can have the same value.

Execution of the statement body begins at the selected TermList and proceeds until the TermList end of body or until a Break or Continue statement transfers control out of the body.

The Default statement is executed if no Case Value matches the value of Switch (expression). If the Default statement is omitted, and no Case match is found, none of the statements in the Switch body are executed. There can be at most one Default statement. The Default statement can appear anywhere in the body of the Switch statement.

A Case or Default term can only appear inside a Switch statement. Switch statements can be nested. (Compatibility Note) The Switch, Case, and Default terms were first introduced in ACPI 2.0. However, their implementation is backward compatible with ACPI 1.0 AML interpreters.

Example

Use of the Switch statement usually looks something like this:

```
Switch (expression)
{
    Case (value) {
        Statements executed if Lequal (expression, value)
    }
    Case (Package () {value, value, value}) {
        Statements executed if Lequal (expression, any value in package)
```

(continues on next page)

(continued from previous page)

```

}
Default {
    Statements executed if expression does not equal any case constant-expression
}
}

```

Note

Compiler Note: The following example demonstrates how the Switch statement should be translated into ACPI 1.0-compatible AML:

```

Switch (Add (ABCD ( ),1)
{
    Case (1) {
        ...statements1...
    }
    Case (Package ( ) {4,5,6}) {
        ...statements2...
    }
    Default {
        ...statements3...
    }
}

```

is translated as:

```

Name (_T_I, 0) // Create Integer temporary variable for result
While (One)
{
    Store (Add (ABCD ( ), 1), \_T_I)
    If (LEqual (_T_I, 1)) {
        ...statements1...
    }
    Else {
        If (LNotEqual (Match (Package ( ) {4, 5, 6}, MEQ, \_T_I, MTR, 0, 0), Ones)) {
            ...statements2...
        }
        Else {
            ...statements3...
        }
    }
    Break
}

```

The While (One) is emitted to enable the use of Break and Continue within the Switch statement. Temporary names emitted by the ASL compiler should appear at the top level of the method, since the Switch statement could appear within a loop and thus attempt to create the name more than once.

Note: If the ASL compiler is unable to determine the type of the expression, then it will generate a warning and assume a type of Integer. The warning will indicate that the code should use one of the type conversion operators (Such as ToInteger, ToBuffer, ToDecimalString or ToHexString). Caution: Some of these operators are defined starting with ACPI 2.0 and as such may not be supported by ACPI 1.0b compatible interpreters.

For example:

```
Switch (ABCD ()) // Cannot determine the type because methods can return anything.
{
    ...case statements...
}
```

will generate a warning and the following code:

```
Name (_T_I, 0)
Store (ABCD (), \_T_I)
```

To remove the warning, the code should be:

```
Switch (ToInteger (ABCD ()))
{
    ...case statements...
}
```

19.6.135 ThermalZone (Declare Thermal Zone)

Syntax:

```
ThermalZone (ThermalZoneName) {TermList}
```

Arguments

Declares a Thermal Zone object named ThermalZoneName. ThermalZone opens a name scope.

Each use of a ThermalZone term declares one thermal zone in the system. Each thermal zone in a system is required to have a unique ThermalZoneName.

Description

A thermal zone may be declared in the namespace anywhere within the _SB scope. For compatibility with operating systems implementing ACPI 1.0, a thermal zone may also be declared under the _TZ scope. An ACPI-compatible namespace may define Thermal Zone objects in either the _SB or _TZ scope but not both.

For example ASL code that uses a ThermalZone statement, see [Section 11](#)

The thermal object list is encoded as TermList, so that rather than describing a static thermal object list, it is possible to describe a dynamic thermal object list according to the system settings. See “*Definition Block Loading*”.

19.6.136 Timer (Get 64-Bit Timer Value)

Syntax:

```
Timer => Integer
```

Description

The timer opcode returns a monotonically increasing value that can be used by ACPI methods to measure time passing, this enables speed optimization by allowing AML code to mark the passage of time independent of OS ACPI interpreter implementation.

The Sleep opcode can only indicate waiting for longer than the time specified.

The value resulting from this opcode is 64 bits. It is monotonically increasing, but it is not guaranteed that every result will be unique, i.e. two subsequent instructions may return the same value. The only guarantee is that each subsequent evaluation will be greater-than or equal to the previous ones.

The period of this timer is 100 nanoseconds. While the underlying hardware may not support this granularity, the interpreter will do the conversion from the actual timer hardware frequency into 100 nanosecond units.

Users of this opcode should realize that a value returned only represents the time at which the opcode itself executed. There is no guarantee that the next opcode in the instruction stream will execute in any particular time bound.

The OSPM can implement this using the ACPI Timer and keep track of overrun. Other implementations are possible. This provides abstraction away from chipset differences

Note

Compatibility Note New for ACPI 3.0

19.6.137 ToBCD (Convert Integer to BCD)

Syntax:

```
ToBCD (Value, Result) => Integer
```

Arguments

Value is evaluated as an integer

Description

The ToBCD operator is used to convert *Value* from a numeric (Integer) format to a BCD format and optionally store the numeric value into *Result*.

19.6.138 ToBuffer (Convert Data to Buffer)

Syntax:

```
ToBuffer (Data, Result) => Buffer
```

Arguments

Data must be an Integer, String, or Buffer data type.

Description

Data is converted to buffer type and the result is optionally stored into *Result*. If *Data* is an integer, it is converted into *n* bytes of buffer (where *n* is 4 if the definition block has defined integers as 32 bits or 8 if the definition block has defined integers as 64 bits as indicated by the Definition Block table header's Revision field), taking the least significant byte of integer as the first byte of buffer. If *Data* is a buffer, no conversion is performed. If *Data* is a string, each ASCII string character is copied to one buffer byte, including the string null terminator. A null (zero-length) string will be converted to a zero-length buffer.

19.6.139 ToDecimalString (Convert Data to Decimal String)

Syntax:

```
ToDecimalString (Data, Result) => String
```

Arguments

Data must be an Integer, String, or Buffer data type.

Description

Data is converted to a decimal string, and the result is optionally stored into Result. If Data is already a string, no action is performed. If Data is a buffer, it is converted to a string of decimal values separated by commas. (Each byte of the buffer is converted to a single decimal value.) A zero-length buffer will be converted to a null (zero-length) string.

19.6.140 ToHexString (Convert Data to Hexadecimal String)

Syntax:

```
ToHexString (Data, Result) => String
```

Arguments

Data must be an Integer, String, or Buffer data type.

Description

Data is converted to a hexadecimal string, and the result is optionally stored into Result. If Data is already a string, no action is performed. If Data is a buffer, it is converted to a string of hexadecimal values separated by commas. A zero-length buffer will be converted to a null (zero-length) string.

19.6.141 ToInteger (Convert Data to Integer)

Syntax:

```
ToInteger (Data, Result) => Integer
```

Arguments

Data must be an Integer, String, or Buffer data type.

Description

Data is converted to integer type and the result is optionally stored into Result. If Data is a string, it must be either a decimal or hexadecimal numeric string (in other words, prefixed by “0x”) and the value must not exceed the maximum of an integer value. If the value is exceeding the maximum, the result of the conversion is unpredictable. A null (zero-length) string is illegal. If Data is a Buffer, the first 8 bytes of the buffer are converted to an integer, taking the first byte as the least significant byte of the integer. A zero-length buffer is illegal. If Data is an integer, no action is performed.

19.6.142 ToPLD (Creates a _PLD Buffer Object)

Syntax:

```
ToPLD (PLDKeywordList) => \_PLD Buffer Object
```

Arguments

PLDKeywordList is a list of *PLDKeyword* types that describe elements of a Physical Layer Description (_PLD) buffer that can be assigned values. The table below shows the available *PLDKeyword* types and their assignable types. Refer to the _PLD section for a description of the _PLD method object.

Table 19.39: **PLD Keywords and Assignment Types**

PLDKeyword	Assignment Type
PLD_Revision	Integer
PLD_IgnoreColor	Integer
PLD_Red	Integer
PLD_Green	Integer
PLD_Blue	Integer
PLD_Width	Integer
PLD_Height	Integer
PLD_UserVisible	Integer
PLD_Dock	Integer
PLD_Lid	Integer
PLD_Panel	Integer or String
PLD_VerticalPosition	Integer or String
PLD_HorizontalPosition	Integer or String
PLD_Shape	Integer or String
PLD_GroupOrientation	Integer
PLD_GroupToken	Integer
PLD_GroupPosition	Integer
PLD_Bay	Integer
PLD_Ejectable	Integer
PLD_EjectRequired	Integer
PLD_CabinetNumber	Integer
PLD_CardCageNumber	Integer
PLD_Reference	Integer
PLD_Rotation	Integer
PLD_Order	Integer
PLD_VeriticalOffset	Integer
PLD_HorizontalOffset	Integer

A subset of PLDKeyword types can be assigned string values for improved readability. Those types and their assignable values are shown in the table below.

Table 19.40: **PLD Keywords and assignable String Values**

PLDKeyword	Assignable String Values
PLD_Panel	“TOP”, “BOTTOM”, “LEFT”, “RIGHT”, “FRONT”, “BACK”, “UNKNOWN”
PLD_VerticalPosition	“UPPER”, “CENTER”, “LOWER”
PLD_HorizontalPosition	“LEFT”, “CENTER”, “RIGHT”

continues on next page

Table 19.40 – continued from previous page

PLDKeyword	Assignable String Values
PLD_Shape	“ROUND”, “OVAL”, “SQUARE”, “VERTICALRECTANGLE”, “HORIZONTALRECTANGLE”, “VERTICALTRAPEZOID”, “HORIZONTALTRAPEZOID”, “UNKNOWN”

Description

The ToPLD macro converts a list of PLDKeyword types into a _PLD buffer object.

Example

The following ASL shows an example using ToPLD to construct a _PLD buffer/package object:

```
Name (_PLD, Package (0x01) // \_PLD: Physical Location of Device
{
    ToPLD (
        PLD_Revision = 0x2,
        PLD_IgnoreColor = 0x1,
        PLD_Red = 0x37,
        PLD_Green = 0x44,
        PLD_Blue = 0xFF,
        PLD_Width = 0x4,
        PLD_Height = 0x19,
        PLD_UserVisible = 0x1,
        PLD_Dock = 0x0,
        PLD_Lid = 0x1,
        PLD_Panel = "TOP",
        PLD_VerticalPosition = "CENTER",
        PLD_HorizontalPosition = "RIGHT",
        PLD_Shape = "VERTICALRECTANGLE",
        PLD_GroupOrientation = 0x1,
        PLD_GroupToken = 0xA,
        PLD_GroupPosition = 0x21,
        PLD_Bay = 0x1,
        PLD_Ejectable = 0x0,
        PLD_EjectRequired = 0x1,
        PLD_CabinetNumber = 0x1E,
        PLD_CardCageNumber = 0x17,
        PLD_Reference = 0x0,
        PLD_Rotation = 0x7,
        PLD_Order = 0x3,
        PLD_VerticalOffset = 0x141,
        PLD_HorizontalOffset = 0x2C
    )
}
)
```

19.6.143 ToString (Convert Buffer To String)

Syntax:

```
ToString (Source, Length,*Result) => String
```

Arguments

Source is evaluated as a buffer. *Length* is evaluated as an integer data type.

Description

Starting with the first byte, the contents of the buffer are copied into the string until the number of characters specified by *Length* is reached or a null (0) character is found. If *Length* is not specified or is Ones, then the contents of the buffer are copied until a null (0) character is found. If the source buffer has a length of zero, a zero length (null terminator only) string will be created. The result is copied into the *Result*.

19.6.144 ToUUID (Convert String to UUID Macro)

Syntax:

```
ToUUID (AsciiString) => Buffer
```

Arguments

AsciiString is evaluated as a String data type.

Description

This macro will convert an ASCII string to a 128-bit buffer. The string must have the following format:

```
aabbccdd-eeff-gghh-ii jj-kkllmmnnnoopp
```

where aa - pp are one byte hexadecimal numbers, made up of hexadecimal digits. The resulting buffer has the format shown in the following table:

Table 19.41: UUID Buffer Format

String	Offset In Buffer
aa	3
bb	2
cc	1
dd	0
ee	5
ff	4
gg	7
hh	6
ii	8
jj	9
kk	10
ll	11
mm	12
nn	13
oo	14
pp	15

Note**Compatibility Note:** New for ACPI 3.0**19.6.145 UARTSerialBusV2 (UART Serial Bus Connection Resource Descriptor Version 2 Macro)****Syntax:**

```
UARTSerialBusV2 (InitialBaudRate, BitsPerByte, StopBits, LinesInUse, IsBigEndian, Parity,
↪ FlowControl, ReceiveBufferSize,
TransmitBufferSize, ResourceSource, ResourceSourceIndex, ResourceUsage, DescriptorName, ↪
↪ Shared, VendorData)
```

Arguments

InitialBaudRate evaluates to a 32-bit integer that specifies the default or initial connection speed in bytes per second that the device supports. The bit field `_SPE` is automatically created to refer to this portion of the resource descriptor.

BitsPerByte is an optional argument that specifies whether five bits (`DataBitsFive`), six bits (`DataBitsSix`), seven bits (`DataBitsSeven`), eight bits (`DataBitsEight`) or nine bits (`DataBitsNine`) contain data during transfer of a single packet or character. `DataBitsEight` is the default. The bit field `DescriptorName._LEN` is automatically created to refer to this portion of the resource descriptor.

StopBits is an optional argument that specifies whether there are two bits (`StopBitsTwo`), one and a half bits (`StopBitsOnePlusHalf`), one bit (`StopBitsOne`) or no bits (`StopBitsZero`) used to signal the end of a packet or character. `StopBitsOne` is the default. The bit field `_STB` is automatically created to refer to this portion of the resource descriptor.

LinesInUse evaluates to an integer representing 8 1-bit flags representing the presence ('1') or absence ('0') of a particular line. The bit field `_LIN` is automatically created to refer to this portion of the resource descriptor.

Table 19.42: UART Serial Bus Connection Resource Descriptor - Version 2 Macro

Bit Mask	UART Line
Bit 7 (0x80)	Request To Send (RTS)
Bit 6 (0x40)	Clear To Send (CTS)
Bit 5 (0x20)	Data Terminal Ready (DTR)
Bit 4 (0x10)	Data Set Ready (DSR)
Bit 3 (0x08)	Ring Indicator (RI)
Bit 2 (0x04)	Data Carrier Detect (DTD)
Bit 1 (0x02)	Reserved. Must be 0.
Bit 0 (0x01)	Reserved. Must be 0.

IsBigEndian is an optional argument that specifies whether the device is expecting big endian (`BigEndian`) or little endian (`LittleEndian`) data formats. `LittleEndian` is the default. The bit field `_END` is automatically created to refer to this portion of the resource descriptor.

Parity is an optional argument that specifies whether the type of parity bits included after the data in a packet are to be interpreted as space parity (`ParityTypeSpace`), mark parity (`ParityTypeMark`), odd parity (`ParityTypeOdd`), even parity (`ParityTypeEven`) or no parity (`ParityTypeNone`). `ParityTypeNone` is the default. The bit field `PAR` is automatically created to refer to this portion of the resource descriptor.

FlowControl is an optional argument that specifies whether there is hardware-based flow control (*FlowControlHardware*), software-based flow control (*FlowControlXon*) or no flow control (*FlowControlNone*) used when communicating with the device. *FlowControlNone* is the default. The bit field *_FLC* is automatically created to refer to this portion of the resource descriptor.

ReceiveBufferSize evaluates to a 16-bit integer that specifies the upper limit in bytes of the receive buffer that can be optimally utilized while communicating with this device. The bit field *_RXL* is automatically created to refer to this portion of the resource descriptor.

TransmitBufferSize evaluates to a 16-bit integer that specifies the upper limit in bytes of the transmit buffer that can be optimally utilized while communicating with this device. The bit field *_TXL* is automatically created to refer to this portion of the resource descriptor.

ResourceSource is a string which uniquely identifies the UART bus controller referred to by this descriptor. *ResourceSource* can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.

ResourceSourceIndex is an optional argument and is assumed to be 0 for this revision.

ResourceUsage is an optional argument and is assumed to be *ResourceConsumer* for this revision.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Shared is an optional argument and can be either *Shared* or *Exclusive*. If not specified, *Exclusive* is assumed. The bit field name *_SHR* is automatically created to refer to this portion of the resource descriptor.

VendorData is an optional argument that specifies an object to be decoded by the OS driver. It is a *RawDataBuffer*. The bit field name *_VEN* is automatically created to refer to this portion of the resource descriptor.

Description

The *UARTSerialBusV2* macro evaluates to a buffer that contains a *UART Serial Bus Connection Resource Descriptor - Version 2 Macro*. This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.146 Unicode (String To Unicode Conversion Macro)

Syntax:

```
Unicode (String) => Buffer
```

Arguments

This macro will convert a string to a Unicode (UTF-16) string contained in a buffer. The format of the Unicode string is 16 bits per character, with a 16-bit null terminator.

19.6.147 VendorLong (Long Vendor Resource Descriptor)

Syntax:

```
VendorLong (DescriptorName) {VendorByteList}
```

Arguments

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer.

VendorByteList evaluates to a comma-separated list of 8-bit integer constants, where each byte is added verbatim to the body of the VendorLong resource descriptor. A maximum of n bytes can be specified. UUID and UUID specific descriptor subtype are part of the VendorByteList.

Description

The VendorLong macro evaluates to a buffer that contains a vendor-defined resource descriptor. The long form of the vendor-defined resource descriptor can be found in [Section 6.4.3.2](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

This is similar to *VendorShort*, except that the number of allowed bytes in VendorByteList is 65,533 (instead of 7).

19.6.148 VendorShort (Short Vendor Resource Descriptor)

Syntax:

```
VendorShort (DescriptorName) {VendorByteList}
```

Arguments

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer.

Description

The VendorShort macro evaluates to a buffer that contains a vendor-defined resource descriptor. The short form of the vendor-defined resource descriptor can be found in [Section 6.4.2.8](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

This is similar to *VendorLong*, except that the number of allowed bytes in VendorByteList is 7 (instead of 65,533).

19.6.149 Wait (Wait for a Synchronization Event)

Syntax:

```
Wait (SyncObject, TimeoutValue) => Boolean
```

Arguments

SyncObject must be an event synchronization object. *TimeoutValue* is evaluated as an Integer. The calling method blocks while waiting for the event to be signaled.

Description

The pending signal count is decremented. If there is no pending signal count, the processor is relinquished until a signal count is posted to the Event or until at least *TimeoutValue* milliseconds have elapsed.

This operation returns a non-zero value if a timeout occurred and a signal was not acquired. A *TimeoutValue* of 0xFFFF (or greater) indicates that there is no time out and the operation will wait indefinitely.

19.6.150 While (Conditional Loop)

Syntax:

```
While (Predicate) {TermList}
```

Arguments

Predicate is evaluated as an integer.

Description

If the Predicate is non-zero, the list of terms in TermList is executed. The operation repeats until the Predicate evaluates to zero.

Note: Creation of a named object more than once in a given scope is not allowed. As such, unconditionally creating named objects within a While loop must be avoided. A fatal error will be generated on the second iteration of the loop, during the attempt to create the same named object a second time.

19.6.151 WordBusNumber (Word Bus Number Resource Descriptor Macro)

Syntax:

```
WordBusNumber (ResourceUsage, IsMinFixed, IsMaxFixed, Decode, AddressGranularity, ↵  
↵AddressMinimum, AddressMaximum,  
AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource, DescriptorName)
```

Arguments

ResourceUsage specifies whether the bus range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

IsMinFixed specifies whether the minimum address of this bus number range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName. _MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this bus number range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName. _MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

Decode specifies whether or not the device decodes the bus number range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName. _DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

AddressGranularity evaluates to a 16-bit integer that specifies the power-of-two boundary (- 1) on which the bus number range must be aligned. The 16-bit field DescriptorName. _GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 16-bit integer that specifies the lowest possible bus number for the bus number range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 16-bit field DescriptorName. _MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 16-bit integer that specifies the highest possible bus number for the bus number range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 16-bit field DescriptorName. _MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 16-bit integer that specifies the offset to be added to a secondary bus number which results in the corresponding primary bus number. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 16-bit field *DescriptorName._TRA* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 16-bit integer that specifies the total number of bus numbers decoded in the bus number range. The 16-bit field *DescriptorName._LEN* is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by *ResourceSource*. If this argument is specified, the *ResourceSource* argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the *ResourceSourceIndex* argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The *WordBusNumber* macro evaluates to a buffer that contains a 16-bit bus-number resource descriptor. The format of the 16-bit bus number resource descriptor can be found in [Section 6.4.3.5.3](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.152 WordIO (Word IO Resource Descriptor Macro)

Syntax:

```
WordIO (ResourceUsage, IsMinFixed, IsMaxFixed, Decode, ISARanges, AddressGranularity,
↪AddressMinimum, AddressMaximum,
AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource, DescriptorName,
↪TranslationType, TranslationDensity)
```

Arguments

ResourceUsage specifies whether the I/O range is consumed by this device (*ResourceConsumer*) or passed on to child devices (*ResourceProducer*). If nothing is specified, then *ResourceConsumer* is assumed.

IsMinFixed specifies whether the minimum address of this I/O range is fixed (*MinFixed*) or can be changed (*MinNotFixed*). If nothing is specified, then *MinNotFixed* is assumed. The 1-bit field *DescriptorName._MIF* is automatically created to refer to this portion of the resource descriptor, where '1' is *MinFixed* and '0' is *MinNotFixed*.

IsMaxFixed specifies whether the maximum address of this I/O range is fixed (*MaxFixed*) or can be changed (*MaxNotFixed*). If nothing is specified, then *MaxNotFixed* is assumed. The 1-bit field *DescriptorName._MAF* is automatically created to refer to this portion of the resource descriptor, where '1' is *MaxFixed* and '0' is *MaxNotFixed*.

Decode specifies whether or not the device decodes the I/O range using positive (*PosDecode*) or subtractive (*SubDecode*) decode. If nothing is specified, then *PosDecode* is assumed. The 1-bit field *DescriptorName._DEC* is automatically created to refer to this portion of the resource descriptor, where '1' is *SubDecode* and '0' is *PosDecode*.

ISARanges specifies whether the I/O ranges specifies are limited to valid ISA I/O ranges (*ISAOnly*), valid non-ISA I/O ranges (*NonISAOnly*) or encompass the whole range without limitation (*EntireRange*). The 2-bit field *DescriptorName._RNG* is automatically created to refer to this portion of the resource descriptor, where '1' is *NonISAOnly*, '2' is *ISAOnly* and '0' is *EntireRange*.

AddressGranularity evaluates to a 16-bit integer that specifies the power-of-two boundary (-1) on which the I/O range must be aligned. The 16-bit field *DescriptorName._GRA* is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 16-bit integer that specifies the lowest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in *AddressGranularity* is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 16-bit field *DescriptorName._MIN* is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 16-bit integer that specifies the highest possible base address of the I/O range. The value must have '0' in all bits where the corresponding bit in *AddressGranularity* is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 16-bit field *DescriptorName._MAX* is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 16-bit integer that specifies the offset to be added to a secondary bus I/O address which results in the corresponding primary bus I/O address. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 16-bit field *DescriptorName._TRA* is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 16-bit integer that specifies the total number of bytes decoded in the I/O range. The 16-bit field *DescriptorName._LEN* is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by *ResourceSource*. If this argument is specified, the *ResourceSource* argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the *ResourceSourceIndex* argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

TranslationType is an optional argument that specifies whether the resource type on the secondary side of the bus is different (*TypeTranslation*) from that on the primary side of the bus or the same (*TypeStatic*). If *TypeTranslation* is specified, then the primary side of the bus is Memory. If *TypeStatic* is specified, then the primary side of the bus is I/O. If nothing is specified, then *TypeStatic* is assumed. The 1-bit field *DescriptorName._TTP* is automatically created to refer to this portion of the resource descriptor, where '1' is *TypeTranslation* and '0' is *TypeStatic*. See *_TTP* for more information

TranslationDensity is an optional argument that specifies whether or not the translation from the primary to secondary bus is sparse (*SparseTranslation*) or dense (*DenseTranslation*). It is only used when *TranslationType* is *TypeTranslation*. If nothing is specified, then *DenseTranslation* is assumed. The 1-bit field *DescriptorName._TRS* is automatically created to refer to this portion of the resource descriptor, where '1' is *SparseTranslation* and '0' is *DenseTranslation*. See *_TRS* for more information.

Description

The *WordIO* macro evaluates to a buffer that contains a 16-bit I/O range resource descriptor. The format of the 16-bit I/O range resource descriptor can be found in [Section 6.4.3.5.3](#). This macro is designed to be used inside of a *ResourceTemplate* (*Resource To Buffer Conversion Macro*).

19.6.153 WordPCC (WordPCC Resource Descriptor Macro)

Syntax:

```
WordPCC (PccChannel, ResourceSourceIndex, ResourceSource,
DescriptorName)
```

Arguments

PccChannel evaluates to an 8-bit integer that specifies the PCCT Index of the PCC Subspace consumed by this Resource.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by ResourceSource. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a value of zero is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The WordPCC macro evaluates to a buffer that contains a 16-bit Address Space resource descriptor. The format of the 16-bit Address Space resource descriptor can be found in [Table 6.47](#). This macro is designed to be used inside of a *ResourceTemplate* (*Resource To Buffer Conversion Macro*). The PccChannel field is used to populate the Address field in the resultant Resource Buffer.

19.6.154 WordSpace (Word Space Resource Descriptor Macro)

Syntax:

```
WordSpace (ResourceType, ResourceUsage, Decode, IsMinFixed, IsMaxFixed,
↪TypeSpecificFlags, AddressGranularity, AddressMinimum,
AddressMaximum, AddressTranslation, RangeLength, ResourceSourceIndex, ResourceSource,
↪DescriptorName)
```

Arguments

ResourceType evaluates to an 8-bit integer that specifies the type of this resource. Acceptable values are 0xC0 through 0xFF.

ResourceUsage specifies whether the bus range is consumed by this device (ResourceConsumer) or passed on to child devices (ResourceProducer). If nothing is specified, then ResourceConsumer is assumed.

Decode specifies whether or not the device decodes the bus number range using positive (PosDecode) or subtractive (SubDecode) decode. If nothing is specified, then PosDecode is assumed. The 1-bit field DescriptorName. _DEC is automatically created to refer to this portion of the resource descriptor, where '1' is SubDecode and '0' is PosDecode.

IsMinFixed specifies whether the minimum address of this bus number range is fixed (MinFixed) or can be changed (MinNotFixed). If nothing is specified, then MinNotFixed is assumed. The 1-bit field DescriptorName. _MIF is automatically created to refer to this portion of the resource descriptor, where '1' is MinFixed and '0' is MinNotFixed.

IsMaxFixed specifies whether the maximum address of this bus number range is fixed (MaxFixed) or can be changed (MaxNotFixed). If nothing is specified, then MaxNotFixed is assumed. The 1-bit field DescriptorName. _MAF is automatically created to refer to this portion of the resource descriptor, where '1' is MaxFixed and '0' is MaxNotFixed.

TypeSpecificFlags evaluates to an 8-bit integer. The flags are specific to the ResourceType.

AddressGranularity evaluates to a 16-bit integer that specifies the power-of-two boundary (- 1) on which the bus number range must be aligned. The 16-bit field DescriptorName. _GRA is automatically created to refer to this portion of the resource descriptor.

AddressMinimum evaluates to a 16-bit integer that specifies the lowest possible bus number for the bus number range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 16-bit field DescriptorName. _MIN is automatically created to refer to this portion of the resource descriptor.

AddressMaximum evaluates to a 16-bit integer that specifies the highest possible bus number for the bus number range. The value must have '0' in all bits where the corresponding bit in AddressGranularity is '1'. For bridge devices which translate addresses, this is the address on the secondary bus. The 16-bit field DescriptorName. _MAX is automatically created to refer to this portion of the resource descriptor.

AddressTranslation evaluates to a 16-bit integer that specifies the offset to be added to a secondary bus bus number which results in the corresponding primary bus bus number. For all non-bridge devices or bridges which do not perform translation, this must be '0'. The 16-bit field DescriptorName. _TRA is automatically created to refer to this portion of the resource descriptor.

RangeLength evaluates to a 16-bit integer that specifies the total number of bus numbers decoded in the bus number range. The 16-bit field DescriptorName. _LEN is automatically created to refer to this portion of the resource descriptor.

ResourceSourceIndex is an optional argument which evaluates to an 8-bit integer that specifies the resource descriptor within the object specified by ResourceSource. If this argument is specified, the ResourceSource argument must also be specified.

ResourceSource is an optional argument which evaluates to a string containing the path of a device which produces the pool of resources from which this I/O range is allocated. If this argument is specified, but the ResourceSourceIndex argument is not specified, a zero value is assumed.

DescriptorName is an optional argument that specifies a name for an integer constant that will be created in the current scope that contains the offset of this resource descriptor within the current resource template buffer. The predefined descriptor field names may be appended to this name to access individual fields within the descriptor via the Buffer Field operators.

Description

The WordSpace macro evaluates to a buffer that contains a 16-bit Address Space resource descriptor. The format of the 16-bit Address Space resource descriptor can be found in [Section 6.4.3.5.3](#). This macro is designed to be used inside of a *ResourceTemplate (Resource To Buffer Conversion Macro)*.

19.6.155 XOr (Integer Bitwise Xor)

Syntax:

```
XOr ( Source1, Source2, Result ) => Integer
Result = Source1 ^ Source2 => Integer
Result ^= Source => Integer
```

Arguments

Source1 and Source2 are evaluated as Integers.

Description

A bitwise XOR is performed and the result is optionally stored into Result.

19.6.156 Zero (Constant Zero Integer)**Syntax:**

```
Zero => Integer
```

Description

The Zero operator returns an Integer with the value 0. Writes to this object are not allowed. The use of this operator can reduce AML code size, since it is represented by a one-byte AML opcode.

19.6.157 ClockInput (Clock Input Resource Descriptor Macro)**Syntax:**

```
ClockInput (FrequencyNumerator, FrequencyDivisor, Scale, FixedMode, ResourceSource, ↵
↵ResourceSourceIndex)
```

Arguments

FrequencyNumerator is an integer argument describing the numerator of the input frequency of the clock. This argument is required. If *FixedMode* is **Variable** this value describes the initial/default input clock frequency. The unit type for the value in this argument is conveyed in *Scale*. The bit field name `_SPE` is automatically created to refer to this portion of the resource descriptor.

FrequencyDivisor is an integer argument describing the denominator of the input frequency of the clock. If *FixedMode* is **Variable** this value describes the initial/default input clock frequency.

Scale is one of Hz | kHz | MHz. This required argument conveys the unit of the frequency value determined by (*FrequencyNumerator* / *FrequencyDivisor*).

FixedMode is one of Fixed | Variable. **Fixed** denotes that the clock frequency cannot be adjusted. **Variable** denotes that the clock frequency is variable, and is controllable through the Device referenced by *ResourceSource*.

ResourceSource is a string which uniquely identifies the Device which is the source of the clock signal described by this Resource. If *FixedMode* is **Variable**, this argument is required. *ResourceSource* can be a fully-qualified name, a relative name or a name segment that utilizes the namespace search rules.

ResourceSourceIndex is an integer specifying the clock output of the Device specified by *ResourceSource*. Clock output index values begin at 0. If *FixedMode* is **Variable**, this argument is required. If *ResourceSource* is not supplied, this argument is ignored. If the clock source referenced by *ResourceSource* has only one clock output, this argument shall be 0.

Examples

The following examples show the use of the ClockInput macro for two different cases.

Case 1: Simple clock input descriptor, with a fixed frequency of 33 1/3 MHz and no Device shown as a clock source.

```
ClockInput (100, 3, MHz, Fixed,,)
```

Case 2: Clock input descriptor referencing a variable clock input of 19.2 kHz, with a source device and index.

ClockInput (19200, 1, Hz, Variable, "_SB.PCI0.CLK4", 3)